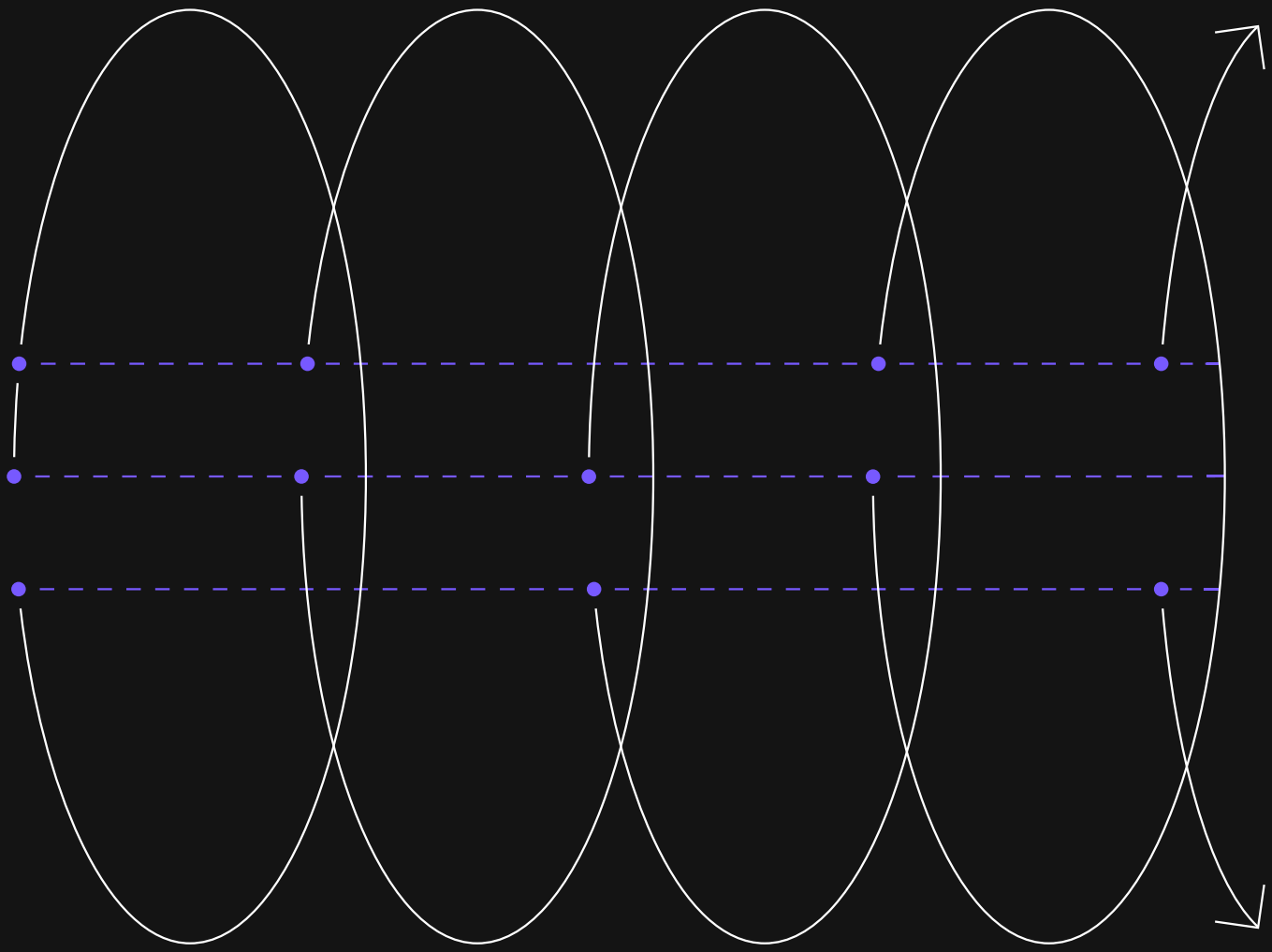
Машинное обучение

Лонгрид

Неделя 9

Градиентный бустинг

ЧАСТЬ 2



Содержание

1. **Частные реализации градиентного бустинга** 4
2. **«Классический алгоритм» XGBoost** 5
   - 2.1. Регуляризация в XGBoost 7
     - 2.1.1. L2-регуляризация 7
     - 2.1.2. Регуляризация на количество листьев в дереве 8
     - 2.1.3. L1-регуляризация 9
     - 2.1.4. Итоги регуляризации XGBoost 10
   - 2.2. Структура деревьев 11
   - 2.3. Как выбрать вершину для сплита 12
   - 2.4. Как оценить качество произвольного сплита 13
   - 2.5. Какие (f, A) выбрать в качестве сплитов 15
     - 2.5.1. Полный перебор 15
     - 2.5.2. Алгоритм Pre-sorted 15
     - 2.5.3. Алгоритм weighted quantile sketch 17
     - 2.5.4. Алгоритм histogram-based — основа «классического алгоритма» 17
   - 2.6. Критерии останова построения дерева 21
   - 2.7. Итоги выбора структуры дерева 23
     - 2.7.1. Как строятся деревья в XGBoost 23
     - 2.7.2. Основные подходы к выбору сплитов 24
     - 2.7.3. Критерии останова построения дерева 24
   - 2.8. DART 24
   - 2.9. Пример использования XGBoost 26
   - 2.10. Итоги XGBoost 28
3. **Библиотека LightGBM — «расширение» XGBoost** 29
   - 3.1. Lossguide-деревья ((leafwise-деревья) 30
     - 3.1.1. Главный недостаток и как с ним бороться 31
     - 3.1.2. Итог Lossguide деревьев 31
   - 3.2. Ускорение построения деревьев 31
     - 3.2.1. GOSS 31
     - 3.2.2. EFB 32
     - 3.2.3. Итоги ускорения построения деревьев 33
   - 3.3. Сплиты для категориальных признаков 33
     - 3.3.1. Итоги построения сплитов по категориальным признакам 35
   - 3.4. Элементы практики 36
     - 3.4.1. Early stopping (критерий раннего останова) 36
     - 3.4.2. Веса на объектах 37
     - 3.4.3. Кастомные метрики и лоссы 37
     - 3.4.4. Деревья в виде датафрейма 38
     - 3.4.5. «Режим» случайного леса 38
     - 3.4.6. Итоги элементов практики 39
   - 3.5. Пример использования LightGBM 39

3.6. Итоги LightGBM 42
4. Библиотека CatBoost: как предотвратить утечку данных в градиентном бустинге 43
4.1. Регуляризация в листьях 44
4.2. Ленивые деревья 45
4.3. Скоринг сплитов — оценка схожести с антиградиентом 45
4.3.1. Оценка качества сплита: Gain vs. антиградиент 46
4.3.2. Зачем сравнивать именно с антиградиентом? 48
4.3.3. Итоги построения деревьев 49
4.4. Prediction Shift 49
4.5. Ordered-философия для борьбы с target leakage 50
4.6. Plain-режим — подсчёт Ordered TS для категорий 51
4.6.1. Роль нулевой перестановки 53
4.7. Ordered-режим 54
4.7.1. Итоги Ordered-философии 56
4.8. Элементы практики 56
4.8.1. Задание уточняющих параметров 56
4.8.2. Генерируемые «на лету» комбинации признаков 57
4.8.3. Деревья в виде датафрейма 57
4.8.4. Встроенные средства интерпретации модели 57
4.8.5. CatBoost Plain vs CatBoost Ordered vs LightGBM 58
4.8.6. Итоги «практики» 60
4.9. Пример использования Catboost 60
4.9.1. Итоги «практики» 62
4.10. Итоги Catboost 63
5. SketchBoost — градиентный бустинг для multiclass-/multilabel-/multiregression-задач 63
5.1. Типы multitarget-задач 64
5.2. Как решают эту задачу другие библиотеки 65
5.3. Multivariate Trees — общий подход 66
5.3.1. Итог Multivariate Trees 68
5.4. Аппроксимация матриц 68
5.4.1. Random Sampling 68
5.4.2. Random Projection 69
5.4.3. Итоги аппроксимации 69
5.5. Пример использования PyBoost 70
5.6. Итоги Sketchboost 71
6. Yggdrasil Decision Forests (YDF): унифицированный фреймворк от Google 72
6.1. Пример использования YDF 74
6.2. Итоги YDF 76
7. Сравнение библиотек 77
Итоги 78

Градиентный бустинг (часть 2)

Контекст

В неделе 8 мы разобрали общую идею градиентного бустинга над решающими деревьями: как ансамбль последовательно исправляет ошибки, почему опирается на антиградиент функции потерь и как значение листа получают ньютоновским шагом по первой и второй производным. Оттуда мы забираем обозначения g_i, h_i, R_j и w_j — ими эта часть пользуется с первых формул. Теперь мы переходим от теории к инструментам — к промышленным библиотекам, которые реализуют эти принципы по-разному. В этой части мы изучим XGBoost, LightGBM, CatBoost, SketchBoost и Yggdrasil Decision Forests. У каждой из них свой взгляд на построение деревьев, обработку признаков, регуляризацию и борьбу с переобучением.

Применение

Понимание различий между реализациями градиентного бустинга — ключ к выбору правильного инструмента под конкретную задачу. Одна библиотека может быть быстрее на огромных выборках, другая — аккуратнее работать с категориями, третья — эффективнее решать multitarget-задачи. Современные библиотеки умеют работать с табличными данными практически любого размера, поддерживают тексты, эмбеддинги и автоматическую генерацию признаков. Зная их особенности, ты сможешь не только выжимать максимум качества, но и разумно расходовать вычислительные ресурсы.

Краткий план

- → Частные реализации градиентного бустинга
- → «Классический алгоритм» XGBoost
- → Библиотека LightGBM — «расширение» XGBoost
- → Библиотека CatBoost: как предотвратить утечку данных в градиентном бустинге
- → SketchBoost — градиентный бустинг для multiclass-/multilabel-/multiregression-задач
- → Yggdrasil Decision Forests (YDF): унифицированный фреймворк от Google
- → Сравнение библиотек
- → Итоги

1. Частные реализации градиентного бустинга

На первый взгляд может показаться, что достаточно один раз реализовать алгоритм и все будут пользоваться им одинаково. Но на практике разные библиотеки градиентного бустинга делают очень разные выборы: как строить структуру деревьев, как искать оптимальные сплиты, как работать с пропусками и категориальными признаками, как бороться с переобучением. Эти решения сильно влияют на скорость обучения, объём потребляемой памяти и, конечно, на итоговое качество модели. Поэтому инженеру или исследователю важно понимать, чем отличаются основные инструменты и когда какой из них стоит выбрать.

В этом лонгриде мы познакомимся с ключевыми промышленными реализациями градиентного бустинга: **XGBoost**, **LightGBM**, **CatBoost**, а также с более специализированными **SketchBoost** и **Yggdrasil Decision Forests**. Мы пойдём от «классического» алгоритма, заложенного в XGBoost, к улучшениям и альтернативным подходам, появившимся позже в следующем порядке.

- **XGBoost** — первая по-настоящему масштабируемая библиотека, в которой сформировался «классический» гистограммный алгоритм построения деревьев (depthwise-рост, Gain-скоринг, биннинг признаков). Здесь мы разберём базовые принципы, общие для многих реализаций.
- **LightGBM** развивает идеи XGBoost, добавляя приёмы ускорения — градиентный семплинг объектов (GOSS), объединение взаимоисключающих признаков (EFB), а также «листовой» (lossguide) рост деревьев. В LightGBM впервые появилась эффективная работа с категориальными признаками прямо «из коробки».
- **CatBoost** предлагает альтернативный взгляд: борьба с переобучением через ordered-философию и «ленивые» (oblivious) деревья. Особый упор сделан на «честное» кодирование категориальных признаков без утечки целевой переменной. CatBoost также умеет автоматически генерировать комбинации признаков и работать с текстами и эмбеддингами.
- **SketchBoost** (реализован в библиотеке pyboost) — специализированное решение для multitarget-задач (multilabel, multiregression). Он обобщает бустинг на многомерные предсказания и ускоряет вычисления с помощью скетчинга — аппроксимации матрицы градиентов матрицей меньшей размерности.
- **Yggdrasil Decision Forests** — современный фреймворк от Google, предоставляющий единый интерфейс для обучения как случайных лесов, так и градиентного бустинга. Мы добавили его в этот лонгрид, чтобы познакомить вас с одним из самых новых инструментов; его особенности и отличия от остальных библиотек мы разберём в соответствующем разделе.

Каждый из инструментов мы рассмотрим с точки зрения алгоритмических решений, важных гиперпараметров и практических рекомендаций.

2. «Классический алгоритм» XGBoost

Полезные материалы

- Оригинальная статья XGBoost: A Scalable Tree Boosting System
- Официальный репозиторий проекта на Github

XGBoost — первая крупная библиотека градиентного бустинга, которая сфокусировалась на том, как применить алгоритм обучения к большим объёмам данных. Авторы сделали фундаментальную, алгоритмически и аппаратно оптимизированную реализацию, от которой позже отталкивались другие известные библиотеки.

Изначально функциональность XGBoost была сфокусирована на скорости и масштабируемости, но со временем она значительно расширилась. Библиотека не только развивала собственные оптимизации, но и быстро интегрировала новые идеи из исследований. Например, уже в 2016 году в библиотеке появилась реализация DART (Dropout Additive Regression Trees), которая применяет идею dropout из глубокого обучения на уровне деревьев для борьбы с переобучением.

Прежде чем разбирать доработки, вспомним, с чем мы пришли из предыдущей части.

Классический градиентный бустинг строит ансамбль последовательно: на каждой итерации для всех объектов считается антиградиент функции потерь $s_i = -g_i$, на него по квадратичной ошибке обучается обычное регрессионное дерево, а в каждый лист кладётся поправка. Значение листа — ньютоновский шаг $w_j = -G_j/H_j$, где G_j и H_j — суммы первых и вторых производных функции потерь по объектам листа R_j.

$$g_i = \frac{\partial L(y_i, z)}{\partial z}\Big|_{z=\hat{y}_i}, \qquad h_i = \frac{\partial^2 L(y_i, z)}{\partial z^2}\Big|_{z=\hat{y}_i}, \qquad G_j = \sum_{i \in R_j} g_i, \qquad H_j = \sum_{i \in R_j} h_i$$

Ключевая деталь, от которой мы будем отталкиваться дальше: **в классической схеме вторая производная участвует ровно в одном месте — в значении листа.** Структуру дерева искал обычный регрессионный CART по квадратичной ошибке, и этот критерий ничего не знал о функции потерь ансамбля: он мерил только то, насколько дерево близко к вектору антиградиентов.

XGBoost меняет именно это. Ниже — что переходит без изменений, а что перестраивается.

	Классический градиентный бустинг	**XGBoost**
Что считается для каждого объекта	Антиградиент $s_i = -g_i$	g_i и h_i
Как выбирается сплит	Регрессионный CART по квадратичной ошибке на s_i	Gain, выведенный из разложения функции потерь
Что кладётся в лист	$w_j = -G_j/H_j$	$w_j = -G_j/(H_j + \lambda)$
Где используется вторая производная	Только в значении листа	И в значении листа, и в выборе сплита
Чем ограничены веса листьев	Темпом обучения η	η, а также λ и γ

XGBoost сохраняет общую схему классического градиентного бустинга, но дорабатывает несколько её важных частей:

- большие веса листьев и сложные деревья могут приводить к переобучению, поэтому в функцию потерь добавляются L1- и L2-регуляризация, а также штраф за усложнение структуры;
- для выбора сплита нужен критерий, учитывающий текущую ошибку всего ансамбля, поэтому в XGBoost используется Gain, выведенный из функции потерь бустинга;
- точный перебор всех признаков и порогов слишком дорог на больших данных, поэтому применяются приближённый поиск порогов и гистограммный подход;
- для дополнительной борьбы с переобучением используются семплирование объектов и признаков, а также DART.

Дальше мы последовательно разберём, как регуляризация меняет веса листьев, как из функции потерь получается Gain, как с его помощью строится дерево и как XGBoost ускоряет поиск сплитов.

Начнём с регуляризации. Для этого сначала кратко вспомним, как в классическом градиентном бустинге находилось значение листа (Неделя 8, раздел «Значение листа: ньютоновский шаг»).

1. Функция потерь для каждого объекта (всего их N) раскладывается до второго порядка в ряд Тейлора;
2. Вычисляется производная по весу листа;
3. Производная приравнивается к нулю;
4. Получаем формулу оптимального веса.

$$L^{(k)*} = \min_{\vec{w}^{(k)}} \sum_i \Big[\underbrace{L_i^{(k-1)}}_{\text{const}} + g_i^{(k-1)} w_{q_k(x_i)}^{(k)} + \frac{1}{2} h_i^{(k-1)} \Big(w_{q_k(x_i)}^{(k)} \Big)^2 \Big]$$

$$\approx \min_{\vec{w}^{(k)}} \sum_{R_j} \Big[\Big(\sum_{i \in R_j} g_i^{(k-1)} \Big) w_j^{(k)} + \frac{1}{2} \Big(\sum_{i \in R_j} h_i^{(k-1)} \Big) \big(w_j^{(k)} \big)^2 \Big] \quad \leftarrow \text{возьмем производную}$$

$$\nabla_j = \Big(\sum_{i \in R_j} g_i^{(k-1)} \Big) + \Big(\sum_{i \in R_j} h_i^{(k-1)} \Big) w_j^{(k)} = 0 \quad \rightarrow \quad w_j^{(k)*} = - \frac{\sum_{i \in R_j} g_i^{(k-1)}}{\sum_{i \in R_j} h_i^{(k-1)}}$$

оптимальный вес на j-м листе

В классической постановке величина веса листа и сложность дерева напрямую не ограничиваются. Поэтому сначала посмотрим, какие регуляризаторы добавляет XGBoost и как они меняют уже знакомую формулу оптимального веса.

2.1. РЕГУЛЯРИЗАЦИЯ В XGBOOST

2.1.1. L2-регуляризация

В неделе 8 мы уже видели крайний случай такого поведения: если все объекты листа имеют одинаковую метку, у одномерной задачи вообще нет конечного решения, и формула возвращает конечное число лишь потому, что квадратичное приближение перестаёт описывать потери вдали от текущей точки.

Если внимательно рассмотреть итоговую формулу весов, можно заметить, что в ней нет никакого контроля на количество объектов, попавших в лист. Например, в левый лист попал 1 объект $g_i = 5$ и $h_i = 0.25$, следовательно $w_L = -\frac{5}{0.25} = -20$. А в правый лист попало 3 объекта с идентичными значениями. Тогда $w_R = -\frac{5+5+5}{0.25+0.25+0.25} = -20$.

То есть получается, что величина сдвига предсказания одинаковая для двух разных листьев, причём значения для левого листа получены по меньшему количеству объектов, то есть менее стабильны. Нам некоторым образом необходимо штрафовать такие неуверенные предсказания и занижать для них сдвиги, чтобы не переобучаться.

Авторами XGBoost было предложено использовать L2-регуляризацию на предсказанные веса. То есть алгоритму становится невыгодно делать новый лист с большим весом, если в него попадает малое количество объектов, так как для одного объекта лосс снизится сильно, но мы получим большой штраф за это. Гораздо выгоднее сделать большой вес для листа, где много объектов, чтобы сильно снизить лосс с превышением величины штрафа за это.

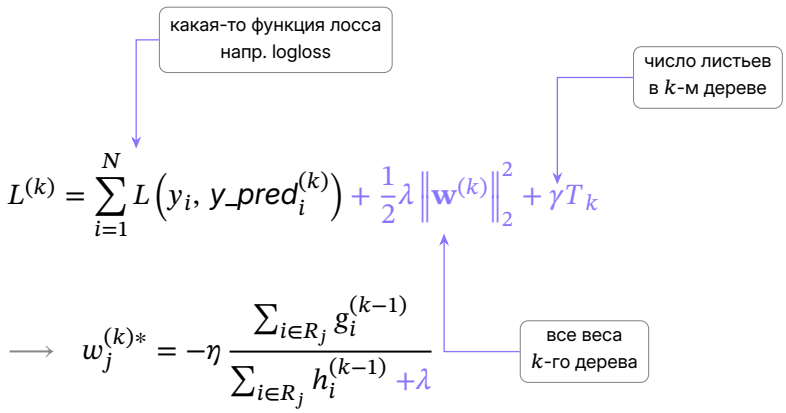
В итоге получаем следующий вид функции потерь и оптимальных весов (выводятся аналогично как для классической функции потерь).



Таким образом, L2 сглаживает предсказания, делая их более консервативными, уменьшает влияние шумных объектов на предсказание.

2.1.2. Регуляризация на количество листьев в дереве

Можно заметить, что данная функция потерь никак не контролирует глубину дерева. То есть мы можем только жёстко ограничить глубину дерева гиперпараметром, но хотелось бы более мягко: штрафовать модель за каждый новый сплит. Таким образом, при выборе нового разбиения значение функции потерь для объектов в листьях может снижаться, но если это снижение меньше штрафного коэффициента, то данное разбиение просто невыгодно. Таким образом, добавим к функции потерь ещё один штрафной коэффициент на глубину дерева, что ограничит его возможности по переобучению.



Как можно заметить, формула оптимальных весов не поменялось. Она и не должна была поменяться, ведь γT не зависит от w, соответственно, и при вычислении градиента, данное слагаемое просто обнулится. Тем не менее данная константа влияет на выбор оптимального сплита, о котором мы поговорим в разделе ниже.

Данный вид функции потерь и весов для XGBoost и LightGBM считается классическим, в последующих выводах формул для простоты мы будем работать именно с ним.

2.1.3. L1 - регуляризация

Данного типа регуляризации не было в оригинальной реализации XGBoost, она появилась только со временем, поэтому в формуле функции потерь соответствующий штраф отображён серым.

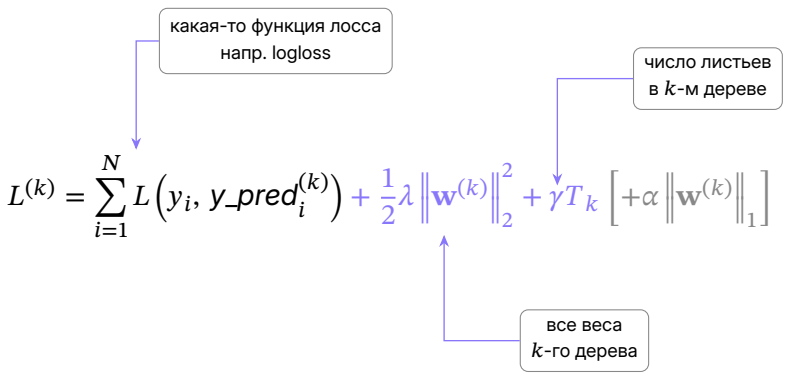
Не секрет, что L2 делает веса маленькими, но никогда не обнуляет их полностью. Даже если сумма градиентов в листе близка к нулю (лист почти не даёт полезного сигнала), вес всё равно будет ненулевым — просто очень маленьким.

Соответственно, в высокоразмерной задаче при использовании глубоких деревьев может возникать ситуация, когда многие листья будут содержать малое количество объектов с противоречивыми градиентами (положительные и отрицательные компенсируют друг друга). L2 сделает их веса крошечными (например, 0.001), но они всё равно будут вносить вклад в итоговое предсказание.

Это создаёт две проблемы.

1. **Накопление шума:** тысячи крошечных вкладов от слабых листьев могут суммироваться и давать заметный эффект, особенно на шумных данных.
2. **Разреженность модели:** в задачах с высокой размерностью хочется, чтобы модель была разреженной — то есть большинство весов были строго нулевыми, а ненулевые веса соответствовали действительно важным паттернам. Это ускоряет инференс и улучшает интерпретируемость.

Чтобы получить **строгий ноль** для слабых сигналов, в XGBoost добавили L1-регуляризацию с коэффициентом α.



Формула весов ищется аналогично, приравнивая градиент по весам к нулю. Производная модуля не зависит от величины веса, поэтому при малых градиентах L1 полностью обнуляет вес, а не просто делает его маленьким. Итоговая формула веса выглядит следующим образом.

$$w_j^{(k)*} = -\eta \frac{sign(\sum_{i \in R_j} g_i^{(k-1)}) \cdot \max(0, \| \sum_{i \in R_j} g_i^{(k-1)} \| - \alpha)}{\sum_{i \in R_j} h_i^{(k-1)} + \lambda}$$

Что это даёт на практике?

1. **Борьба с шумными микросплитами.** Если в лист попало мало объектов или градиенты противоречивы (сумма близка к нулю), L1 принудительно зануляет вес листа. Модель «игнорирует» этот лист, оставляя предсказание на уровне предыдущих деревьев, вместо того чтобы делать крошечный шумный шаг.

2. **Косвенный отбор признаков.** Хотя L1 в бустинге не обнуляет признаки напрямую (как в Lasso), она делает это косвенно: если признак полезен только для слабых сплитов, веса листьев на его основе будут занулены, и признак перестанет вносить вклад в предсказания.
3. **Разреженность модели.** Дерево с множеством нулевых весов по сути содержит «мёртвые ветки». В оптимизированных движках инференса это позволяет пропускать вычисления для целых веток, ускоряя предсказания.

Несмотря на теоретическую пользу, эффект L1 в градиентном бустинге **ограничен** по двум причинам.

1. **Ансамбль нивелирует разреженность.** В линейной модели Lasso обнуление веса признака окончательно удаляет его из модели. В бустинге, если вес листа в дереве №5 занулился из-за L1, дерево №6 может снова попытаться исправить эту ошибку и сделать сплит по тому же признаку. Поэтому «отбор признаков» через L1 работает гораздо слабее, чем в линейных моделях.
2. γ **и** `min_child_weight` (минимальная сумма гессианов в листе) делают то же самое, но понятнее. Эти параметры предотвращают создание слабых листьев на этапе выбора сплитов, а не постфактум. Инженеры почти всегда предпочитают настраивать их, потому что они интуитивнее и напрямую контролируют структуру дерева.

Когда L1 всё же полезна? В задачах с **крайне зашумленными данными** и **экстремально высокой размерностью**, где нужно дополнительно подавить влияние слабых сигналов, которые не отсекаются через γ.

2.1.4. Итоги регуляризации XGBoost

В XGBoost используется три взаимодополняющих механизма регуляризации, каждый из которых решает свою проблему.

Параметр	Что штрафует	Как влияет на модель	Когда особенно полезен
λ **(L2)**	Величину весов листьев	Делает предсказания консервативными, снижает влияние шумных объектов и листьев с малым числом объектов	Почти всегда — базовая защита от переобучения
γ	Количество листьев в дереве	Мягко ограничивает глубину: сплит делается только если выигрыш в лоссе превышает штраф	Когда нужно контролировать сложность дерева без жёсткого ограничения `max_depth`
α **(L1)**	Модуль весов листьев	Обнуляет веса слабых листьев, создаёт разреженную модель	Высокоразмерные зашумленные задачи, где нужно подавить микросплиты

Как они работают вместе

- **L2** не даёт весам расти слишком большими, но никогда не обнуляет их полностью.
- γ контролирует структуру дерева — не даёт алгоритму создавать бесполезные сплиты.

- **L1** дополняет L2, превращая крошечные веса в строгий ноль, что полезно для разреженности и ускорения инференса.

2.2. СТРУКТУРА ДЕРЕВЬЕВ

Теперь определим, каким образов в градиентном бустинге производится построение деревьев.

Формально структуру деревьев в XGBoost можно описать так.

- Чтобы построить дерево, необходимо посчитать для всех объектов g_i и h_i на основе предсказаний уже построенного ансамбля (если это первое дерево, то по начальному приближению).
- Построенное дерево бинарное и строится последовательно от корня вниз и вширь до выполнения любого критерия останова.
- Пока не выполнен ни один критерий останова, выбираются:
  - вершина, в которой необходимо сделать разбиение;
  - «оптимальный сплит» для неё — предикат вида $f^* \leq A^*$, где f^*, A^* — это признак и число, которые дают «наилучший сплит» (то есть идет сравнение некоторого j-го вектора признака и числа).

  ЗАМЕЧАНИЕ Поддержки категориальных признаков изначально не было. Мы поговорим про категориальные сплиты в блоке про LightGBM.

- Объекты, удовлетворяющие предикату, направляются в левую ветку, остальные — в правую, пропуски — в «лучшую» ветвь (алгоритм сам определяет, в какую сторону выгоднее направлять объекты с пропусками).
- Если выполнен критерий останова, построение дерева завершается: все вершины без сплитов объявляются листьями, для них вычисляются оптимальные веса по формуле выше с помощью уже посчитанных g_i и h_i.

То есть фактически строится обычное бинарное дерево, но оно предсказывает не итоговый ответ, а **поправки** (веса в листьях), которые добавляются к текущему предсказанию ансамбля. Эти поправки вычисляются через градиенты и гессианы функции потерь так, чтобы минимизировать ошибку и приблизить предсказание к правильному ответу.

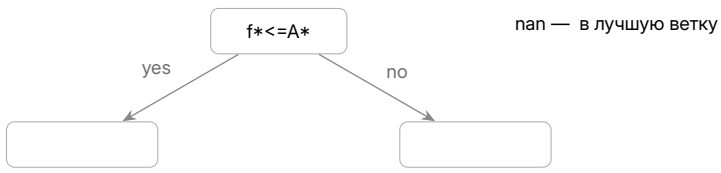


Рис. 1. Сплит в листе

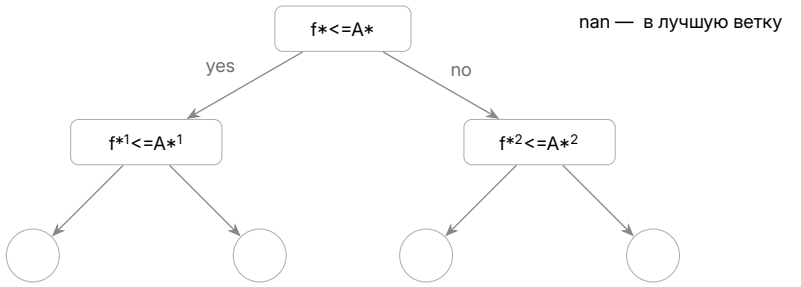


Рис. 2. Продолжаем сплитить на следующем уровне

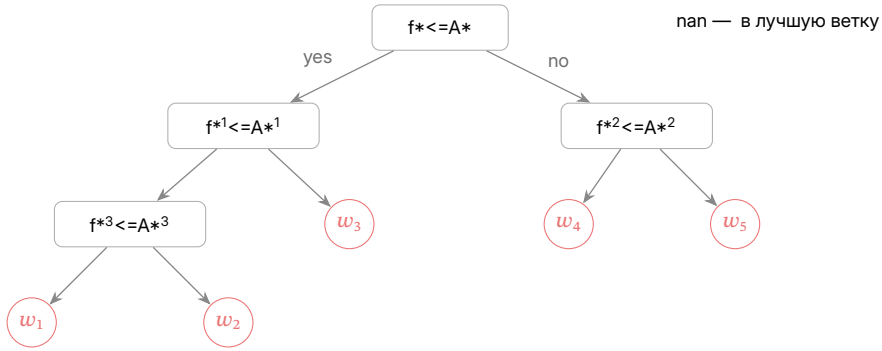


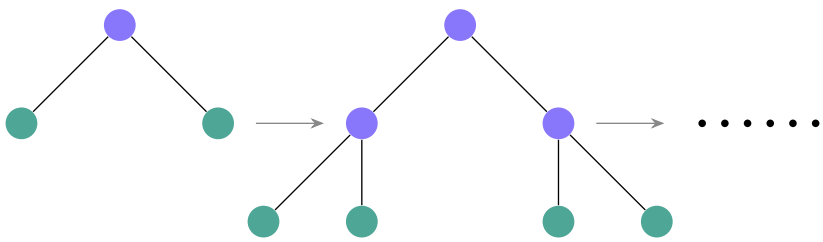
Рис. 3. После ещё одного сплита на следующем уровне заканчиваем построение

Дальше мы разберём:

- как выбрать вершину, в которой будем делать сплит;
- как выбираются оптимальные сплиты;
- когда заканчивать строить одно конкретное дерево и ансамбль;
- как сэкономить на операциях без потери в качестве.

2.3. КАК ВЫБРАТЬ ВЕРШИНУ ДЛЯ СПЛИТА

Исторически в XGBoost сплиты совершались последовательно на всех вершинах каждого уровня глубины, постепенно продвигаясь «вниз» (levelwise/depthwise-подход).



Level-wise tree growth

Рис. 4. Источник: https://bangdasun.github.io/2019/03/21/38-practical-comparison-xgboost-lightgbm/

То есть:

- дерево растёт по уровням;

- на каждом уровне мы рассматриваем все вершины этого уровня;
- для каждой вершины на уровне ищем свой оптимальный сплит;
- после того как нашли сплиты для всех вершин уровня, делаем их все сразу.
- переходим к следующему уровню.

2.4. КАК ОЦЕНИТЬ КАЧЕСТВО ПРОИЗВОЛЬНОГО СПЛИТА

Рассмотрим для примера алгоритм оценки сплита $f \leq A$. Он нужен, чтобы выбрать сплит с максимальным скором. Скорером здесь будет Gain — разница функции потерь до и после разбиения.

ЗАМЕЧАНИЕ

Для вычисления Gain мы сравниваем значение функции потерь до и после сплита, что очень похоже на построение расщепления в дереве решений. По сути это оно и есть, только в качестве меры неоднородности используется разложенная в ряд Тейлора функция потерь. **В классическом бустинге в этом месте стояла квадратичная ошибка между предсказанием дерева и антиградиентом — мера, которая о самой функции потерь ничего не знала.**

На рисунке ниже уже построена какая-то часть дерева, и мы хотим сделать разбиение в очередной вершине.

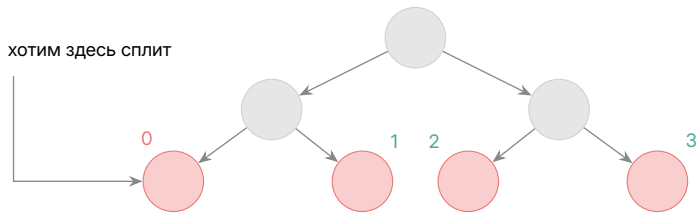


Рис. 5. Выбираем вершину, где строим сплит

На рисунке серым обозначены вершины, в которых уже есть сплиты, а оранжевым — где их ещё нет. В одной из них мы и выберем сплит — для каждой предложенной пары (f, A) посчитаем Gain — разницу в лоссах до и после разбиения. Чтобы её посчитать, нужно снова выразить лосс через его первую и вторую производные с помощью разложения в ряд Тейлора.

Подставив в полный лосс, разложенный по Тейлору, формулу для оптимальных весов при выбранном разбиении, мы получим следующее.

$$L^{(k)*} \approx -\frac{1}{2}\eta \sum_{j=1}^{T_k} \frac{\left(\sum_{i \in R_j} g_i^{(k-1)}\right)^2}{\left(\sum_{i \in R_j} h_i^{(k-1)}\right) + \lambda} + \gamma T_k$$

ибо Тейлор

лосс в k-м дереве, в котором T листьев

Здесь T_k — количество листьев в дереве «на момент выбора сплита» (разберём это подробнее позже).

Чтобы оценить качество разбиения, нам нужно посчитать Gain — разницу лоссов до и после разбиения.

До разбиения вершина, в которой мы хотим выбрать сплит, была листом (соответственно, у нас 4 листа в дереве). Рассчитаем лосс до разбиения (для удобства специально выделили в отдельное слагаемое вершину R_0, по которой будем делать сплит). Оранжевым выделено слагаемое функции потерь для листа R_0, синим — для остальных листьев.

$$L_{\text{до}} \approx -\frac{1}{2} \cdot \frac{\left(\sum_{i \in R_0} g_i\right)^2}{\sum_{i \in R_0} h_i + \lambda} - \frac{1}{2} \sum_{j=1}^{3} \frac{\left(\sum_{i \in R_j} g_i\right)^2}{\sum_{i \in R_j} h_i + \lambda} + \gamma \cdot T_k$$

Если применить сплит (f, A) для разбиения в вершине, то она перестанет быть листом. Зато две новые вершины, которые получились после сплита, временно станут листьями.

Это отразится на количестве листьев в дереве ($Tk \to Tk + 1$) и на формуле лосса после разбиения (R_L — левый лист, R_R — правый лист). Фиолетовым выделен лосс для новых листьев R_L и R_R.

$$L_{\text{после}} \approx -\frac{1}{2} \left[\frac{\left(\sum_{R_L} g_j\right)^2}{\sum_{R_L} h_i + \lambda} + \frac{\left(\sum_{R_R} g_j\right)^2}{\sum_{R_R} h_i + \lambda} \right] - \frac{1}{2} \sum_{j=1}^{3} \frac{\left(\sum_{i \in R_j} g_i\right)^2}{\sum_{i \in R_j} h_i + \lambda} + \gamma \cdot (T_k + 1)$$

Запишем формулу для Gain.

$$Gain = L_{\text{до}} - L_{\text{после}}.$$

$$\text{Gain} = \frac{1}{2} \Bigg[\underbrace{-\frac{\left(\sum_{i \in R_{\text{head}}} g_i\right)^2}{\left(\sum_{i \in R_{\text{head}}} h_i\right) + \lambda}}_{const\, \forall f, A} + \frac{\left(\sum_{i \in R_L} g_i\right)^2}{\left(\sum_{i \in R_L} h_i\right) + \lambda} + \frac{\left(\sum_{i \in R_R} g_i\right)^2}{\left(\sum_{i \in R_R} h_i\right) + \lambda} \Bigg] \underbrace{- \gamma}_{const}$$

1-ое слагаемое (родительская вершина, она же R_0) не изменяется для любого сплита, поэтому в целях оптимизации может быть вычислено только один раз. Максимизация будет вестись по всем предлагаемым сплитам (f, A) только для 2-ого и 3-его слагаемого.

Важно отметить, что, если максимальный Gain < 0:

- сплит мы делать не будем, и наша вершина остаётся листом;
- лосс после сплита становится больше, то есть любой сплит только увеличивает ошибку.

Здесь более явно становится видна роль параметра γ — это минимальный «plain Gain» (разница лоссов без учёта γ) для сплита. Обычно γ оставляют равным нулю (это значение по умолчанию), так как иначе данная константа будет жёстко ограничивать сильное приближение к оптимуму (переобучение), но, если данных много, на них просто невозможно переобучиться, тем не менее жёсткое ограничение сохранится. Тем не менее иногда может быть полезен для ограничения слабых сплитов. Подобрать можно только экспериментально.

2.5. КАКИЕ (f, A) ВЫБРАТЬ В КАЧЕСТВЕ СПЛИТОВ

По своей сути построение дерева — это операции «предложить сплит» и «проскорить сплит», повторённые много раз. Всё остальное занимает значительно меньше времени. Нам критически важно, чтобы алгоритм работал как можно быстрее (на обучении много последовательных деревьев), поэтому разберём этот момент подробнее.

ЗАМЕЧАНИЕ В этом разделе мы будем иногда работать с O-нотацией достаточно вольно, чтобы было проще разобраться в теме.

Посмотрим, сколько времени и памяти будут занимать у нас различные подходы к операциям «предложить сплит» и «проскорить сплит».

2.5.1. Полный перебор

Самый очевидный способ предлагать сплиты для одной конкретной вершины — это полный перебор. В качестве порогов для каждого числового признака предлагаются все его уникальные значения (в порядке появления в выборке), после чего для каждой пары (f, A) считается Gain.

1. naive

for f :
  for A_f :
    calcGain $\longrightarrow O(n_{leaf})$

speed : $O(Fn_{leaf}) \times$ #nodes

mem : $O(1)$

Рис. 6. Алгоритм полного перебора порогов

Если количество признаков равно F, а N_{leaf} — количество объектов, попавших в вершину, которую мы хотим разбить сплитом, то в худшем случае, когда у признака все значения в выборке уникальные, Gain мы будем считать $O(F \cdot N_{leaf})$ раз, затрачивая на каждый $O(N_{leaf})$ операций.

Итого для каждой вершины, в которой нужно найти оптимальный сплит, требуется:

- $O(F \cdot N_{leaf}^2)$ по скорости;
- $O(1)$ по дополнительной памяти.

2.5.2. Алгоритм Pre-sorted

Pre-sorted — модификация рассмотренного выше алгоритма. Перед тем как построить ансамбль, отсортируем и сохраним каждый признак.

Это требует:

- $O(F \cdot N \cdot logN)$ по скорости (сортируем только один раз в начале);

- $O(F \cdot N)$ по памяти для хранения отсортированной выборки, что довольно «дорого», если данных много.

Зато мы сможем ускориться.

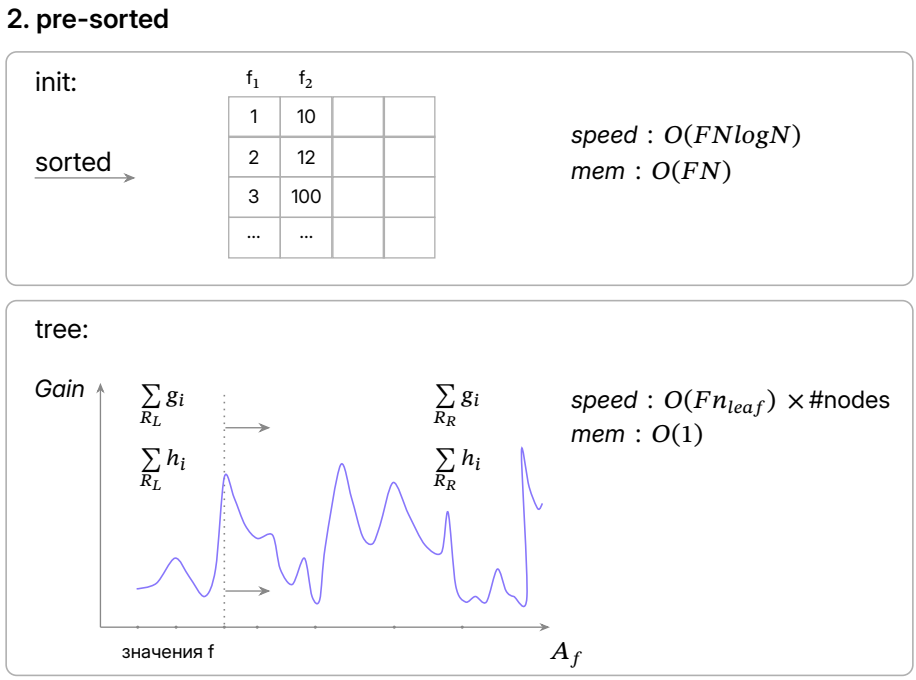


Рис. 7. Pre-sorted алгоритм

Теперь будем поддерживать $sum(g_i), sum(h_i)$ для двух групп: объектов, которые попадают в правую и левую ветки после сплита (f, A). Для конкретного признака f посчитать их нужно один раз — для самого маленького значения A признака f. Это требует $O(N_{leaf})$. Зная для конкретного сплита (f, A) эти 4 суммы, можно посчитать Gain за $O(1)$:

$$Gain = \frac{sum_g rad_L^2}{sum_h ess_L + \lambda} + \frac{sum_g rad_R^2}{sum_h ess_R + \lambda}.$$

Далее нужно перебрать значения признака f в отсортированном порядке (это легко, так как теперь мы их храним), что даст нам преимущество: переходя от сплита $f \le A$ к сплиту $f \le B$, где $B > A$, мы точно будем знать индексы объектов, которые после второго сплита «поменяют ветку».

В первом случае они бы пошли в правую ветку, а во втором — в левую, тогда:

- за $O(n_migrated)$ можно вычесть их g_i, h_i из левых сумм;
- прибавить к правым;
- рассчитать Gain за $O(1)$.

Так для каждого признака мы:

- один раз считаем $sum(g_i)_L, sum(h_i)_L, sum(g_i)_R, sum(h_i)_R$ для минимального A — $O(N_{leaf})$;
- для каждого порога A:
  - вычитаем из левых $sum(g_i), sum(h_i)$ значения поменявших ветку объектов с $O(n_migrated)$,
  - считаем Gain по изменённым суммам — $O(1)$.

Сумма *n_migrated* по всем порогам приблизительно равна N_{leaf}, потому что при переходе от минимального порога к максимальному почти все объекты поменяют свои ветки, за исключением самых левых и самых правых. Gain мы тоже считаем $O(N_{leaf})$ раз, так как у нас в худшем случае столько перебираемых порогов.

Итого:

- по скорости $O(F \cdot N_{leaf})$ для каждой вершины + $O(F \cdot N \cdot logN)$ один раз;
- $O(F \cdot N)$ по дополнительной памяти — довольно много на больших данных.

ВАЖНО

Если есть отсортированные значения порогов, которые мы хотим предлагать, то мы сможем один раз посчитать левые и правые суммы за $O(N_{leaf})$ для каждого признака, зато потом посчитать Gain за $O(1)$ для каждого порога A.

2.5.3. Алгоритм weighted quantile sketch

Этот подход мы не будем рассматривать. Он редко применяется на практике и не важен для последовательного рассказа об эволюции классического алгоритма. Почитать про weighted quantile sketch можно в оригинальной статье XGBoost.

Если кратко, данный алгоритм очень похож на histogram-based подход, описанный ниже, но пороги бинов (что за бины — будет понятно ниже) выбираются адаптивно на основе весов объектов (Гессианов). Различия в качестве со следующим подходом на практике зачастую отсутствуют, но данный подход вычислительно немного сложнее.

2.5.4. Алгоритм histogram-based — основа «классического алгоритма»

Про этот алгоритм не написано в оригинальной статье XGBoost, но, судя по статье LightGBM, он уже был реализован в XGBoost на момент выхода LightGBM, поэтому расскажем о нём здесь.

Он улучшает работу pre-sorted-алгоритма по времени и памяти.

Рассмотрим histogram-based в работе.

1. Снова отсортируем все признаки, но теперь объединим их значения в гистограммы (разобьём на B бинов так, чтобы в каждом находилось примерно равное количество объектов).

Histogram-based

init:

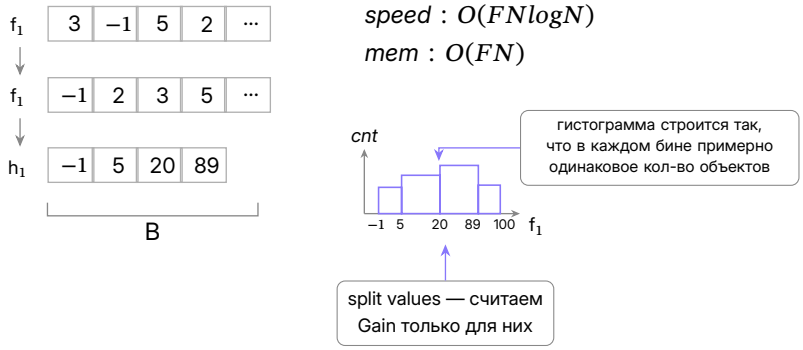


Рис. 8. Histogram-based алгоритм

2. Сохраним только эти гистограммы (отсортированные) вместо всех отсортированных значений признаков. Гистограмма — это массив из $B \ll N$ чисел (обычно $B = 255$) — левые границы бинов, поэтому это требует $O(F \cdot B)$ по памяти. Получить такие гистограммы асимптотически не сложнее, чем отсортировать каждый признак (см., например, код LightGBM, строка 144 и дальше), так что это требует $O(F \cdot N \cdot logN)$ по времени и делается один раз.
3. Будем использовать значения бинов b в качестве предлагаемых порогов для сплитов.

> Если на этом остановиться, мы получим следующие оценки (по сравнению с pre-sorted-алгоритмом).
>
> - $O(F \cdot N_{leaf}) \to O(F \cdot N_{leaf})$ по скорости для каждой вершины;
> - $O(F \cdot N) \to O(F \cdot B)$ по дополнительной памяти.
>
> Порогов стало меньше, однако, как и в случае pre-sorted алгоритма, количество объектов *n_migrated*, которые поменяют свои ветки, всё равно остаётся $O(N_{leaf})$, поэтому асимптотическая оценка скорости не поменяется. Зато уже снизилось потребление памяти.
>
> Но на этом мы не остановимся!

4. Добьёмся ускорения.
Рассмотрим процесс построения дерева от корня, в котором мы ещё не выбрали сплит.
Перед этим посчитаем для нашего корня «гистограммы» (их нужно как-то отличать от гистограмм, которые мы считали в начале, поэтому в этом лонгриде будем записывать их в кавычках).
Для каждого признака для каждого бина (см. пункт 1) посчитаем $sum(g_i), sum(h_i)$ всех объектов, которые попали в этот бин. Иными словами, наши новые «гистограммы» для каждого признака — это два массива длины B с элементами — суммами g_i, h_i, соответствующих бинам объектов.
То есть ещё до построения самого дерева мы заранее считаем все градиенты и гессианы в каждом бине нашей гистограммы для исходного признака. Получаем «гистограммы» для градиентов и гессианов.

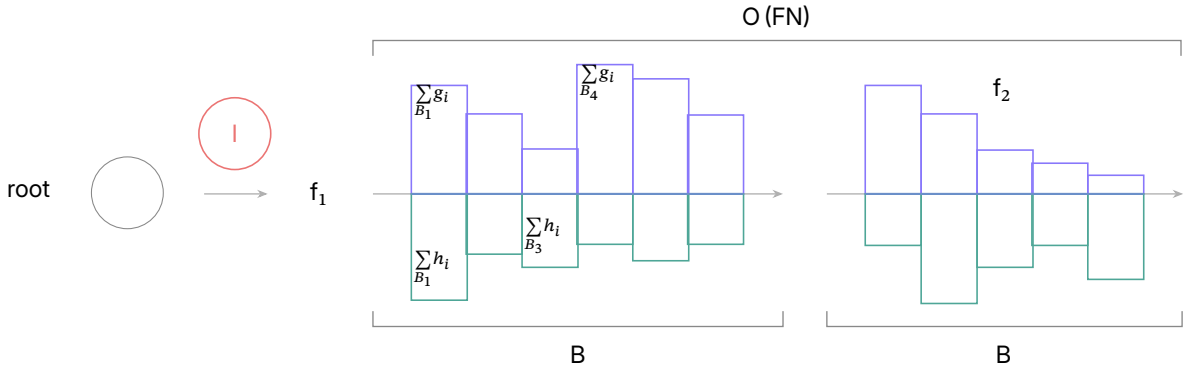


Рис. 9. Строим «гистограммы» для корня

Это будет стоить $O(F \cdot N)$, потому что в корне «находятся» все объекты обучающей выборки. Перейдём к выбору оптимального сплита.

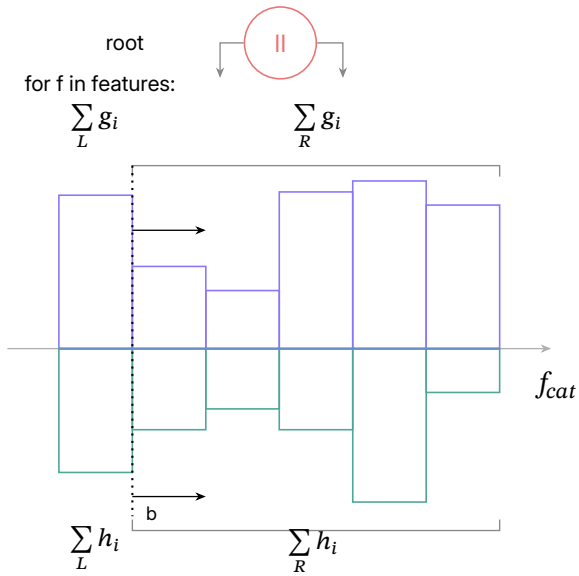


Рис. 10. Использование «гистограмм» для выбора сплита

5. В качестве порогов предложим для каждого признака построенные нами в начале бины (B штук). То есть теперь пороги перебираются не по каждому объекту, а по определенным порогам бинов. Для первого порога посчитаем снова левые и правые суммы g_i и h_i (это будет стоить $O(B)$, поскольку локальные суммы в каждом бине мы уже посчитали).
6. Теперь:
   - посчитаем Gain за $O(1)$;
   - поменяем значения этих сумм при смене порога за $O(1)$ — вместо $O(n_migrated)$ в прошлом варианте, так как граница сдвигается ровно на 1 бин, а не n объектов.

   Тогда получается, что для одного признака сложность скоринга всех порогов составит $O(N)$ — «гистограмма» + $O(B)$ — перебор порогов и скоринг. Асимптотически это не быстрее, чем pre-sorted-подход. Однако мы сможем добиться ускорения, если перейдём от корня к последующим вершинам.

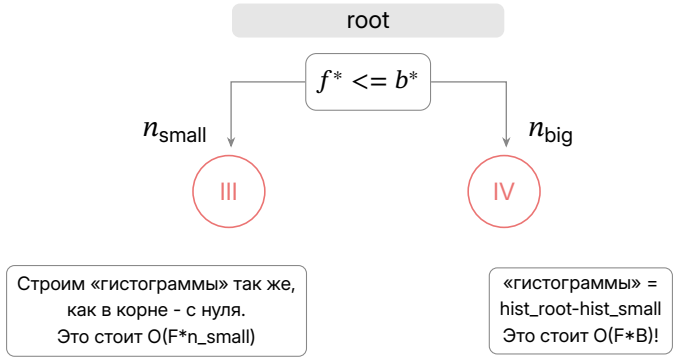


Рис. 11. Подсчёт «гистограмм» для дочерних вершин

7. После выбора оптимального сплита для корня появились две новые вершины. В них тоже предстоит выбирать сплит. Чтобы это сделать по нашему алгоритму, снова построим для них «гистограммы», переберём все (f, b) и проскорим. Подойдём теперь к построению «гистограмм» следующим образом:
   - для вершины, куда попало меньшее количество объектов n_{small}, построим «гистограммы» с нуля — это будет стоить $O(F \cdot n_{small})$. Затем можно выбирать в этой вершине оптимальный сплит;
   - чтобы получить значение «гистограммы» для вершины, куда попало бóльшее количество объектов n_{big}, вычитаем из «гистограмм» корня «гистограммы» «меньшей» вершины. Таким образом, имея родительскую гистограмму и меньшую по количеству объектов дочернюю гистограмму, можно быстро найти гистограмму большей дочерней вершины. Напомним, что «гистограммы» — это просто массивы чисел. Это будет стоить $O(B)$. После того как дочерние «гистограммы» построены, родительские «гистограммы» можно не хранить;
   - после построения «гистограмм» алгоритм не меняется — всё так же перебираем (f, b) и скорим каждый сплит за $O(1)$.

   При таком подходе мы выигрываем по скорости при построении «гистограмм» для вершин с бóльшим количеством объектов: вместо операции построения «гистограмм», стоящей $O(n_{big})$, мы выполняем быструю операцию вычитания «гистограммы» «меньшей» вершины из родительской, которая стоит $O(B)$.

Итого для pre-sorted

- Скорость: $O(F \cdot N_{leaf})$ для каждой вершины + $O(F \cdot N \log N)$ один раз.
- После выбора каждого сплита выбрать по сплиту для двух появившихся вершин занимает $O(F \cdot (n_{small} + n_{big})) = O(F \cdot N_{head})$.
- Память: $O(F \cdot N)$.

Итого для histogram-based

- Скорость: $O(F \cdot NlogN)$ один раз.
- Для маленьких вершин: построение «гистограмм» $O(F \cdot n_{small})$ + $O(F \cdot B)$ скоринг.
- Для больших вершин: построение «гистограмм» $O(F \cdot B)$ + $O(F \cdot B)$ скоринг.

- После каждого выбранного сплита выбрать по сплиту для двух появившихся вершин стоит $O(F(n_{small} + B)) \leq O(F \cdot N_{head}/2)$, если пренебречь $B \ll N$.
- Память: хранение гистограмм $O(F \cdot B)$ + хранение «гистограмм» всех вершин $O(F \cdot B) \rightarrow O(F \cdot B)$.

Получилось, что histogram-based-алгоритм даже в худшем случае даёт примерно двойной выигрыш по скорости и в N/B раз лучше по памяти. Единственный минус: мы предлагаем только бины вместо всех порогов, поэтому можно потерять в точности сплита. Но в ансамбле потери компенсируются последующими деревьями, поэтому это можно смело называть не погрешностью, а регуляризацией!

2.6. КРИТЕРИИ ОСТАНОВА ПОСТРОЕНИЯ ДЕРЕВА

Все истории когда-то заканчиваются, и наш градиентный бустинг — не исключение. Разберём условия, по которым завершается построение конкретного дерева и всего ансамбля.

Критерии останова для построения одного дерева:

- нельзя больше сделать ни один сплит:
  - Gain < 0 ~ loss_change < gamma;
  - `min_data_in_leaf` — ограничение на количество объектов в одном листе;
  - `min_sum_hessian_in_leaf` — ограничение на сумму h_i в одном листе.
- достигнуты `max_depth` и `max_leaves`.

Критерии останова для построения ансамбля:

- `num_boost_round` — пользователь задаёт количество деревьев;
- `early_stopping(K)` — если метрика качества не растёт K деревьев подряд, откатываемся к последнему дереву с наилучшей метрикой и останавливаемся.

Мы рассмотрели «классический» алгоритм на примере библиотеки XGBoost, а также принципы построения ансамбля.

- Вид деревьев — бинарные depthwise-деревья.
- Скорер сплитов — Gain (разница лоссов до и после сплита).
- Подход к предложению порогов и скорингу — гистограммный (предварительный биннинг признаков + вычитание «гистограмм» для большей вершины).

В заключение рассмотрим на примере процесс построения ансамбля. В нём опущены некоторые детали, чтобы не перегружать код.

```
# 1. ПРЕДВАРИТЕЛЬНАЯ ПОДГОТОВКА (делаем один раз в начале)
# Разбиваем числовые признаки на бины (гистограммы).
# Это требует O(F * N * logN) времени и O(F * B) памяти.
histograms = construct_feature_hists(X_tr[numeric_features])

# Начальное приближение предсказаний (например, среднее значение таргета или просто нули)
y_pred = np.zeros(X_tr.shape[0])
```

```
# 2. ЦИКЛ ПОСТРОЕНИЯ АНСАМБЛЯ (деревьев)
# Критерии останова ансамбля: заданное кол-во деревьев (num_trees)
# или срабатывание early_stopping (если метрика на валидации не растёт K итераций)
for tree_num in range(num_trees) and not early_stopping_condition():

    # Считаем градиенты (g_i) и гессианы (h_i) для всех объектов
    # на основе текущих предсказаний y_pred и выбранной функции потерь (Loss)
    g_i, h_i = calc_grad_hess(y_tr, y_pred, Loss)

    # Начинаем строить новое дерево с корневого узла
    root = new_node()
    # Очередь узлов для обработки. Для depthwise-подхода мы обрабатываем
    # все узлы текущего уровня, прежде чем перейти к следующему.
    nodes = {root}

    # 3. ЦИКЛ ПОСТРОЕНИЯ ОДНОГО ДЕРЕВА
    # Критерии останова дерева: достигнут max_depth, max_leaves,
    # или в узлах слишком мало данных (min_data_in_leaf / min_sum_hessian_in_leaf)
    while not stopping_conditions() and nodes is not empty:

        # Выбираем узел для разбиения (при depthwise это узлы текущего уровня)
        node = select_node(nodes)

        # Строим локальные «гистограммы» градиентов и гессианов для этого узла.
        # Для «меньшей» дочерней вершины считаем с нуля O(F * n_small),
        # для «большей» используем трюк вычитания из родительской гистограммы
          за O(F * B).
        hist_grad_hess = construct_feature_hists_grad_hess(node, histograms, g_i, h_i)

        max_gain = 0  # Инициализируем лучший Gain (порог для сплита, обычно gamma=0)
        optimal_split_info = None

        # Перебираем все признаки, чтобы найти лучший сплит
        for f in features:
            # Инициализируем суммы g_i и h_i для левого и правого ребёнка
            # для самого первого (минимального) порога (бина) признака f.
            # Это стоит O(N_leaf) или O(B), если брать из гистограммы.
            left_sum_g_i, left_sum_h_i, right_sum_g_i, right_sum_h_i = calc_sums(node,
              hist_grad_hess, f)

            # Перебираем все возможные пороги (бины) для признака f
            for b in histograms[f]:
                if not b.is_first():
                    # МАГИЯ O(1): вместо пересчёта с нуля мы «переносим» объекты,
                    # которые при новом пороге f <= b переходят из правой ветки в левую.
                    # Это требует O(n_migrated) операций, что в сумме по всем бинам
                      даёт O(B).
                    left_sum_g_i, left_sum_h_i, right_sum_g_i, right_sum_h_i =
                      recalc_sums(node, hist_grad_hess, f, b)

                # Считаем Gain (разницу лоссов до и после сплита) за O(1),
                # используя обновлённые суммы и формулу с регуляризацией lambda.
                Gain = calc_gain(left_sum_g_i, left_sum_h_i,
                  right_sum_g_i, right_sum_h_i)

                # Если текущий сплит лучше предыдущих, запоминаем его
```

```
                if Gain > max_gain:
                    max_gain = Gain
                    optimal_split_info = save_split_info(f, b, left_sum_g_i,
                      left_sum_h_i, ...)

        # 4. ПРИНЯТИЕ РЕШЕНИЯ О СПЛИТЕ
        # Если лучший Gain <= 0 (или < gamma), либо сработали другие ограничения
        # (например, min_data_in_leaf), сплит делать нецелесообразно.
        if max_gain <= 0 or other_stopping_conditions():
            # Узел становится листом. Вычисляем его оптимальный вес по формуле:
            # w = - sum(g_i) / (sum(h_i) + lambda)
            node.leaf_weight = calc_weight(node, g_i, h_i)
        else:
            # Сплит полезен! Применяем его.
            optimal_split = optimal_split_info

            # Добавляем новые дочерние узлы в очередь для обработки на следующем
              уровне глубины
            nodes.add(optimal_split.left_node)
            nodes.add(optimal_split.right_node)

            # Записываем условие разбиения (например, "Возраст <= 30")
            node.set_split(optimal_split.f <= optimal_split.b)
            node.set_children(optimal_split.left_node, optimal_split.right_node)

    # Дерево построено, добавляем его предсказания к y_pred и переходим к следующей
      итерации цикла
    y_pred += predict_tree(root, X_tr)
```

2.7. ИТОГИ ВЫБОРА СТРУКТУРЫ ДЕРЕВА

2.7.1. Как строятся деревья в XGBoost

Levelwise/depthwise подход: дерево растёт **по уровням** — на каждом уровне рассматриваются все вершины одновременно, для каждой находится свой оптимальный сплит, и все сплиты применяются сразу. Затем переходим к следующему уровню.

Gain — критерий качества сплита: для каждого кандидата на сплит (f,A)(f,A) считается разница лоссов до и после разбиения:

$$Gain = L_{\text{до}} - L_{\text{после}}$$

Сплит выбирается только если $Gain > 0$ (или $Gain > \gamma$ с учётом регуляризации). Иначе вершина остается листом.

2.7.2. Основные подходы к выбору сплитов

Подход	Скорость	Память	Точность
Полный перебор	$O(F \cdot N_{leaf}^2)$	$O(1)$	Максимальная
Pre-sorted	$O(F \cdot N_{leaf}) + O(F \cdot NlogN)$	$O(F \cdot N)$	Высокая
Histogram-based	$O(F \cdot NlogN)$	$O(F \cdot B)$	Высокая
Weighted Quantile Sketch	$O(F \cdot NlogN)$ + определение квантилей	$O(F \cdot B)$	Высокая

Ключевое наблюдение: histogram-based — оптимальный баланс скорости и памяти для современных задач. Он даёт примерно **двукратный выигрыш по скорости** и **снижение потребления памяти в N/B раз** по сравнению с pre-sorted.

2.7.3. Критерии останова построения дерева

Критерий	Что контролирует
gamma	Полезность сплита
min_data_in_leaf	Минимальное количество объектов в листе
min_sum_hessian_in_leaf	Минимальная сумма гессианов в листе
max_depth	Глубина дерева
max_leaves	Максимальное количество листьев

ВАЖНО

Эти параметры взаимосвязаны. Например, при фиксированной глубине `max_leaves` определяется автоматически. Чаще всего контролируют `min_data_in_leaf` и `max_depth`.

2.8. DART

Исторически алгоритм DART (Dropouts meet Additive Regression Trees) был предложен именно в 2015 году, а затем реализован в XGBoost. Цель подхода — решить одну специфическую проблему классического бустинга (MART — Multiple Additive Regression Trees).

В классическом бустинге предсказание ансамбля после k итераций записывается так.

$$\hat{y}_i^{(k)} = \sum_{m=1}^{k} w^{(m)}(x_i)$$

где $w^{(m)}(x_i)$ — значение листа в m-м дереве, куда попал объект x_i.

Но при большом количестве деревьев возникает проблема переспециализации (over-specialization): новые деревья начинают подстраиваться не под общие закономерности данных, а под специфические, иногда шумные «костыли», которые построили предыдущие деревья. То есть ансамбль начинает подстраиваться не только под данные (в которых есть шум), но и под шум, который создают предыдущие деревья.

Авторы позаимствовали идею решения проблемы из нейросетей, где слой Dropout (надеемся, ты его помнишь из ВвИИ, но, если что, вот оригинальная статья), не позволял последующим нейронам подстраиваться под шум, выученный прошлыми нейронами.

В нейросетях Dropout случайным образом «отключает» часть нейронов во время обучения, заставляя сеть учить более робастные и независимые признаки. В DART применяется тот же принцип, но на уровне деревьев.

1. Перед обучением нового k-го дерева мы случайным образом «выбрасываем» (игнорируем) часть деревьев из текущего ансамбля $1 \dots k-1$ с вероятностью p (параметр `rate_drop`). Обозначим множество «выброшенных» деревьев как D_k.
2. Новое дерево k обучается исправлять ошибки не полного ансамбля, а **«прореженного»** (thinned) ансамбля.

$$\hat{y}_{i,\text{thinned}}^{(k-1)} = \sum_{m \notin D_k} w^{(m)}(x_i)$$

ВАЖНО

«Выброшенные» деревья **не удаляются** из финальной модели! Они просто не учитываются при расчёте градиентов для текущего шага. В итоговом предсказании участвуют все построенные деревья.

Математический нюанс: нормализация

Если мы просто обучим дерево на прореженном ансамбле, его предсказания будут иметь другой масштаб (ведь базовый ансамбль стал «слабее»). Чтобы компенсировать это, DART вводит **нормализующий вес** для нового дерева.

Пусть $|D_k|$ — количество выброшенных деревьев. Тогда вклад нового дерева масштабируется коэффициентом: $\alpha_k = \frac{|D_k|}{|D_k|+\eta}$, η — learning rate (для режима нормализации `tree`); $\alpha_k = \frac{1}{1+\eta}$ (для режима нормализации`forest`);

Итоговое предсказание после k-й итерации записывается так.

$$\hat{y}_i^k = \hat{y}_{i,thinned}^{k-1} + \alpha_k \left(\hat{y}_{i,dropped}^{k-1} + w^{(k)}(x_i) \right)$$

$\hat{y}_{i,\text{dropped}}^{(k-1)} = \sum_{m \in D_k} w^{(m)}(x_i)$ — сумма предсказаний деревьев, которые были выброшены на этапе обучения k-го дерева.

> Фактически, нормализация сохраняет общий масштаб предсказаний ансамбля после возвращения дропнутых деревьев и предотвращает чрезмерный вклад нового дерева.

Преимущества

- **Мощная регуляризация:** DART действует как встроенный механизм борьбы с переобучением, часто позволяя использовать более глубокие деревья или больший learning rate без риска переобучиться.
- **Робастность:** ансамбль становится более устойчивым, напоминая по свойствам Random Forest, но сохраняя направленность градиентного спуска.

Недостатки

- **Замедление обучения:** на каждом шаге нужно пересчитывать предсказания прореженного ансамбля, что добавляет вычислительные накладные расходы (особенно заметно на CPU).
- **Сложность интерпретации:** из-за стохастичности процесса важность признаков (feature importance) может быть менее стабильной между запусками.

2.9. ПРИМЕР ИСПОЛЬЗОВАНИЯ XGBOOST

Ниже представлен пример использования библиотеки XGBoost с описанием основных гиперпараметров, которые упоминались в лонгриде.

```
import xgboost as xgb
```

```
# В XGBoost данные оборачиваются в оптимизированную структуру DMatrix.
# Это «классический» и самый быстрый способ подачи данных в библиотеку.
dtrain = xgb.DMatrix(X_tr, label=y_tr)
dval = xgb.DMatrix(X_val, label=y_val)

params = {
    # ==================== ОСНОВНЫЕ ПАРАМЕТРЫ ====================
    'objective': 'binary:logistic',  # Функция потерь (logloss
      для бинарной классификации)
    'eta': 0.1,                      # Learning rate (шаг градиентного спуска)
    'device': 'cpu',                 # 'cpu' или 'cuda' для GPU
    'nthread': 8,                    # Количество потоков для параллелизации

    # ==================== РЕГУЛЯРИЗАЦИЯ ====================
    # L2-регуляризация на веса листьев (λ в формулах)
    'lambda': 1.0, # reg_lambda alias
    # L1-регуляризация на веса листьев (α в формулах)
    'alpha': 0.0, # reg_alpha alias
    # Регуляризация на количество листьев (γ в формулах)
    'gamma': 0.0, # Минимальная потеря лосса для сплита

    # ==================== СТРУКТУРА ДЕРЕВЬЕВ ====================
    # Максимальная глубина дерева (критерий останова)
```

```
    # Для depthwise-подхода: дерево растёт по уровням до этой глубины
    'max_depth': 6,
    # Максимальное количество листьев в дереве (альтернативный критерий останова)
    # Если указано, игнорируется max_depth
    # 'max_leaves': 64,              # Опционально, по умолчанию 0 (не ограничено)

    # ==================== КРИТЕРИИ ОСТАНОВА  ====================
    # Минимальная сумма гессианов (h_i) в листе для создания сплита
    # Аналог min_data_in_leaf, но учитывает «уверенность» объектов (вторые производные)
    # Если sum(h_i) < min_child_weight, сплит не делается
    'min_child_weight': 1.0,

    # Минимальное количество объектов в листе (альтернатива min_child_weight)
    # 'min_data_in_leaf': 10,        # Более простой, но менее точный критерий

    # ==================== АЛГОРИТМ ПОСТРОЕНИЯ ДЕРЕВЬЕВ ====================
    # Метод построения деревьев
    # 'hist' — histogram-based (быстрый, O(F*B) памяти, рекомендуется)
    # 'approx' — weighted quantile sketch (приближенный)
    # 'exact' — pre-sorted (точный, но медленный и прожорливый)
    'tree_method': 'hist',

    # Количество бинов для гистограмм (B в формулах)
    # Обычно 256, но можно уменьшить для экономии памяти или увеличить для точности
    'max_bin': 256,

    # ==================== DART (Dropout Additive Regression Trees) ====================
    # По умолчанию XGBoost использует бустер 'gbtree'.
    # DART — это альтернатива, которая применяет идею dropout из глубокого обучения
    # на уровне деревьев: на каждой итерации случайно «выбрасывается» часть предыдущих
    # деревьев, что делает ансамбль более робастным и снижает переобучение.
    #
    # Чтобы включить DART, нужно задать booster='dart' и настроить параметры:
    # 'booster': 'dart',                        # Включаем DART вместо 'gbtree'
    # 'sample_type': 'uniform',                 # Тип семплирования: 'uniform'
      или 'weighted'
    # 'normalize_type': 'tree',                 # Тип нормализации: 'tree' или 'forest'
    # 'rate_drop': 0.1,                         # Вероятность дропнуть дерево (0.0 - 1.0)
    # 'skip_drop': 0.1,                         # Вероятность пропустить дроп (0.0 - 1.0)

    # ==================== ДОПОЛНИТЕЛЬНЫЕ ПАРАМЕТРЫ ====================
    # Доля признаков для каждого дерева (случайный выбор подмножества
      как в случайном лесе)
    # 'colsample_bytree': 1.0,       # Регуляризация через случайность

    # Доля объектов для каждого дерева (семплирование случайных объектов)
    # 'subsample': 1.0,              # Регуляризация через случайность

    # Сид для воспроизводимости
    # 'random_state': 42,
}

# Обучение модели
model = xgb.train(
    params,
    dtrain,
```

```
    num_boost_round=100,            # Количество деревьев (критерий останова ансамбля)
    evals=[(dtrain, 'train'), (dval, 'val')],  # Выборки для отслеживания метрик
    early_stopping_rounds=10,       # Early stopping: остановка, если метрика не растёт
      10 итераций
    verbose_eval=10                 # Вывод метрик каждые 10 итераций
)

# Предсказания
y_prob = model.predict(dval)                    # Вероятности (после сигмоиды)
y_raw = model.predict(dval, output_margin=True) # Сырые скоры (сумма весов листьев)

# Важность признаков (анализ модели)
importance = model.get_score(importance_type='gain')  # Важность по Gain (разнице лоссов)
```

2.10. ИТОГИ XGBOOST

XGBoost стал фундаментом современного градиентного бустинга, впервые сделав этот алгоритм по-настоящему масштабируемым и быстрым. Его алгоритм задал планку, от которой отталкивались все последующие библиотеки.

Ключевые выводы

1. **Три уровня регуляризации.** В отличие от базового бустинга, XGBoost борется с переобучением комплексно:
   - **L2 (λ)** сглаживает веса, делая предсказания консервативными;
   - γ мягко ограничивает сложность дерева, требуя минимального выигрыша в лоссе для каждого нового сплита;
   - **L1 (α)** зануляет веса слабых листьев, создавая разреженную модель и ускоряя инференс.
2. **Depthwise-рост и предсказание поправок.** Деревья строятся последовательно по уровням (levelwise).
3. **Gain как мера качества.** Алгоритм ищет разбиение, которое максимизирует **Gain** (разницу лоссов до и после сплита с учётом штрафа γ).
4. **Гистограммный подход (Histogram-based) как стандарт.** Это главный инженерный прорыв XGBoost. Предварительное разбиение признаков на B бинов снижает потребление памяти с $O(F \cdot N)$ до $O(F \cdot B)$ и даёт двукратный выигрыш в скорости за счёт трюка с вычитанием гистограмм для больших дочерних вершин.
5. **Мягкие критерии останова.** Вместо жёсткого ограничения только через `max_depth` XGBoost позволяет гибко контролировать рост дерева через `min_child_weight` (минимальная сумма гессианов в листе) и `gamma`, предотвращая создание бесполезных микросплитов.
6. **DART — dropout на уровне деревьев.** DART был предложен как модификация классического MART, а затем реализован в XGBoost и других библиотеках. На каждой итерации часть ранее построенных деревьев временно исключается при вычислении градиентов, что помогает бороться с переспециализацией ансамбля.

3. Библиотека LightGBM — «расширение» XGBoost

Полезные материалы

- Оригинальная статья LightGBM: A Highly Efficient Gradient Boosting Decision Tree
- Официальный репозиторий Github

XGBoost дал быструю и масштабируемую основу, но оставил несколько узких мест, которые ограничивают его эффективность на очень больших или разреженных данных, а также при работе с категориями. LightGBM целенаправленно устраняет именно эти ограничения, не меняя базовую идею гистограммного бустинга. Среди них есть следующие.

- **Равномерный рост деревьев.** XGBoost по умолчанию строит деревья levelwise (depthwise): на каждом уровне сплиты делаются сразу во всех листьях, независимо от того, насколько каждый из них перспективен. Это ведёт к лишней работе и перерасходу памяти. LightGBM заменяет levelwise на lossguide (leafwise) рост: на каждом шаге разбивается один лист с максимальным Gain. Это позволяет быстрее находить сложные закономерности, но требует более аккуратной регуляризации.
- **Построение гистограмм дорого по числу объектов.** В histogram-based алгоритме для «меньшей» дочерней вершины гистограмму всё ещё нужно строить с нуля за $O(F \cdot n_{small})$. Чтобы снизить n_{small} и общий N, LightGBM предлагает **Gradient-based One-Side Sampling (GOSS)**: объекты с наибольшими по модулю градиентами сохраняются полностью, а из остальных делается случайная подвыборка с нормировкой. Это уменьшает эффективное число объектов без катастрофической потери информации.
- **Поиск сплитов дорог при большом числе разреженных признаков (категорий).** Когда признаков много и они разрежены (например, one-hot кодированные категории или текстовые мешки слов), перебор всех F признаков остаётся затратным. LightGBM вводит **Exclusive Feature Bundling (EFB)** — технику, которая группирует взаимоисключающие признаки (те, что не принимают ненулевых значений одновременно) в один «бандл». Это уменьшает эффективное число признаков F без потери информации.
- **Изначально XGBoost вообще не умел работать с категориальными переменными.** От пользователя требовалось вручную преобразовывать их в числовое представление (чаще всего — one-hot). Это раздувает признаковое пространство и никак не учитывает естественную структуру данных. LightGBM впервые вводит **нативные категориальные сплиты**, перебирая осмысленные подмножества категорий на основе сортировки по отношению $sum(g)$ $sum(h)$. В зависимости от числа уникальных значений применяется One-hot split (для небольшого числа категорий) или более экономный Partition split. Это позволяет строить разбиения напрямую, без явного представления категорий в виде разреженной матрицы.

Таким образом, LightGBM представляет собой не просто «ускоренный XGBoost», а продуманный набор улучшений, каждое из которых нацелено на конкретное узкое место классического гистограммного бустинга. Разберём, как именно эти механизмы позволяют библиотеке обходить оригинальный XGBoost по скорости, часто не теряя (а иногда и выигрывая) в качестве.

ВАЖНО Lossguide-деревья и обработка категорий были впоследствии заимствованы и разработчиками XGBoost, но впервые они появились именно LightGBM.

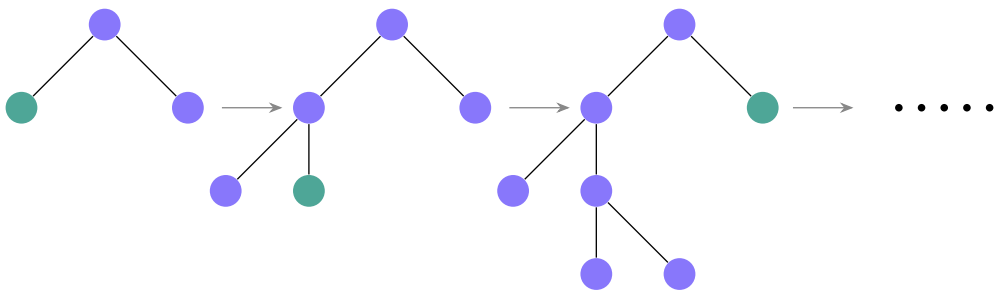
Кроме ключевых алгоритмических улучшений в разделе «Элементы практики» мы дополнительно рассмотрим удобные инструменты LightGBM: early stopping, веса объектов, кастомные метрики и лоссы, вывод деревьев в DataFrame, режим случайного леса, GBDT-PL (линейные модели в листьях) и CEGB (cost-efficient boosting).

3.1. LOSSGUIDE-ДЕРЕВЬЯ ((LEAFWISE-ДЕРЕВЬЯ)

В отличие от классического depthwise-подхода (который исторически использовался в XGBoost по умолчанию), где дерево растёт строго по уровням, LightGBM использует стратегию **leafwise** (или **lossguide**).

Механика построения выглядит так.

1. Алгоритм начинает с корневого узла, содержащего все объекты.
2. На каждом шаге он оценивает все возможные сплиты для **всех текущих листьев** дерева (а не только для листьев последнего уровня).
3. Из всех кандидатов выбирается **один-единственный** сплит, который даёт максимальный Gain во всём дереве.
4. Этот сплит применяется: выбранный лист разделяется на два новых.
5. Процесс повторяется, пока не будет достигнут лимит по одному из аналогичных XGBoost гиперпараметрам (максимальная глубина, минимальное количество объектов в листе, минимальный Gain, минимальная сумма гессианов, количество листьев и т. д.).



Leaf-wise tree growth

Рис. 12. Источник: https://bangdasun.github.io/2019/03/21/38-practical-comparison-xgboost-lightgbm/

Такой «жадный» подход даёт leafwise-деревьям несколько серьёзных преимуществ перед depthwise.

- **Максимальный Gain на каждом шаге.** Поскольку на каждой итерации выбирается единственная вершина с максимальным Gain во всём дереве, каждый новый сплит даёт максимально возможное уменьшение лосса «здесь и сейчас».
- **Лучшее качество при том же количестве листьев.** Depthwise-дерево вынуждено делать сплиты на всех вершинах текущего уровня, даже если некоторые из них дают мизерный Gain. Leafwise «инвестирует» сплиты только туда, где они действительно полезны. При одинаковом гиперпараметре `num_leaves` leafwise-модель часто выигрывает в качестве.

- **Адаптивная глубина веток.** Алгоритм автоматически подстраивается под локальную сложность данных: в «сложных» областях (где много объектов с разными таргетами) ветки растут глубже, а в «простых» (где объекты однородны) остаются мелкими. Depthwise же строит дерево по уровням: на каждом уровне сплиты пытаются сделать во всех доступных для разбиения листьях.

3.1.1. Главный недостаток и как с ним бороться

Из-за своей «жадности» leafwise-деревья могут быстро подстроиться под шум и **сильнее переобучаться**, особенно на маленьких датасетах. Алгоритм может вырастить одну очень длинную ветку («иглу»), которая идеально запомнит несколько выбросов, в ущерб обобщающей способности.

Решение: В LightGBM критически важно не полагаться только на `num_leaves`. Необходимо жёстко контролировать гиперпараметр `max_depth`. Это не даёт алгоритму уйти в чрезмерную глубину, сохраняя при этом все преимущества leafwise-роста.

3.1.2. Итог Lossguide деревьев

Таким образом, leafwise-рост делает деревья более «экономными»: каждый новый сплит даёт максимально возможное снижение ошибки, а глубина ветвей естественно подстраивается под локальную сложность данных. Это часто приносит выигрыш в точности при том же числе листьев, что ускоряет обучение.

3.2. УСКОРЕНИЕ ПОСТРОЕНИЯ ДЕРЕВЬЕВ

Улучшив структуру деревьев, LightGBM не останавливается на достигнутом. Даже с гистограммным подходом скорость обучения всё ещё зависит от количества объектов и признаков. Два следующих механизма — **GOSS** и **EFB** — адресованы именно этим факторам: первый уменьшает эффективный размер выборки, второй — эффективную размерность признакового пространства. Оба работают как «надстройка» над базовым алгоритмом и могут применяться совместно, дополнительно ускоряя построение ансамбля без заметной потери качества.

При Histogram-based-алгоритме мы избавились от перебора всех объектов в большой ветке (заменив его на быстрое вычитание гистограмм за $O(F \cdot B)$, но для меньшей ветки гистограмму всё ещё нужно строить с нуля за $O(F \cdot n_{small})$. Следовательно, чтобы ускорить алгоритм ещё сильнее, можно уменьшить количество признаков F или количество объектов в выборке N (что автоматически уменьшит и N_{leaf}, и n_{small}).

3.2.1. GOSS

Идея GOSS (Gradient-based One Side Sampling) заключается в том, чтобы для построения каждого нового дерева использовать не всю обучающую выборку, а только её информативное подмножество (то есть уменьшаем N). Случайный семплинг (как в Random Forest) отбросил бы важную информацию, поэтому GOSS использует для отбора сами градиенты. Алгоритм работает в данном порядке.

1. Полный расчёт: на основе предсказаний уже построенного ансамбля мы считаем градиенты g_i и гессианы h_i для всех N объектов обучающей выборки.

2. Сортировка: мы сортируем объекты по абсолютному значению градиента $|g_i|$ по убыванию. Объекты с большим $|g_i|$ — это те, где текущая модель ошибается сильнее всего («сложные» объекты).
3. Отбор топ-ошибок: мы гарантированно берем $top_rate \cdot N$ объектов с наибольшими $|g_i|$. Они составят основу для обучения нового дерева.
4. Случайная выборка: из оставшихся объектов мы случайно выбираем ещё $other_rate \cdot N$ объектов. Это необходимо, чтобы сохранить исходное распределение данных и не позволить модели переобучиться только на «сложных» примерах, игнорируя общую структуру.
5. Нормировка: поскольку мы взяли лишь часть «простых» объектов, общая сумма градиентов изменится, что исказит оценку Gain при поиске сплитов. Чтобы это исправить, градиенты и гессианы случайно выбранных объектов домножаются на нормировочный коэффициент: $\frac{1-top_rate}{other_rate}$. Это математически компенсирует уменьшение выборки, и оценка Gain остается несмещённой, как если бы мы использовали все N объектов.
6. Построение дерева: новое дерево строится только на этом уменьшенном наборе данных размером $(top_rate + other_rate) \cdot N$.

GOSS можно использовать, когда нужно ускорить процесс обучения, не слишком опасаясь за потерю качества. По умолчанию этот метод семплирования отключён, и, чтобы его активировать, нужно выставить гиперпараметры.

Таким образом, GOSS позволяет радикально уменьшить количество объектов, участвующих в построении конкретного дерева (уменьшает N_{leaf}), фокусируя вычислительные ресурсы там, где модель нуждается в улучшениях больше всего.

3.2.2. EFB

Чтобы уменьшить количество признаков, используется алгоритм сжатия непересекающихся признаков в так называемые «бандлы» (уменьшаем F). Идея EFB (Exclusive Feature Bundling): объединять в один признаки с неотрицательными значениями, которые никогда не принимают ненулевые значения одновременно (см. формулу ниже). Чтобы это сделать, нужно ко всем ненулевым значениям второго признака прибавить $f_1.max()$. Тогда по значению нашего нового признака-бандла f^* можно будет однозначно восстановить изначальные значения f_1 и f_2: если $f^* > f_1.max()$, то $f_1 = 0, f_2 = f^* - f_1.max()$. В обратном случае значения находятся аналогично.

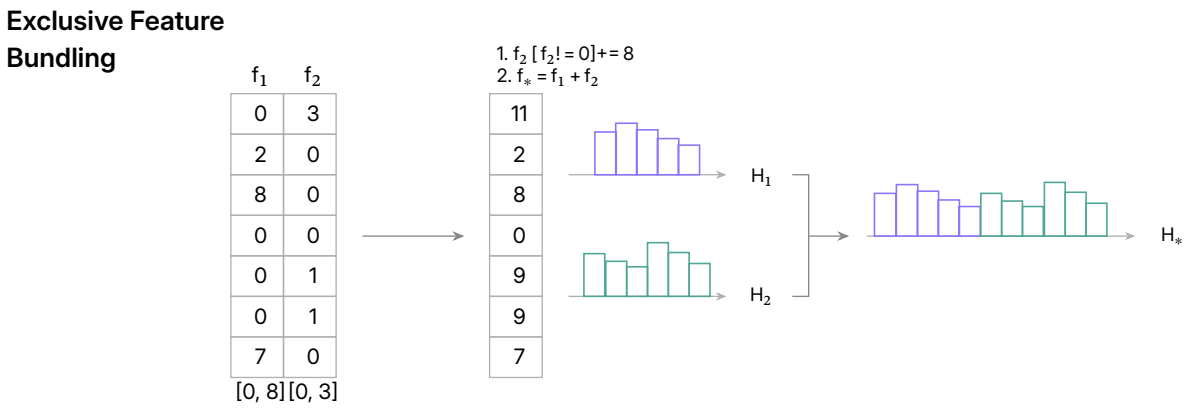


Рис. 13. EFB

EFB особенно эффективен:

- когда у тебя есть **one-hot-кодированные признаки** (где для каждого объекта ненулевым является только один признак);
- ты работаешь с **текстовыми данными** (TF-IDF представление часто разреженное);
- у тебя **категориальные признаки с большим количеством уникальных значений**.

EFB работает по умолчанию. Отключить его можно, изменив гиперпараметр `enable_bundle`, но лучше этого не делать, так как найти такие признаки крайне просто вычислительно, при этом сложность алгоритма при поиске сплита снижается с $O(F \cdot B)$ до $O(F_{new} \cdot B)$, где $F_{new} \leq F$.

3.2.3. Итоги ускорения построения деревьев

Оба метода адресованы ключевым источникам вычислительной сложности: GOSS уменьшает эффективное число объектов, EFB — эффективное число признаков. GOSS фокусируется на «трудных» примерах, ускоряя обучение без серьёзной потери точности, но требует большую выборку и осторожного подбора `top_rate` и `other_rate` — слишком агрессивный семплинг может ухудшить обобщение. EFB практически «бесплатно» сжимает разреженные признаки, снижая сложность поиска сплитов с $(F \cdot B)$ до $O(F_{new} \cdot B)$, однако наибольший выигрыш даёт именно на разреженных данных, на полностью плотных числовых признаках эффекта не будет.

3.3. СПЛИТЫ ДЛЯ КАТЕГОРИАЛЬНЫХ ПРИЗНАКОВ

Ускорив построение деревьев, LightGBM обращается к ещё одному ограничению классического XGBoost — отсутствию встроенной поддержки категориальных признаков. Вместо того чтобы заставлять пользователя вручную кодировать категории, библиотека предлагает **нативные категориальные сплиты**, которые строятся непосредственно по значениям признака без создания разреженной one-hot-матрицы. Рассмотрим их подробнее.

За сплиты для категориальных признаков здесь отвечает гиперпараметр `max_cat_to_onehot`:

- если количество уникальных значений у категориального признака f_{cat} не больше `max_cat_to_onehot`, то применяется «One-hot split» вида $f_{cat} == A^*$, где A^* — значение признака f_{cat}. То есть как будто бы делается обычный one-hot encoding, но внутри самого алгоритма без явного представления.
- если количество уникальных значений признака больше `max_cat_to_onehot`, то применяется «Partition split» вида $f_{cat} \in \{A, B, C\}$, где A, B, C — значения признака. То есть некоторые категории как будто бы объединяются в одну. При этом объединение выбирает сам алгоритм.

Перебирать все возможные варианты подмножеств для сплита очень долго (их 2^K, где K — количество уникальных значений признака f_{cat}). Тогда была реализована следующая идея из этой статьи: достаточно перебирать лишь те подмножества категорий (какие категории после разбиения пойдут влево, а какие вправо), которые получены в результате введения на множестве значений признака специального отношения порядка. Если писать более обывательским языком, то мы придумали правило, по которому можно выстроить категории от «худших» к «лучшим».

О каких же подмножествах речь? Разберём на примере.

1. Мы считаем «гистограмму» для признака f_{cat}, как мы делали это ранее для числовых признаков, только значениями бинов будут все уникальные значения признака.

Напомним, что «гистограмма» — это посчитанные значения $sum(g_i), sum(h_i)$ для объектов из каждого «бина». Как и ранее, для «меньшей» вершины строим «гистограммы» с нуля, а для «большей» — через вычитания из «гистограммы» родительской вершины.

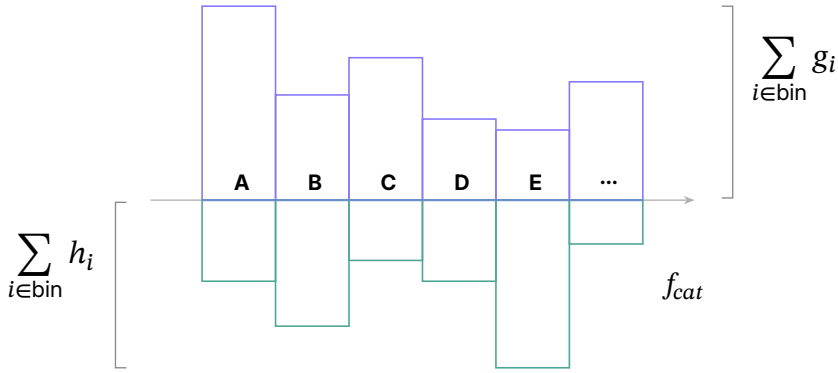


Рис. 14. «Гистограмма» для категориального признака

2. Задаём порядок: сортируем получившиеся «бины» по показателю $sum(g_i)/sum(h_i)$. Можно добавить ещё коэффициенты регуляризации (об этом позже).

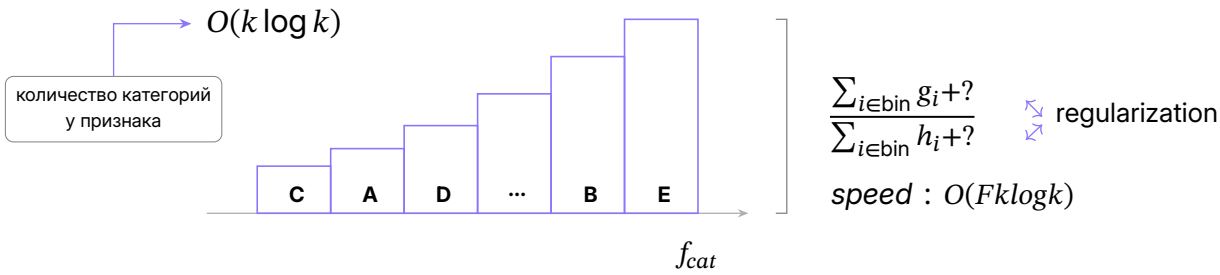


Рис. 15. Упорядочивание «гистограммы» для категориального признака

3. После пересортировки мы будем работать с получившимися «бинами» как с обычным числовым признаком, представляя, что значения категорий равны соответствующим им значениям $sum(g_i)/sum(h_i)$.
   Такой подход и даст нам нужные подмножества. Например, на рисунке ниже первым предлагаемым подмножеством будут $\{C\}$, затем $\{C, A\}$, затем $\{C, A, D\}$ и так далее. Получается, что вместо 2^K подмножеств мы перебираем всего K.
   Если категорий в признаке меньше `max_cat_to_onehot`, то разрешены множества с одной категорией (например, $\{C\}$). Если больше, то категории будут объединяться (например, $\{C, A\}$).

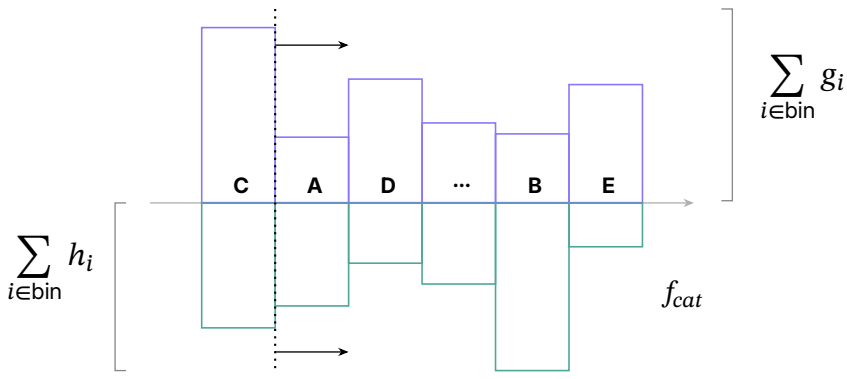


Рис. 16. Использование упорядоченной «гистограммы» для выбора сплита

Перебора только этих подмножеств достаточно, чтобы получить сплит не хуже, чем при полном переборе. Объясняется это тем, что в этом случае мы сортируем по $sum(g_i)/sum(h_i)$, что очень похоже на формулу оптимальных весов, от которых зависит Gain. То есть, следуя базовому алгоритму, хороший сплит — это такой сплит, который максимально разделяет объекты с разными «идеальными прогнозами», упорядочив наши категории таким же образом, мы сможем найти лучший с точки зрения Gain сплит.

Параметры, отвечающие за кодировку категорий:

- `max_cat_to_onehot` — если количество уникальных значений категориального признака $K \leq$ `max_cat_to_onehot` , то применяется One-hot split, иначе Partition split;
- `max_cat_threshold` — максимальное количество бинов для категориальных признаков (если $K >$ `max_cat_threshold`, то маленькие бины сливаются в один);
- `min_data_per_group` — минимальное количество объектов в категориальных бинах (если $K <$ `min_data_per_group`, то бины сливаются в один).

Как мы сказали выше, на самом деле сортировка категориальных бинов идёт по значению.

$$\frac{sum(g_i) + cat_smooth}{sum(h_i) + cat_l2}.$$

Следовательно, возникают ещё 2 гиперпараметра:

- `cat_l2` — регуляризатор в знаменатель;
- `cat_smooth` — регуляризатор в числитель.

Эти гиперпараметры сглаживают значения для редких категорий, для которых сумма градиентов и гессианов маленькая (потому что объектов такой категории мало), но при этом отношение может быть очень большим (как будто бы объект очень важный). Сглаживание позволяет бороться с этой проблемой, подтягивая полученный скор важности к некоторому среднему значению для редких категорий.

3.3.1. Итоги построения сплитов по категориальным признакам

Нативные категориальные сплиты избавляют от необходимости вручную кодировать категории, предотвращая раздувание признакового пространства. Сортировка «бинов» по отношению $sum(h_i)$ $sum(g_i)$ и перебор последовательных подмножеств позволяют находить оптимальные разбиения за $O(K \cdot B)$, не перебирая 2^K вариантов.

One-hot split удобен для небольшого числа категорий, а Partition split объединяет редкие значения, снижая переобучение. Параметры сглаживания `cat_smooth` и `cat_l2` защищают от шума на редких категориях, подтягивая их скор к среднему.

Ограничение: при очень большом числе категорий поиск сплитов всё ещё может замедляться, а неправильная настройка сглаживания способна исказить порядок категорий.

3.4. ЭЛЕМЕНТЫ ПРАКТИКИ

Помимо перечисленных алгоритмических улучшений, LightGBM предоставляет богатый набор практических инструментов, которые делают повседневную работу с моделями удобнее и гибче. Ниже мы кратко рассмотрим наиболее полезные из них: early stopping, веса объектов, кастомные метрики и лоссы, сохранение деревьев в DataFrame, режим случайного леса, GBDT-PL и CEGB.

3.4.1. Early stopping (критерий раннего останова)

Критерий раннего останова определяет, когда нужно закончить строительство ансамбля.

Когда речь шла об XGBoost, мы уже говорили про этот механизм — он отслеживает значение метрики на валидационной выборке после построения каждого дерева. Если выбранная метрика не улучшается достаточно долго, алгоритм перестанет строить новые деревья и вернётся к последнему дереву, которое дало улучшение метрики.

На практике всё аналогично и выглядит следующим образом.

```
lgb_tr = lgb.Dataset(X_tr, y_tr)
lgb_val = lgb.Dataset(X_val, y_val)

params = {
    'metric': 'auc' # Метрика, на которую смотрим - ROC-AUC. '' - мерить objective loss,
      'None' - метрика отсутствует
}

model = lgb.train(
    params, train_set=lgb_tr, num_boost_round=1000,
    valid_sets=[lgb_val],
    valid_names=['необязательное название выборки'],
    callbacks=[lgb.early_stopping(10), lgb.log_evaluation(10)]
)
```

```
Training until validation scores don't improve for 10 rounds
[10]    необязательное название выборки's auc: 0.917829
[20]    необязательное название выборки's auc: 0.923798
[30]    необязательное название выборки's auc: 0.927117
[40]    необязательное название выборки's auc: 0.92942
[50]    необязательное название выборки's auc: 0.930936
[60]    необязательное название выборки's auc: 0.931988
[70]    необязательное название выборки's auc: 0.93257
[80]    необязательное название выборки's auc: 0.932927
[90]    необязательное название выборки's auc: 0.933092
Early stopping, best iteration is:
[88]    необязательное название выборки's auc: 0.933124
```

Рис. 17. Пример логгов LightGBM во время обучения

3.4.2. Веса на объектах

Часто необходимо задать веса на объектах, чтобы перераспределить их вклад в обучение модели. Например:

- в задачах бинарной классификации, если объектов класса 1 значительно меньше, чем объектов класса 0, можно задать им бóльший вес, чтобы модель относилась к ним более чутко;
- в задачах, где у каждого объекта есть какой-то временной таймстемп (например, дата заказа товара). В таком случае логичнее учитывать более новые заказы с бóльшим весом, чем прошлогодние, информация о которых уже могла устареть. При этом лучше задать небольшие веса старым заказам, чем не учитывать такие данные.

Как именно учитываются веса на объектах в обучении модели? Достаточно знать, что они будут множителями при компонентах лосса, соответствующих объектам: $L_{total} = \sum_{i=1}^{N} w_i \cdot L(y_i, \hat{y}_i) + regularization.$

Тогда в формулах для оптимальных весов (= скоров в листьях), а следовательно, и для Gain $g_i \to w_i g_i, h_i \to w_i h_i$: все «градиенты» и «гессианы» будут взвешиваться на веса (= веса на объектах). Так будет происходить во встроенных в LightGBM лоссах. Однако при написании своих лоссов можно использовать веса на объектах по своему усмотрению.

В LightGBM можно явно передать вектор весов через `lgb.Dataset`. А чтобы быстро задать веса в задаче бинарной классификации, можно использовать два аргумента:

- `'is_unbalance'`=True | $w_0 = 1, w_1 = n_0/n_1$. Таким образом, редкие объекты штрафуются сильнее;
- `'scale_pos_weight'`=4.33 | $w_0 = 1, w_1 = scale_pos_weight$. То есть домножаем ошибку на классе 1 на константу `scale_pos_weight`.

Эти варианты задания весов нельзя использовать вместе.

```
w = np.random.randn(X_tr.shape[0]) # Вариант 1
lgb_tr = lgb.Dataset(X_tr, y_tr, weight=w)

params.update({
    'scale_pos_weight': 1, # Вариант 2
    'is_unbalance': True # Вариант 3, вместе не используем
})
```

3.4.3. Кастомные метрики и лоссы

Часто возникает потребность реализовать собственную метрику, которую можно отслеживать в процессе обучения и, вероятно, искать оптимальную точку останова именно по ней. Например, это может быть F1-мера, которой нет в LightGBM, или какая-нибудь метрика, нечувствительная к ошибке величиной меньше ϵ. В LightGBM для этого можно написать функцию по заданному шаблону, который мы рассмотрим на семинаре.

Аналогично можно написать и лосс, по которому будет обучаться алгоритм. Это может потребоваться для того, чтобы алгоритм был больше «сонаправлен» с целевой метрикой.

3.4.4. Деревья в виде датафрейма

В LightGBM построенные деревья можно вывести в виде датафрейма с помощью метода `model.trees_to_dataframe()`. Это открывает обширные возможности для парсинга и извлечения важной информации из получившейся модели.

3.4.5. «Режим» случайного леса

В LightGBM можно обучить случайный лес, а не градиентный бустинг, изменив гиперпараметр `boosting` на значение `'rf'`. Такое обучение эквивалентно построению ансамбля на одних и тех же «градиентах» и «гессианах», посчитанных на начальных приближениях. При обучении случайного леса следует убедиться, что семплирование признаков и/или объектов настроено на каждое дерево (подробнее про настройки поговорим на семинаре).

GBDT-PL (линейные регрессии в листьях вместо констант)

В LightGBM можно организовать процесс обучения так, чтобы в листьях вместо констант обучались линейные регрессии на признаках, участвующих в цепочке сплитов, ведущих из корня в лист.

Каждый объект, попавший в лист, будет получать разный скор, основанный на значении своих признаков. Это существенно повышает экстраполяционные возможности градиентного бустинга, в то время как просадка по времени почти отсутствует. Однако нужно проявить внимательность в подборе параметра регуляризации. Подробнее вопрос можно изучить в этой статье.

```
lgb_tr = lgb.Dataset(X_tr, y_tr)
params.update({
    'linear_tree': True,
    'linear_lambda': 0.13,
})
model = lgb.train(params, train_set=lgb_tr, num_boost_round=200)
```

CEGB

Бывает, что нужно, чтобы модель использовала как можно меньше признаков из тех, что поданы на вход. Например, это может возникать в случаях, когда модель должна работать в онлайне очень быстро, а признаки для модели тянутся из базы. В таком случае нельзя тратить много времени на то, чтобы подтягивать большое количество табличек, поэтому нужно сократить их количество. На этапе обучения можно задать штрафы на признаки, после чего модель будет учитывать, что каждый новый признак, появляющийся в сплитах, добавляет какой-то штраф в лосс. Это называется **CEGB** (Cost-Efficient Gradient Boosting).

В LightGBM есть возможность встроить не только такие ограничения, но и штрафы на глубину построенных в процессе обучения деревьев. Подробнее вопрос можно изучить в этой статье.

3.4.6. Итоги элементов практики

Таким образом, early stopping, веса объектов для контроля обучения, кастомные метрики и лоссы для точной настройки под бизнес-задачи, экспорт деревьев в DataFrame для интерпретации, режим случайного леса как альтернативу бустингу, а также продвинутые техники — GBDT-PL для линейных поправок в листьях и CEGB для штрафа за добавление признаков — делают LightGBM не только быстрым, но и гибким инструментом, позволяющим решать широкий круг задач без дополнительных надстроек.

Ниже приведён полный пример обучения модели LightGBM, в котором используются многие из описанных параметров и возможностей.

3.5. ПРИМЕР ИСПОЛЬЗОВАНИЯ LIGHTGBM

Ниже представлен код, который позволяет обучить модель LightGBM на датафрейме с обучающей выборкой.

1. Его нужно обернуть в специальную структуру `lgb.Dataset`.
2. Передать получившийся результат в функцию `train` вместе с параметрами (большую часть из них мы уже затронули в лонгриде, а более подробно поговорим о них на семинаре).

```
import lightgbm as lgb
import numpy as np

# ==================== 1. ПОДГОТОВКА ДАННЫХ ====================
# В LightGBM данные оборачиваются в lgb.Dataset.
# Ключевая фишка: мы можем явно указать категориальные признаки.
# LightGBM сам применит к ним оптимизированные One-hot- или Partition-сплиты.

# (Опционально) если нужны индивидуальные веса объектов, их можно передать здесь:
# weights = np.array([...])
lgb_tr = lgb.Dataset(
    X_tr,
    label=y_tr,
    categorical_feature=['cat_feature_1', 'cat_feature_2'], # Укажи имена твоих
      кат. признаков
#    weight=weights
)

# Для валидации обязательно указываем reference=lgb_tr, чтобы бины гистограмм
# строились на основе обучающей выборки (избегаем утечки данных).
lgb_val = lgb.Dataset(
    X_val,
    label=y_val,
```

```
    categorical_feature=['cat_feature_1', 'cat_feature_2'],
    reference=lgb_tr
)


# ==================== 2. НАСТРОЙКА ГИПЕРПАРАМЕТРОВ ====================
params = {
    # ==================== ОСНОВНЫЕ ПАРАМЕТРЫ ====================
    'objective': 'binary',           # Функция потерь (logloss
      для бинарной классификации)
    'learning_rate': 0.1,            # learning_rate (аналог eta в XGBoost)
    'device_type': 'cpu',            # 'cpu' или 'gpu'
    'nthread': 8,                    # Количество потоков для распараллеливания
    'boosting': 'gbdt',              # 'gbdt' (градиентный бустинг) или 'rf' (режим
      случайного леса)

    # ==================== СТРУКТУРА ДЕРЕВЬЕВ ====================
    # LightGBM по умолчанию использует leaf-wise (lossguide) рост, а не depth-wise.
    # Поэтому здесь чаще контролируем именно количество листьев, а не глубину.
    'num_leaves': 31,                # Максимальное количество листьев в дереве (главный
      контроль сложности)
    'max_depth': -1,                 # -1 означает отсутствие ограничения по глубине
      (контролируем через num_leaves)

    # ==================== РЕГУЛЯРИЗАЦИЯ ====================
    # L2-регуляризация на веса листьев (аналог lambda в XGBoost)
    'lambda_l2': 1.0,                # Сглаживает предсказания, штрафует большие веса
    # L1-регуляризация на веса листьев (аналог alpha в XGBoost)
    'lambda_l1': 0.0,                # Гарантирует строгий ноль для слабых сигналов
    # Минимальное количество объектов в листе для создания сплита
    'min_data_in_leaf': 20,          # Предотвращает микросплиты
    # Минимальная сумма гессианов в листе (альтернатива min_data_in_leaf)
    'min_sum_hessian_in_leaf': 1e-3,

    # ==================== УСКОРЕНИЕ (GOSS и EFB) ====================
    # EFB (Exclusive Feature Bundling) включен по умолчанию.
    # 'enable_bundle': True,         # Сжатие взаимоисключающих признаков в бандлы

    # GOSS (Gradient-based One Side Sampling) отключён по умолчанию.
    # Чтобы включить, раскомментируй и настрой (сумма top_rate + other_rate < 1):
    # 'data_sample_strategy': 'goss',
    # 'top_rate': 0.2,               # Доля объектов с наибольшими по модулю градиентами
    # 'other_rate': 0.1,             # Доля случайно выбранных оставшихся объектов

    # ==================== КАТЕГОРИАЛЬНЫЕ ПРИЗНАКИ ====================
    # Порог для выбора типа сплита: One-hot или Partition
    'max_cat_to_onehot': 4,          # Если уникальных значений <= 4, будет One-hot
      split, иначе Partition split

    # Защита от переобучения на редких категориях
    'max_cat_threshold': 64,         # Если категорий > 64, самые редкие сливаются в один
      бин "other"
    'min_data_per_group': 100,       # Если в категории < 100 объектов, она сливается
      в бин "other"
```

```
    # Сглаживание (smoothing) формулы сортировки категорий: (sum_g + cat_smooth) /
      (sum_h + cat_l2)
    'cat_smooth': 10.0,              # Регуляризатор в числитель (подтягивает скор редких
      категорий к среднему)
    'cat_l2': 10.0,                  # Регуляризатор в знаменатель (избегает деления
      на малые числа)

    # ==================== DART (Dropout Additive Regression Trees) ====================
    # По умолчанию LightGBM использует классический GBDT (Gradient Boosting
      Decision Trees).
    # DART — это альтернатива, которая применяет идею dropout из глубокого обучения
      на уровне деревьев
    # на каждой итерации случайным образом «выбрасывается» часть предыдущих деревьев,
    # что делает ансамбль более робастным и снижает переобучение.
    #
    # Чтобы включить DART, измени boosting на 'dart' и настрой параметры
    # 'boosting': 'dart',            # Включаем режим DART вместо 'gbdt'
    # 'drop_rate': 0.1,              # Вероятность дропнуть дерево из ансамбля
      (0.0 - 1.0)
    # 'max_drop': 50,                # Максимальное количество деревьев для дропа
    #                                # (ускоряет работу на больших итерациях,
      ограничивая перебор)
    # 'skip_drop': 0.1,              # Вероятность пропустить дроп и использовать
      стандартный GBDT
    #                                # (защита от слишком сильного разреживания ансамбля)
    # 'uniform_drop': False,         # Если True — равномерное семплирование деревьев
      для дропа
    #                                # Если False — взвешенное (деревья с большими весами
      дропаются чаще)
    # 'xgboost_dart_mode': False,    # Если True — режим совместимости с XGBoost DART
    #                                # (влияет на способ нормализации весов
      нового дерева)
    #
    # Важно: при использовании DART обучение может замедлиться,
    # так как на каждом шаге нужно пересчитывать предсказания прореженного ансамбля.
    # DART особенно полезен на зашумлённых данных среднего размера,
    # где классический GBDT быстро переобучается.

    # ==================== ПРОДВИНУТЫЕ ВОЗМОЖНОСТИ ====================
    # GBDT-PL: обучение линейных регрессий в листьях вместо констант
    # 'linear_tree': False,
    # 'linear_lambda': 0.1,          # Регуляризация для линейных моделей в листьях

    # Работа с несбалансированными классами (НЕЛЬЗЯ использовать вместе с is_unbalance)
    # 'scale_pos_weight': 10.0,      # Явный множитель веса для класса 1
      (w_1 = scale_pos_weight)
    # 'is_unbalance': False,         # Если True, LightGBM сам выставит
      scale_pos_weight = n_0 / n_1
}

# ==================== 3. ОБУЧЕНИЕ МОДЕЛИ ====================
model = lgb.train(
    params,
    train_set=lgb_tr,
    num_boost_round=1000,            # Максимальное количество деревьев (критерий
      останова ансамбля)
```

```
    valid_sets=[lgb_val],            # Выборки для отслеживания метрик
    valid_names=['val'],
    callbacks=[
        lgb.early_stopping(stopping_rounds=50), # Останов, если метрика не растёт
          50 итераций подряд
        lgb.log_evaluation(period=50)           # Вывод логов в консоль каждые
          50 итераций
    ]
)

# ==================== 4. ПРЕДСКАЗАНИЯ И АНАЛИЗ ====================
# «Вероятности» (к сумме весов листьев автоматически применена функция активации,
  например, сигмоида)
y_prob = model.predict(X_val)

# «Сырые» скоры (чистая сумма весов листьев до применения функции активации, аналог
  output_margin в XGBoost)
y_raw = model.predict(X_val, raw_score=True)

# Анализ модели: вывод всех деревьев и сплитов в виде pandas DataFrame.
# Это уникальная фишка LightGBM для парсинга и извлечения инсайтов из модели.
# df_trees = model.trees_to_dataframe()
# print(df_trees.head())
```

3.6. ИТОГИ LIGHTGBM

В совокупности всё вышеперечисленное даёт LightGBM следующие преимущества.

- **Высокая скорость и низкое потребление памяти.** Гистограммный биннинг, GOSS и EFB радикально снижают вычислительную сложность и объём оперативной памяти. LightGBM часто обучается в разы быстрее XGBoost при сопоставимом качестве, а на очень больших датасетах может быть единственным практичным выбором.
- **Качество на уровне или выше аналогов.** Leafwise-рост позволяет выжимать максимум из заданного числа листьев, а нативная работа с категориями сохраняет информативные разбиения, которые были бы потеряны при one-hot-кодировании. На большинстве соревновательных табличных задач LightGBM стабильно входит в ансамбли топовых решений.
- **Работа с категориями «из коробки».** Не нужно вручную проектировать one-hot-представление, алгоритм сам находит осмысленные группировки, что экономит время и улучшает результат, особенно при высокой размерности признаков.
- **Гибкость и богатая экосистема.** Поддержка кастомных метрик и лоссов, режим случайного леса, продвинутые опции вроде GBDT-PL и CEGB, а также возможность экспорта деревьев в DataFrame делают LightGBM удобным как для быстрого прототипирования, так и для глубокого анализа моделей.

Ограничения, о которых важно помнить

- **Риск переобучения leafwise.** Жадный рост без контроля может породить излишне глубокие ветви. Основными параметрами контроля сложности leafwise-дерева являются num_leaves и min_data_in_leaf. max_depth можно использовать как дополнительное жёсткое ограничение, но задавать его необязательно.

- **GOSS не универсален.** Метод ускоряет обучение, но при неудачном выборе `top_rate` и/или `other_rate` или на малых датасетах может ухудшить обобщение. По умолчанию он отключён именно по этой причине.
- **EFB эффективен только при наличии разреженности.** На полностью плотных числовых признаках выигрыш от бандлинга близок к нулю, хотя и вреда он не приносит за исключением очень быстрой проверки на возможность сжать категории в бандлы.
- **Категориальные сплиты требуют настройки.** Параметры `cat_smooth`, `cat_l2`, `max_cat_threshold` и `min_data_per_group` критически влияют на поведение для редких категорий. Без их осмысленного подбора возможны шумные разбиения и переобучение на случайных флуктуациях.

Таким образом, LightGBM — это мощный, сбалансированный инструмент, который при правильной настройке гиперпараметров способен стать основным рабочим инструментом для подавляющего большинства табличных задач.

4. Библиотека CatBoost: как предотвратить утечку данных в градиентном бустинге

> Полезные материалы
>
> - Оригинальная статья CatBoost: unbiased boosting with categorical features
> - И ещё оригинальная статья CatBoost: gradient boosting with categorical features support
> - Официальный репозиторий Github

CatBoost создавался как алгоритм, нацеленный на борьбу с **prediction shift** — переобучением, вызванным тем, что при построении деревьев многократно используется информация о целевой переменной одних и тех же объектов. Эта проблема проявляется и на этапе кодирования категориальных признаков, и при вычислении градиентов. CatBoost решает обе с помощью **ordered-философии**, но применяет её по-разному в двух режимах — plain и ordered.

Чтобы не запутаться в деталях, мы пройдём по следующему плану.

1. **Базовое устройство дерева, веса листьев и скоринг сплитов.** Сначала разберём фундамент, не зависящий от ordered-идей: только L2-регуляризованный лосс (в отличие от библиотек выше, где есть L1 и штраф на глубину дерева), симметричные «ленивые» деревья (Oblivious Decision Trees), вычисление оптимальных весов в листьях и скоринг сплитов через похожесть на антиградиент. Эти механизмы будут работать и в plain-, и в ordered-режиме.

2. **Утечка при кодировании категориальных признаков и ordered target statistics.** Затем перейдём к главной «визитной карточке» CatBoost — «честному» преобразованию категорий в числа. Покажем, почему наивные способы (Greedy TS, Leave-one-out TS) допускают утечку целевой переменной, и как ordered target statistics с несколькими случайными перестановками строят числовые представления, не используя таргет текущего объекта. Здесь же рассмотрим plain-режим, где ordered-философия ограничивается только кодированием категорий.
3. **Prediction shift при подсчёте градиентов и ordered-режим.** Наконец, доберёмся до более глубокой утечки: даже при «честных» категориях градиенты всё ещё вычисляются по предсказаниям, которые уже «видели» правильные ответы. Разберём, как ordered-режим пытается устранить этот источник переобучения, и почему на практике он используется заметно реже.
4. Обсудим применение Catboost на практике.

Итак, переходим к базовому устройству дерева.

4.1. РЕГУЛЯРИЗАЦИЯ В ЛИСТЬЯХ

В CatBoost лосс «дополнен» L2-регуляризацией на веса в листьях, что очень похоже на функцию потерь из XGBoost и LightGBM.

#1. Лосс и веса

все веса k-го дерева

$$L^{(k)} = \sum_{i=1}^{N} L\left(y_i,\ y_pred_i^{(k)}\right) + \frac{1}{2}\lambda \left\| \mathbf{w}^{(k)} \right\|_2^2$$

Но, как можно видеть, штрафа на добавление нового листа и L_1 регуляризации у данной библиотеки нет из-за особенностей построения деревьев по одной и той же переменной на весь уровень, подробнее поговорим об этом ниже. То есть структура дерева фиксирована, остается только контролировать веса, чтобы не переобучаться. В XGBoost и LightGBM структура контролируется через α и γ.

В CatBoost рассчитать оптимальный вес в листьях на основе функции потерь можно двумя способами:

- разложить лосс в ряд Тейлора до первого члена (`leaf_estimation_method='Gradient'`, быстрее, но менее точно);
- разложить лосс в ряд Тейлора до второго члена (`leaf_estimation_method='Newton'`, медленнее, но точнее).

Формулы для весов будут выглядеть так (то есть аналогично XGBoost и LightGBM).

→ $$w_j^{(k)} = -\eta \frac{\sum_{i \in R_j} g_i^{(k)}}{\sum_{i \in R_j} h_i^{(k)} + \lambda}$$ if leaf_estimation_method='Newton'

→ $$w_j^{(k)} = -\eta \frac{\sum_{i \in R_j} g_i^{(k)}}{|R_j| + \lambda}$$ if leaf_estimation_method='Gradient'

\+ leaf_estimation_iterations > 1

Также есть возможность задать параметр `leaf_estimation_iterations` > 1, чтобы ещё несколько раз пересчитать веса градиентным спуском. Это займёт больше времени, но результат будет точнее. Разложение Тейлора даёт приближение нашей функции потерь, то есть в алгоритме заложена хоть и маленькая, но ошибка. Градиентный спуск позволяет уточнить решение, оптимизируя настоящий лосс, а не приближение.

4.2. ЛЕНИВЫЕ ДЕРЕВЬЯ

Теперь перейдём к тому, как устроены сами деревья в CatBoost. Их особая форма — симметричные (Oblivious) деревья — объясняет, почему библиотека обходится без штрафов на количество листьев и L1-регуляризации. «Ленивость» этих бинарных деревьев в том, что у них во всех вершинах одного уровня глубины выбирается один и тот же сплит.

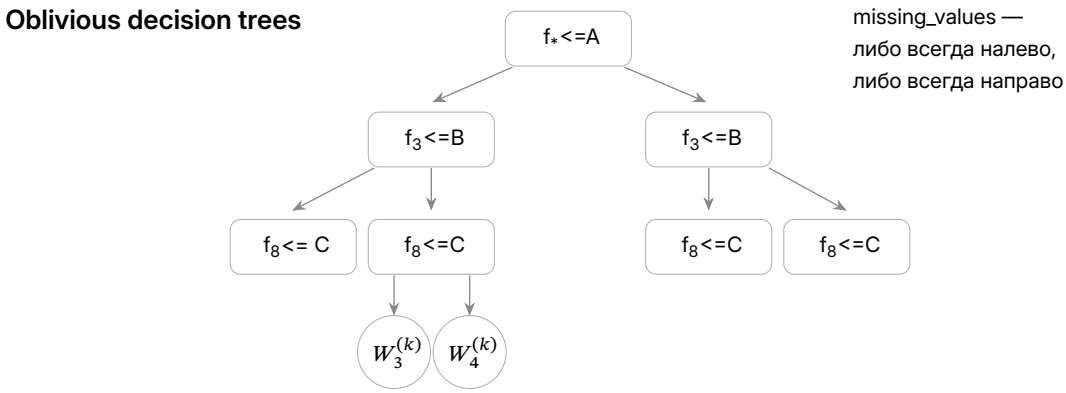


Рис. 18. Oblivious Decision Trees

Такой подход позволяет ускорить построение деревьев, так как необходимо перебирать меньше порогов и переменных на одном уровне построения дерева, что очень важно для ускорения работы идей медленной ordered-философии, о которой поговорим ниже.

4.3. СКОРИНГ СПЛИТОВ — ОЦЕНКА СХОЖЕСТИ С АНТИГРАДИЕНТОМ

В основе поиска сплитов CatBoost лежит уже знакомый гистограммный подход: числовые признаки и TS-признаки, полученные из категориальных, предварительно разбиваются на бины, и в качестве порогов предлагаются значения этих бинов. Два аспекта делают сплиты CatBoost особенными: стратегия обработки пропусков и три типа самих сплитов.

Если в данных есть пропущенные значения, они всегда отправляются либо направо, либо налево по выбору пользователя (гиперпараметр `nan_mode`). При этом алгоритм гарантированно рассматривает сплит, отделяющий пропуски от всех остальных значений. Это отличается от XGBoost и LightGBM, где пропуски адаптивно направляются в «лучшую» ветку.

CatBoost поддерживает три типа сплитов:

- числовой — $f_{num} \leq A$;
- категориальный (One-hot split) — $f_{cat} == A$ (если количество уникальных категорий в f_{cat} не превосходит `one_hot_max_size`);

- категориальный (TS-split или Target Statistics) — $f_{cat_TS} \leq A$ (если количество уникальных категорий в f_{cat} превосходит `one_hot_max_size`). Уникален для catboost на момент написания лонгрида.

Для выполнения TS-split категориальный признак сначала преобразуется в числовой с помощью target statistics (статистик, зависящих от таргета), применяя ordered-философию. Детально этот механизм мы разберём в разделе про Ordered Target Statistics. Затем для полученного числового признака ищется обычный числовой сплит.

> Полезные материалы
>
> - Варианты статистик для категориального сплита
> - Схемы квантизации, описанные в разделе Quantization

То есть сами возможности построения сплита отличаются от XGBoost и LightGBM не кардинально: имеется немного другая обработка пропусков и честная кодировка категорий. В остальном же выбор оптимального сплита похож — оцениваем качество всех возможных сплитов и выбираем лучший.
Но вот оценка качества строится уже не особой идее: теперь мы ищем сплит, не максимизирующий сплит, а максимизирующий схожесть с антиградиентом. Разберем этот момент поподробнее.

4.3.1. Оценка качества сплита: Gain vs. антиградиент

Рассмотрим скоринг на примере числового признака, так как для категориальных процесс аналогичный. Сначала мы построим пример ленивого дерева, а затем сравним, чем отличается выбор оптимальных весов по приросту Gain и схожести с антиградиентом.

Выбираем сплит на втором уровне глубины и проскорим его (f, A). Сплит для каждой вершины уровня глубины предлагается одинаковый, поэтому после сплита (f, A) объекты «распадутся» на 4 вершины. Чтобы проскорить сплит, возьмём случай, при котором вершины — это листья нашего дерева с весами $w_1^*, w_2^*, w_3^*, w_4^*$.

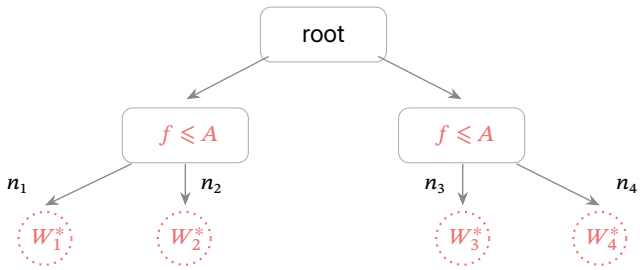


Рис. 19. Построение весов в ODT

Теперь каждому объекту присвоен вес соответствующего листа, а мы получим вектор длины N. Например, $\vec{w} = (w_1^*, \dots, w_4^*)$, где на i-й позиции будет вес листа, куда попал i-й объект после сплита. В зависимости от алгоритма мы по-разному используем N:

- в «классическом» алгоритме с помощью этого вектора мы считали Gain (разницу лоссов до и после сплита);

- в CatBoost решили сделать по-другому, опираясь на то, что лосс убывает в сторону своего антиградиента.

Разберём разницу подходов на двух рисунках ниже: на плоскости показан случай, когда у нас всего два объекта (для рисунка на плоскости). Серым цветом обозначим линии уровня лосса, который зависит от вектора $\vec{w}$ — вектора предсказаний k-го дерева на рассматриваемых объектах, то есть значений листьев, в которые попали эти объекты.

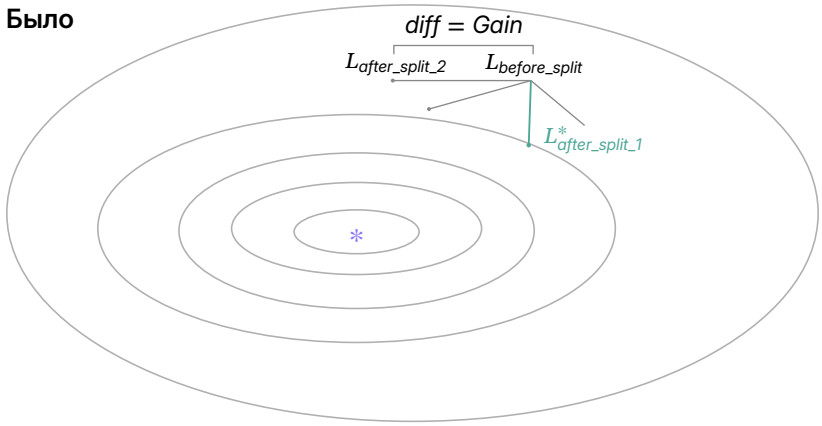


Рис. 20. Мера качества сплита — Gain

У нас есть лосс до сплита, $L_{before}(w_1, w_2)$ и лоссы после каждого предлагаемого сплита. Ранее мы считали разницу между сплитами — Gain (и побеждал сплит с максимальной положительной разницей). То есть в этом подходе опираемся на значения лоссов в точках $\vec{w}$, получаемых после сплитов.

В CatBoost мы выбираем сплит, вектор $\vec{w}$ после которого больше всего похож на антиградиент (то есть смотрим именно на сами точки $\vec{w}$, а не значения лосса в них).

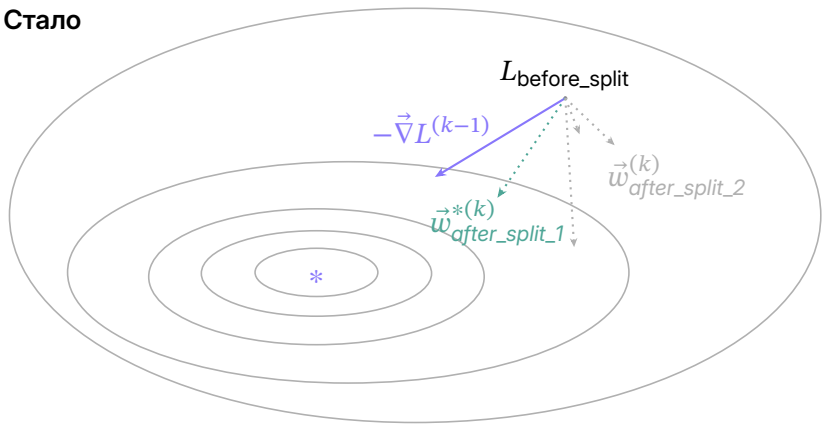


Рис. 21. Мера качества сплита — оценка похожести на антиградиент лосса

Обозначения:

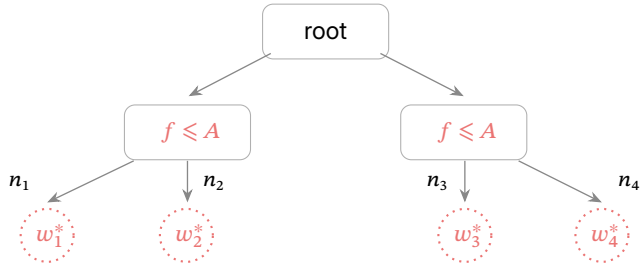
- фиолетовым цветом отмечен вектор антиградиента лосса;
- w_1, w_2, полученные после сплитов, показаны векторами, а не точками, потому что теперь мы ищем вектор сдвига.

Именно эти векторы мы будем сравнивать на похожесть с антиградиентом и выбирать тот сплит, вектор $\vec{w}$ после которого наиболее похож.

ВАЖНО

Подробнее вопрос можно изучить в разделе Score functions, где описаны функции для сравнения с антиградиентом.

Пример подсчёта похожести через L2-меру (одна из популярных Score-функций)



$$L_2 = \sum_{i \in R_1} \left(w_1^* + g_i\right)^2 + \sum_{i \in R_2} \left(w_2^* + g_i\right)^2 + \sum_{i \in R_3} \left(w_3^* + g_i\right)^2 + \sum_{i \in R_4} \left(w_4^* + g_i\right)^2$$

$$L_2 \to \min_{f,A}$$

Рис. 22. Пример подсчёта L2-похожести на антиградиент лосса

Обрати внимание, мы измеряем схожесть с антиградиентом, поэтому g_i идут со знаком плюс.

ВАЖНО

В документации раздела Score functions в этом месте опечатка. Вообще в этом разделе больше всего опечаток и неточностей, поэтому рекомендуем сверяться с лонгридом. Некоторые вопросы мы уточняли напрямую у разработчиков.

4.3.2. Зачем сравнивать именно с антиградиентом?

На первый взгляд может показаться, что это просто другая математическая формулировка одной и той же задачи. Но за этим стоит важное концептуальное различие в том, как алгоритм «смотрит» на процесс обучения.

- **Подход Gain (XGBoost/LightGBM)** ориентирован на *результат*: «Насколько абсолютно уменьшился лосс после этого разделения?». Это прагматичный подход, но он может быть чувствителен к локальным шумам, если небольшое уменьшение лосса достигается за счёт движения в «неправильном» направлении. Также мы получаем интерпретируемую меру важности каждого признака.
- **Подход CatBoost** ориентирован на *направление*: «Двинулись ли мы в сторону наискорейшего спуска?». Оценивая схожесть (например, через скалярное произведение или L2-расстояние), алгоритм гарантирует, что новый сплит не просто немного уменьшает лосс, а делает это максимально согласованно с общим вектором антиградиента. Такой подход в теории делает обучение более стабильным и менее склонным к переобучению. Однако у данного подхода есть небольшие недостатки.
  - В теории возможно пропустить сплит с большим мгновенным выигрышем, если его направление расходится с антиградиентом.
  - Вычисление меры схожести (L2-расстояния или косинусного сходства) требует ресурсов, хоть и крайне малых из-за для гистограммного подхода.

4.3.3. Итоги построения деревьев

Итак, CatBoost строит симметричные (Oblivious) деревья фиксированной глубины с одним и тем же сплитом на каждом уровне. Такой выбор не случаен: он ускоряет обучение и позволяет обойтись без штрафов на количество листьев и L1-регуляризации — сложность контролируется глубиной и минимальным числом объектов в листе. Веса листьев оптимизируются через L2-регуляризованный лосс, причём можно выбирать между быстрым градиентным и более точным ньютоновским методом, а при необходимости — доуточнять их градиентным спуском.

При поиске сплитов CatBoost опирается на гистограммный подход, но с важным отличием. Лучший сплит определяется не по наибольшему уменьшению лосса (Gain), а по тому, насколько хорошо получившийся вектор весов совпадает с направлением антиградиента. Это смещает фокус с немедленного снижения ошибки на согласованность шага с глобальным спуском и служит дополнительной регуляризацией.

Чтобы поставить точку в разборе алгоритма построения деревьев, нужно понять, как получать числовые признаки из категориальных без утечек целевой переменной. Мы говорили выше, что это делается с помощью так называемых Target Statistics (TS) и применения ordered-философии.

4.4. PREDICTION SHIFT

Создатели библиотеки заметили в «классическом» алгоритме переобучение, которое назвали **prediction shift** (иногда называют **conditional shift**) и связали со следующими особенностями:

- g_i, h_i, на которых строится всё дерево для объекта x_i, считаются на основе предсказания предыдущих деревьев. Каждое из них «подглядывало» в таргет y_i, накапливалась «утечка»;
- в классическом алгоритме категориальные признаки преобразовываются в числовые «на лету», используя те же g_i, h_i. Это тоже ведёт к prediction shift.

Эти проблемы создатели библиотеки назвали target leakage и решили с ними бороться, начав со второго пункта. Для начала заменили gradient statistics на target statistics, то есть решили преобразовывать категории в числа, используя лишь информацию о таргете, а не о g_i, h_i. Это не решило проблему, но это был первый шаг.

Авторы пробовали заменить gradient statistics на target statistics, используя три разных подхода:

- Gredy TS,
- Leave-one-out TS,
- Ordered Target Statistics.

Наивный подход Greedy TS — замена категориального значения (например, «A») на средний таргет по всем объектам с таким же значением. Естественно, такой подход ведёт к target leakage, так как для подсчёта TS для i-го объекта используется его метка y_i.

В подходе Leave-one-out TS выяснилось, что target leakage также будет присутствовать, даже если считать TS для i-го объекта без его метки y_i (выкидываем только i-й объект). Несмотря на то, что непосредственно сам таргет наблюдения i не участвует в вычислении статистики, утечка всё равно происходит, так как предыдущие деревья в ансамбле уже запомнили информацию об i-м объекте. То есть другие объекты уже несут косвенную информацию об этом i-м объекте.

В итоге авторы пришли к подходу Ordered Target Statistics, основанному на ordered-философии.

4.5. ORDERED-ФИЛОСОФИЯ ДЛЯ БОРЬБЫ С TARGET LEAKAGE

Рассмотрим пример, в котором нам нужно посчитать f_{cat_TS} для i-го объекта. Мы выяснили, что подходы Greedy TS и Leave-one-out TS нам не подходят. Тогда используем Ordered Target Statistics.

1. Отсортируем объекты согласно некоему известному заранее отношению порядка на объектах.
2. Используем для подсчёта TS для i-го объекта только значения таргета тех объектов, которые идут раньше него по порядку.
   В качестве такого отношения порядка может служить колонка со временем, если она есть в данных (см. рисунок ниже). Например, время заказа или дата сделки по недвижимости. Ситуацию, когда такой колонки нет, мы рассмотрим позже.
   То есть мы по прошлым объектам кодируем будущие. Утечки не будет возникать, потому что на на момент существования прошлых объектов будущих ещё не существовало.

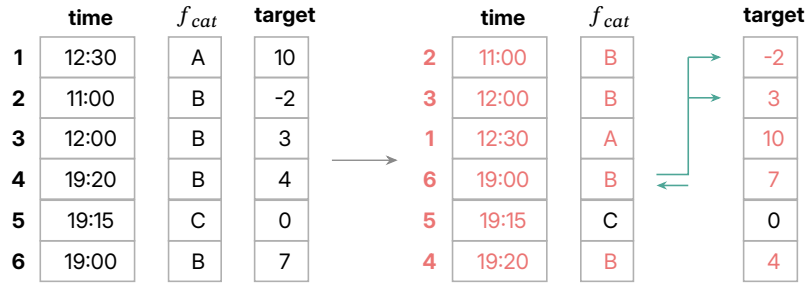


Рис. 23. Пример применения ordered-философии

3. Пересортируем объекты по времени (предположим, что времени заказа). Получим столбец f_{cat}, где порядок объектов уже «имеет какое-то значение».
4. Применяем метод **Mean Target** для объекта с зелёной стрелочкой (4-го по новой сортировке, 6-го — по старой). В подсчёте его TS используем только метки объектов,
   - у которых значение $f_{cat} = B$;
   - позиция раньше нашего объекта в списке (метки объектов 2 и 3 по старой нумерации).

   В этом случае:

- для первых объектов в списке не будет подходящих кандидатов;
- при расчёте у ранних объектов будут шумные TS, поскольку они будут подсчитаны по небольшому количеству объектов (это мы сгладим, добавив регуляризационные константы в формулы);
- в результате каждый объект в общем случае получит своё уникальное числовое представление для признака f_{cat}.

Важно помнить следующее.

- Алгоритм, появившийся в LightGBM, кодирует конкретное уникальное значение признака f_{cat} в конкретное число (идёт сопоставление $f_{cat\ value} \to TS_{value}$), а в Ordered TS подходе каждому отдельному значению будут соответствовать разные числа, при этом для элементов одной категории они будут похожи по значению (то есть $f_{cat\ i} \to TS_i$).
- В ordered-подходе категориальные признаки кодируются один раз перед построением деревьев, после чего полученные TS не меняются.
  В классическом алгоритме сопоставления $f_{cat\ value} \to TS_{value}$ будут отличаться в каждой вершине, где выбирается сплит, поскольку вычисляются на лету на основе g_i, h_i объектов, которые находятся в вершине.

Такой ordered-подход устраняет target leakage, что снижает prediction shift: объекты «подглядывают» только в те метки, которые они уже «видели». Сильно углубляться не будем, детально изучить этот вопрос можно в статьях, хотя всё это имеет скорее эмпирический характер — на практике описанный подход работает хорошо.

Резюмируем: ordered-философия — это подход, в котором объекты сортируются согласно какому-то заданному порядку. После этого подсчёт всех TS (как мы увидим далее, не только TS) происходит так, как будто i-й объект «видит» только те объекты, которые стоят раньше него по списку.

Но важно отметить, что подход работает, только когда порядок можно предъявить. В следующем блоке разберём случаи, когда это невозможно.

4.6. PLAIN-РЕЖИМ — ПОДСЧЁТ ORDERED TS ДЛЯ КАТЕГОРИЙ

Как говорилось ранее, в CatBoost реализованы два алгоритма обучения: plain-режим (используется в библиотеке по умолчанию) и ordered-режим.

Plain подключает ordered-подход только для кодирования категорий в числа, после чего работает по описанным выше принципам.

При этом возможны два случая.

1. Мы объявляем `has_time=True` и предъявляем колонку «время», по которой задаётся порядок (это могут быть любые колонки, у которых есть смысл порядка — например, id, time или date). В этом случае категории кодируются в TS по установленному порядку, после чего начинается процесс построения деревьев.
2. `has_time=False` (по умолчанию). Тут начинается веселье! В этом случае мы признаём, что у нас нет колонки, задающей порядок, или мы не используем её. Вся оставшаяся часть блока будет посвящена такому случаю.

Самый простой способ получить порядок на объектах, когда его нет, — выдумать его! Для этого достаточно сгенерировать случайную перестановку индексов от 0 до $N-1$ и пересортировать объекты по ней. После этого можно было бы притвориться, что у нас случай `has_time=True` и успокоиться.

- Но такой порядок взялся из ниоткуда, и в нём нет смысла.
- Качество кодирования категорий по такому порядку будет довольно низкое.

Чтобы решить эту проблему, нужно сделать следующее.

1. Сгенерировать 4 случайных перестановки (назовём их σ_r).
2. Создать и хранить 4 варианта кодировок категориальных признаков.
   На рисунке ниже нулевая перестановка выделена красным цветом, поскольку выполняет «специальную функцию».

ВАЖНО

В статье говорится о произвольном количестве перестановок $s+1$, но разработчики сказали, что на практике они используют именно 3 + 1.

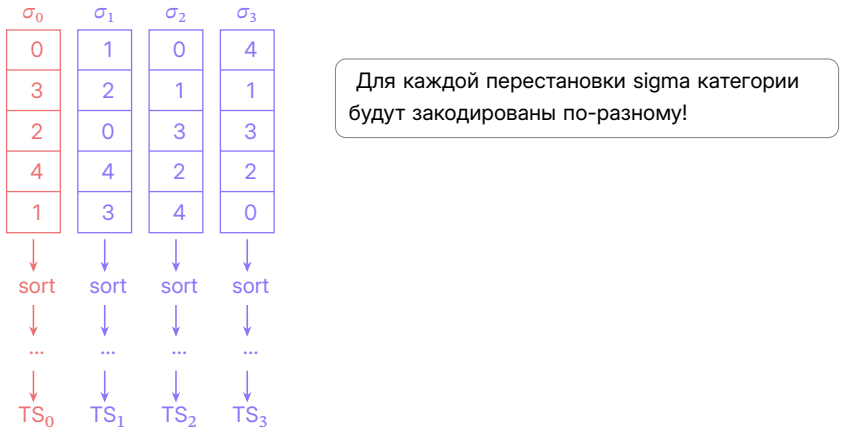


Рис. 24. Построение случайных перестановок

И что делать с этими четырьмя TS для каждого категориального признака?

Казалось бы, можно просто усреднить TS всех перестановок, получить итоговые TS и строить деревья, но авторы выбрали другой путь.

3. Для каждого нового дерева структура строится по случайной перестановке из $\sigma_1, \sigma_2, \sigma_3$.
4. Для всех перестановок (включая 0) по построенной структуре «спускаются» объекты и считаются веса в листьях.

Разберём пример на схеме.

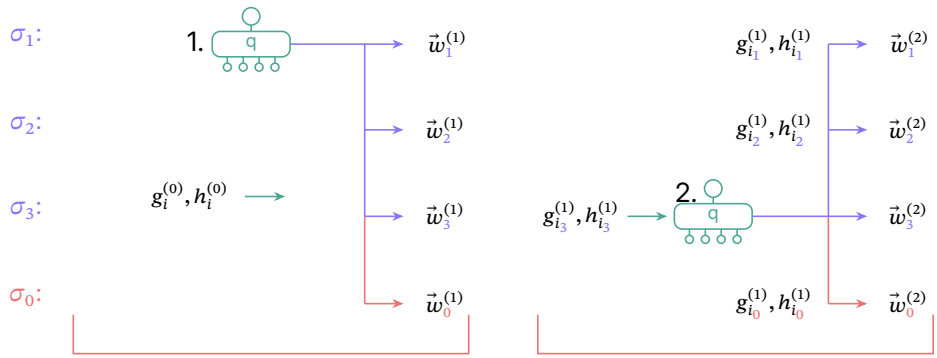


Рис. 25. Схема построения ансамбля для Plain-режима при has_time=False

Допустим, случайно выбралась первая перестановка σ_1 для построения первого дерева. Это значит, что вся структура дерева (сплиты) строится, используя TS_1 вариации кодирования всех категориальных признаков, а также $g_i^{(0)}, h_i^{(0)}$, посчитанные именно по этой перестановке (что это значит — расскажем дальше). Поскольку первое дерево отталкивается от начального приближения, одинакового для всех перестановок, градиенты и гессианы на этом шаге у всех перестановок совпадают — поэтому сейчас неважно, что они взяты по σ_1, они такие же, как по σ_2, σ_3 и σ_0. Но для последующего дерева у каждой перестановки будут свои градиенты и гессианы (что отражено на картинке).

Когда все сплиты первого дерева зафиксированы, мы вычисляем веса в его листьях отдельно для каждой перестановки.

Посчитать веса по перестановке σ_r означает «спустить» все объекты, у которых категориальные признаки закодированы вариацией TS_r, в соответствующие им листы, после чего посчитать веса по выведенной ранее формуле.

Так как сплиты уже фиксированы, но для каждой перестановки TS_r для категориальных признаков отличаются, объекты в зависимости от перестановки будут спускаться в разные ветки на сплитах вида $f_{cat_TS} \leq A$. Так, например, по первой перестановке у i-го объекта $f_{cat_TS1} = 14$, а по второй $f_{cat_TS2} = -1$, и в первом случае объект «спустился» бы вправо, а во втором — влево. Таким образом, в листьях в зависимости от перестановки будут оказываться разные объекты, что приведёт к разным значениям весов.

Также для подсчёта весов r-й перестановки используются g_{ir}, h_{ir}, посчитанные по ней. Это означает, что в формуле, например, $g_{ir} = L'(y_i, \hat{y_{ir}})$ на место предсказаний мы подставляем предсказания, полученные по r-й перестановке, то есть сумму весов, посчитанных именно по ней. Мы можем это сделать, поскольку на каждой итерации считаем веса по всем перестановкам.

Таким образом, для каждой из перестановок $\sigma_1, \sigma_2, \sigma_3$ есть вектор весов длины N, где каждому объекту соответствует значение веса в листе, куда он спустился в рамках структуры дерева, используя f_{cat_TSr} в качестве кодирования категориальных признаков f_{cat}. Мы считаем веса для перестановок $\sigma_1, \sigma_2, \sigma_3$, поскольку дальше эти перестановки тоже будут использоваться для построения структуры конкретного дерева, для чего нам понадобятся g_{ir}, h_{ir}, которые зависят от предсказаний предыдущих деревьев на объектах. Эти предсказания мы будем считать, используя веса выпавшей перестановки. Например, если следующей выпавшей случайно перестановкой будет σ_3, то для построения второго дерева мы будем использовать $g_{i3}^{(1)} = L'(y_i, w_{i,3}^{(1)})$, $h_{i3}^{(1)} = L''(y_i, w_{i,3}^{(1)})$.

4.6.1. Роль нулевой перестановки

После того как дерево построено и веса посчитаны для всех перестановок, алгоритм сохраняет структуру дерева и считает веса по нулевой перестановке σ_0. Именно эта пара — структура на основе TS_1 и веса по σ_0 — будет использоваться для предсказаний на новых данных в методе `model.predict()`.

Нулевая перестановка играет роль «чистой» или «эталонной»: она никогда не используется для построения структуры деревьев, поэтому веса, посчитанные по ней, не страдают от переобучения на конкретный случайный порядок. Это аналог hold-out-выборки в кросс-валидации.

ВАЖНО

Для выборки, поданной в качестве аргумента в метод `model.predict(…)`, TS для категориальных признаков считаются по всем объектам обучающей выборки, то есть так, как если бы каждый объект этой выборки стоял по порядку сразу после последнего объекта из обучающей. Каждый тестовый объект «видит» все объекты обучающей выборки.

Повторяя описанную выше процедуру и сохраняя структуры и веса, пока не сработает критерий остановки построения ансамбля, мы получим итоговую Plain-модель. Предсказание для новых объектов будет осуществляться по построенным структурам и весам нулевой перестановки. По сути всё это ради того, чтобы «честным» образом закодировать категориальные признаки. На практике CatBoost обычно работает лучше XGBoost и LightGBM в задачах с существенным количеством важных категориальных признаков.

Для закрепления знаний о plain-алгоритме рассмотрим на примере параметр, используемый по умолчанию, — `has_time=False`.

```
sigma_0, sigma_1, sigma_2, sigma_3 = random_permutations(N)
num_hists = create_numeric_histograms(X)
```

```
cat_TS = []
y_pred = []
for j in range(4):
    cat_TS[j] = create_TS(cat_features, sigma_j) # создание TS для всех кат. признаков
    y_pred[j] = y_pred_init

cat_hists = create_numeric_histograms(cat_TS) # TS тоже бьём на бины


for k in range(np.inf):
    if stopping_criterion():
        break
    for j in range(4):
        g[j], h[j] = calc_grad_hess(y_pred[j])

    r = random([1, 2, 3]) # строим структуру по r-й перестановке
    tree_structure = build_tree(num_hists, cat_hists[r], g[r], h[r])

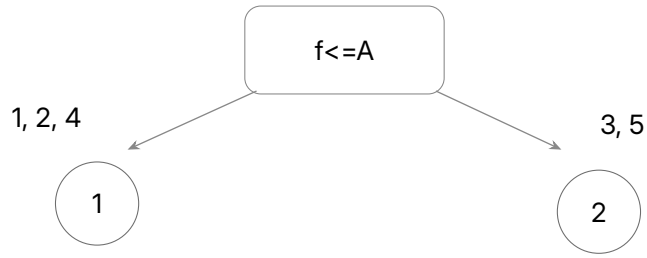
    for j in range(4):
        w[j] = calc_leaf_weights(tree_structure, num_hists, cat_hists[j], g[j], h[j])
        y_pred[j] = y_pred[j] + w[j] # y_pred[j], w[j] - векторы длины N

    save_tree(tree_structure, w[0]) # for prediction on new data
```

4.7. ORDERED-РЕЖИМ

В plain-режиме мы считали TS для категориальных признаков, используя ordered-философию, то есть «честно», без target leakage. Но есть проблема: «градиенты» и «гессианы» всё ещё считаются «нечестно», поскольку «чересчур» опираются на собственные метки. Если чуть более углубиться в детали, то, например, перед построением k-го дерева имеем $g_i^{(k-1)} = L'(y_i, \hat{y}_i^{(k-1)})$. По мнению авторов библиотеки, проблема здесь в том, что мы используем одновременно y_i и $\hat{y}_i{}^{(k-1)}$, который на самом деле тоже зависит от y_i, то есть получается что-то вроде «рекурсивного подглядывания» в таргет. В то время как от использования y_i для подсчёта g_i нам никуда не деться, авторы предлагают использовать ordered-философию для того, чтобы предсказание для i-го объекта $\hat{y}_i{}^{(k-1)}$ сильно меньше зависело от y_i.

Идею ordered-режима опишем коротко ниже, так как подробное описание очень громоздкое.



Было $M: \quad [w_1, w_1, w_2, w_1, w_2]$

Стало $M_1: \quad [\varnothing, -g_1(1), \ 0, -g_1(1), \ 0]$

$$M_2: \quad [\varnothing, \ \varnothing, \ 0, -\frac{g_2(1)+g_2(2)}{2}, \ 0]$$

$$M_3: \quad [\varnothing, \ \varnothing, \ \varnothing, \ -\frac{g_3(1)+g_3(2)}{2}, \ -g_3(3)]$$

$$M_4: \quad [\varnothing, \ \varnothing, \ \varnothing, \ \varnothing, \ -g_4(3)]$$

$$M_5: \quad [\varnothing, \ \varnothing, \ \varnothing, \ \varnothing, \ \varnothing]$$

Рис. 26. Ordered-режим на пальцах

В plain-режиме решающий пень на рисунке выше (предположим, построенный на какой-то перестановке σ_r) «генерировал» бы одну «модель», которая являла бы собой вектор скоров на объектах, где каждому объекту был бы сопоставлен вес листа, в который он «упал» по этой структуре. При этом каждый объект получил бы скор либо w_1, либо w_2 в зависимости от листа.

В ordered-режиме такая структура будет «генерировать» N «моделей» по числу перестановок, каждая из которых будет имитировать «модель» из plain-режима с условием того, что i-я модель «знает» только о существовании первых i объектов перестановки включительно. Например, модель M_3 будет в процессе всего обучения строиться так, что будет использовать явно только информацию об объектах $1, 2, 3$. Это нужно для того, чтобы в процессе обучения «градиенты» и «гессианы» i-го объекта были посчитаны на модели M_{i-1}, то есть, например, для «градиентов» $g_i \to g_{i-1}(i) = L'(y_i, M_{i-1}(i))$. Если действовать так, то получается, что второй аргумент производной $M_{i-1}(i)$ теперь не так явно зависит от y_i, поскольку эта «модель» строилась лишь на первых $i-1$ объектах, поэтому алгоритм становится честнее. Вся эта история с поддерживанием N «моделей» распространяется ещё и на каждую перестановку, кроме σ_0.

Мы обрисовали ordered-режим очень поверхностно, на уровне идеи, поскольку на практике он работает очень долго и почти всегда эта «честность» не приводит к заметным улучшениям результата, однако советуем попробовать этот режим на очень малых выборках (<10k объектов), поскольку prediction shift в таких случаях проявляется заметнее, и «честность» может дать ощутимый эффект.

Для того чтобы включить ordered-режим, нужно задать аргумент `boosting_type='Ordered'`.

4.7.1. Итоги Ordered-философии

Итак, CatBoost вырос из попытки решить проблему prediction shift — переобучения, вызванного тем, что и кодирование категорий, и вычисление градиентов в классическом бустинге многократно используют одни и те же метки объектов. Ordered-философия, предложенная авторами, заключается в том, чтобы для каждого объекта учитывать только «прошлое» относительно него — те объекты, которые предшествовали ему в некотором заданном порядке.

Для кодирования категориальных признаков этот подход воплотился в Ordered Target Statistics. Вместо того чтобы кодировать категорию одним числом для всех, CatBoost генерирует несколько случайных перестановок и считает TS так, будто каждый объект видит только предшествующие ему в данной перестановке. Благодаря этому таргет текущего объекта не участвует в вычислении его target statistics, что снижает прямую утечку при кодировании категорий. На практике используется plain-режим (по умолчанию): структура каждого дерева строится по одной из трёх рабочих перестановок, а веса листьев вычисляются по всем перестановкам. Для финального предсказания сохраняются веса, полученные на четвёртой, «эталонной» перестановке, которая не участвовала в выборе сплитов. Именно такой механизм делает CatBoost особенно сильным на данных с большим числом категориальных признаков.

Для градиентов ordered-подход реализован в ordered-режиме. Здесь для каждой перестановки поддерживается N моделей, и градиент для i-го объекта вычисляется только на основе модели, обученной на первых $i-1$ объектах. Это ещё сильнее снижает зависимость предсказания от собственного таргета объекта, но требует существенно больших вычислительных затрат. На практике ordered-режим почти не используется, так как выигрыш от дополнительной «честности» заметен лишь на очень малых выборках, а время обучения возрастает многократно.

Таким образом, главный практический итог этого раздела таков: основное оружие CatBoost против переобучения — это честное кодирование категорий через случайные перестановки (plain-режим), а ordered-режим остаётся нишевым инструментом для экстремально маленьких данных. Теперь, понимая, как CatBoost строит деревья и борется с утечкой, перейдём к практическим возможностям библиотеки.

4.8. ЭЛЕМЕНТЫ ПРАКТИКИ

Интерфейс CatBoost схож с другими библиотеками бустинга и включает почти все уже рассмотренные возможности LightGBM. Чтобы не повторяться, опишем только уникальные особенности — остальное разберём на семинаре.

4.8.1. Задание уточняющих параметров

Разберём пример. Нужно посчитать метрику M, у которой есть параметр a, отвечающий за степень некой скобки. В LightGBM такой параметр был бы задан в виде отдельного параметра (например, коэффициент регуляризации при обучении линейных регрессий в листьях).

```
params = {'metric': 'M', 'm_alpha': 2}
```

В CatBoost применяется «система уточняющих параметров», которые задаются через двоеточие (в той же строчке, что и основной).

```
params = {'metric': 'M:alpha=2'}
```

Может показаться, что так даже удобнее, но из-за этого возникают ситуации, когда у основного параметра появляется несколько уточняющих, и тогда можно наблюдать примерно следующее.

Format

```
['CtrType[:TargetBorderCount=BorderCount][:TargetBorderType=BorderType][:CtrBorderCount=Count][:CtrB
 'CtrType[:TargetBorderCount=BorderCount][:TargetBorderType=BorderType][:CtrBorderCount=Count][:Ct
 ...]
```

Рис. 27. Пример не самого удобного способа задавать уточняющие гиперпараметры

4.8.2. Генерируемые «на лету» комбинации признаков

На уровне глубины построения дерева d `CatBoost` может использовать всевозможные комбинации признаков размера `max_ctr_complexity`, которые были выбраны для сплитов ранее. Это важный гиперпараметр иногда может тормозить алгоритм, но привносит новые комбинации признаков, потенциально улучшающие его качество. Подробнее вопрос можно изучить в этой статье.

4.8.3. Деревья в виде датафрейма

Ансамбль `model.trees_to_dataframe`, сохранённый в виде датафрейма, — важнейший в LightGBM источник инсайтов о модели. В CatBoost нет аналогичного метода, но информацию можно получить в виде словаря хитрым способом: сохранить модель в формате JSON, затем загрузить обратно и постараться спарсить её после. Подробнее вопрос можно изучить в этом туториале.

4.8.4. Встроенные средства интерпретации модели

CatBoost включает встроенную оценку важности признаков и их взаимодействий на основе изменений предсказаний и лосса. Подробнее о том, как они считаются, можно прочитать в разделе Model analysis.

Эти данные можно получить из уже обученной модели методом `model.get_feature_importance()`. Ниже рассмотрим примеры, которые подробнее разберём на семинарах.

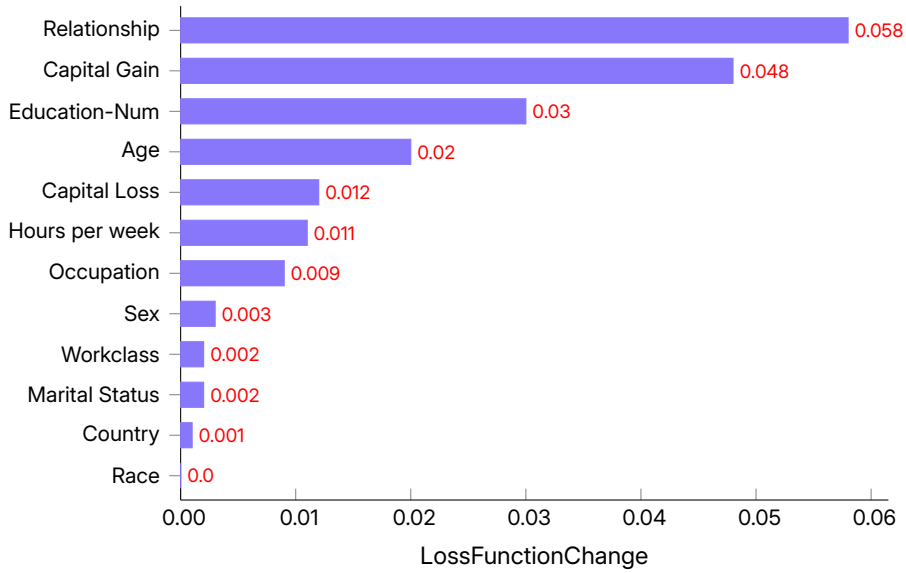


Рис. 28. Гистограмма важности признаков

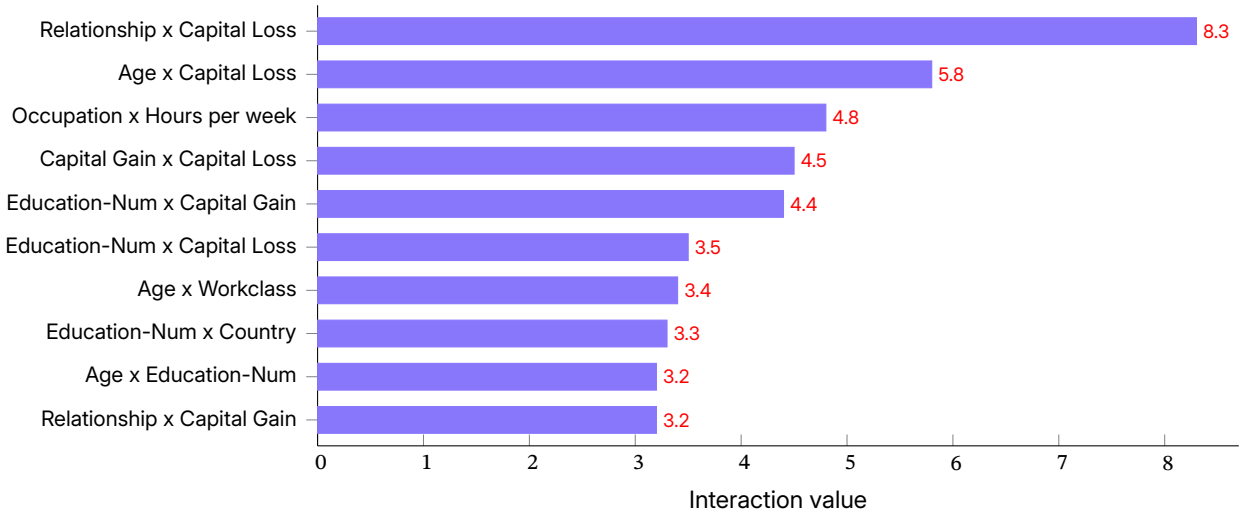


Рис. 29. Гистограмма важности для парных комбинаций категориальных признаков

4.8.5. CatBoost Plain vs CatBoost Ordered vs LightGBM

На рисунке ниже сравниваются скорости обучения модели на GPU и CPU (16 ядер).

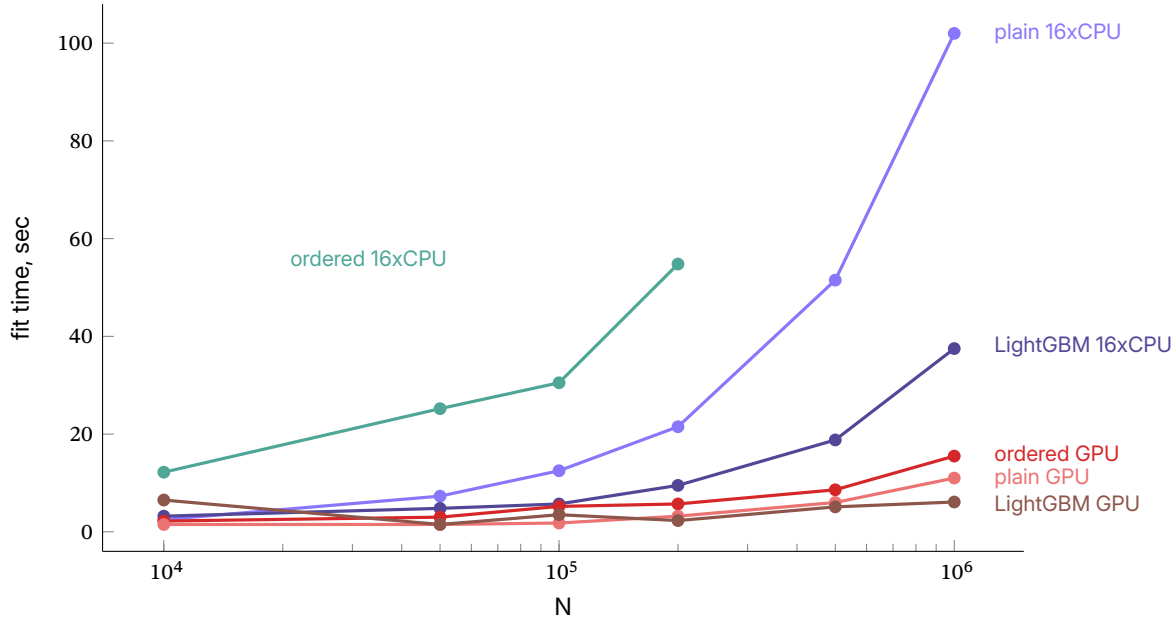


Рис. 30. Сравнение скорости работы LightGBM и Catboost на GPU и CPU в зависимости от объёма данных

Ordered-режим на CPU неконкурентоспособен, однако на GPU показывает себя вполне достойно. В целом, если есть возможность, рекомендуется использовать CatBoost на GPU, но нужно учитывать, что результаты работы на CPU и GPU будут отличаться.

ВАЖНО

В разделе Speeding up training подробно рассказано про различные способы ускорения.

Чем больше категориальных признаков, тем сильнее CatBoost будет проигрывать в скорости LightGBM. С другой стороны, чем больше категориальных признаков, тем больше хочется использовать CatBoost для более «качественной» обработки. При этом сравнение качества на одном конкретном датасете ничего не значит. Для каждого из алгоритмов можно найти задачи, где он будет обходить конкурента, поэтому при выборе лучше опираться на количество и качество категориальных признаков.

Признаки на текстах, эмбеддингах и графах

CatBoost поддерживает генерацию признаков из текстов, эмбеддингов и графов «из коробки». Если говорить о текстах и эмбеддингах, то делает он это не самым оптимальным образом. Но это хороший вариант, чтобы запустить модель с обработкой этих признаков «из коробки», после чего посмотреть на их важности. Если признаки важны, можно уже подключать ручную, более качественную предобработку этих данных.

Полезные материалы из документации

- Текстовые признаки
- Эмбеддинговые признаки
- Графовые признаки (граф задаётся в аргументе `graph` у `cb.Pool`)

4.8.6. Итоги «практики»

Практический инструментарий CatBoost покрывает многие повседневных нужд разработчика ML-моделей, при этом некоторые возможности уникальны и заслуживают отдельного внимания. Поддержка текстов, эмбеддингов и графов прямо «из коробки» позволяет быстро проверить гипотезу, не прибегая к внешним инструментам — это экономит время на старте, хотя для максимального качества часто требуется ручная предобработка. Генерация комбинаций категориальных признаков на лету может улучшить качество модели, но увеличивает время обучения, поэтому параметр `max_ctr_complexity` стоит настраивать аккуратно.

CatBoost также предоставляет встроенные средства интерпретации — важность признаков и анализ взаимодействий — без необходимости писать дополнительный код.

Отсутствие прямого метода `trees_to_dataframe`, как в LightGBM, компенсируется возможностью экспорта модели в JSON и последующего парсинга, хотя это требует чуть больше усилий.

С точки зрения скорости, CatBoost на CPU обычно проигрывает LightGBM, особенно при большом объёме данных, но на GPU разрыв сокращается, а ordered-режим на GPU может быть вполне конкурентоспособен. Основной рекомендацией остаётся: использовать CatBoost, когда в данных много категориальных признаков высокого качества, и по возможности обучать на GPU. Если же данные преимущественно числовые, а требования к скорости высоки, разумнее присмотреться к LightGBM или XGBoost.

Ниже приведён пример обучения CatBoost, демонстрирующий основные настройки и гиперпараметры, рассмотренные в этом разделе.

4.9. ПРИМЕР ИСПОЛЬЗОВАНИЯ CATBOOST

Принципы организации обучения в этой библиотеке аналогичны LightGBM. Чтобы использовать её, нужно обернуть обучающую выборку в специальную структуру данных `cb.Pool` и настроить гиперпараметры.

```
import catboost as cb

# ==================== 1. ПОДГОТОВКА ДАННЫХ ====================
# CatBoost использует специальную структуру Pool.
# Если в данных есть колонка времени (для Ordered-философии), можно
  передать has_time=True.
train_pool = cb.Pool(
    data=X_tr,
    label=y_tr,
    cat_features=['cat_feature_1', 'cat_feature_2'], # Обязательное указание!
    # has_time=True # Раскомментировать, если есть колонка времени для упорядочивания
)

# Для валидации передаём train_pool как reference.
# Это гарантирует, что бины для числовых признаков будут построены только на обучающих
  данных (без утечки).
val_pool = cb.Pool(
    data=X_val,
    label=y_val,
    cat_features=['cat_feature_1', 'cat_feature_2'],
    reference=train_pool
```

```
)

# ==================== 2. НАСТРОЙКА ГИПЕРПАРАМЕТРОВ ====================
params = {
    # ==================== ОСНОВНЫЕ ПАРАМЕТРЫ ====================
    'loss_function': 'Logloss',      # Функция потерь (аналог objective
      в других библиотеках)
    'learning_rate': 0.1,            # Скорость обучения бустинга
    'iterations': 1000,              # Максимальное количество деревьев (критерий
      останова ансамбля)
    'task_type': 'CPU',              # 'CPU' или 'GPU'. Важно: результаты на CPU и GPU
      могут отличаться!
    'thread_count': 8,               # Количество потоков для распараллеливания на CPU

    # ==================== СТРУКТУРА ДЕРЕВЬЕВ (ODT) ====================
    # CatBoost использует Oblivious Decision Trees (симметричные «ленивые» деревья).
    # Глубина фиксируется заранее, параметра num_leaves здесь нет.
    'depth': 6,
    # ==================== РЕГУЛЯРИЗАЦИЯ ====================
    # L2-регуляризация на веса листьев (аналог lambda в XGBoost).
    # Штрафует большие веса, предотвращая переобучение.
    # (L1-регуляризации и штрафа за количество листьев gamma в CatBoost нет).
    'l2_leaf_reg': 3.0,

    # ==================== ОЦЕНКА ВЕСОВ В ЛИСТЬЯХ ====================
    # Метод расчёта оптимальных весов:
    # 'Gradient' (быстрее, разложение Тейлора до 1-го члена)
    # 'Newton' (медленнее, но точнее, разложение до 2-го члена)
    'leaf_estimation_method': 'Newton',

    # Количество итераций градиентного спуска для уточнения весов.
    # Значение > 1 позволяет оптимизировать настоящий лосс, а не его приближение,
    # что даёт более точный результат ценой небольшого замедления.
    'leaf_estimation_iterations': 1,

    # ==================== КАТЕГОРИАЛЬНЫЕ ПРИЗНАКИ ====================
    # Если уникальных значений <= one_hot_max_size, применяется One-hot split.
    # Иначе используется TS-split (Target Statistics) с ordered-философией.
    'one_hot_max_size': 2,

    # Генерация комбинаций признаков «на лету».
    # 1 = только одиночные признаки. 2 и более добавляет комбинации,
    # что может улучшить качество, но сильно замедляет обучение.
    'max_ctr_complexity': 1,

    # ==================== БОРЬБА С ПЕРЕОБУЧЕНИЕМ ====================
    # Тип бустинга: 'Plain' (по умолчанию, быстрее) или 'Ordered'
    # (максимально «честный», без target leakage даже в градиентах,
    # но очень медленный; рекомендуется только для выборок < 10k объектов или на GPU).
    'boosting_type': 'Plain',

    # Overfitting Detector (аналог early stopping)
    'od_type': 'Iter',               # Тип детектора: 'Iter' (классический)
      или 'IncToDec' (статистический тест)
    'od_wait': 50,                   # Количество итераций без улучшения метрики
      до принудительной остановки
```

```
    # ==================== ПРОПУЩЕННЫЕ ЗНАЧЕНИЯ ====================
    # Куда отправлять NaN при сплите: 'Min', 'Max' или 'Forbidden' (запретить сплиты
      по NaN)
    'nan_mode': 'Min',
}

# ==================== 3. ОБУЧЕНИЕ МОДЕЛИ ====================
model = cb.CatBoost(params)

model.fit(
    train_pool,
    eval_set=val_pool,               # Валидационная выборка для отслеживания метрик
      и od_wait
    # early_stopping_rounds=50       # Альтернативный (синонимичный) способ
      задать od_wait
)

# ==================== 4. ПРЕДСКАЗАНИЯ И АНАЛИЗ ====================
# «Вероятности» (для бинарной классификации - вероятность принадлежности к классу 1,
  после сигмоиды)
y_prob = model.predict(X_val, prediction_type='Probability')

# «Сырые» скоры (чистая сумма весов листьев, до применения функции активации)
y_raw = model.predict(X_val, prediction_type='RawFormulaVal')

# Анализ модели: встроенная оценка важности признаков.
# type='PredictionValuesChange' (по умолчанию, как часто меняет предсказание)
# или 'LossFunctionChange' (насколько падает лосс при перемешивании признака)
feature_importance = model.get_feature_importance(prettified=True)
# print(feature_importance.head())
```

4.9.1. Итоги «практики»

Практический инструментарий CatBoost покрывает многие повседневных нужд разработчика ML-моделей, при этом некоторые возможности уникальны и заслуживают отдельного внимания. Поддержка текстов, эмбеддингов и графов прямо «из коробки» позволяет быстро проверить гипотезу, не прибегая к внешним инструментам — это экономит время на старте, хотя для максимального качества часто требуется ручная предобработка. Генерация комбинаций категориальных признаков на лету может улучшить качество модели, но увеличивает время обучения, поэтому параметр `max_ctr_complexity` стоит настраивать аккуратно.

CatBoost также предоставляет встроенные средства интерпретации — важность признаков и анализ взаимодействий — без необходимости писать дополнительный код.

Отсутствие прямого метода `trees_to_dataframe`, как в LightGBM, компенсируется возможностью экспорта модели в JSON и последующего парсинга, хотя это требует чуть больше усилий.

С точки зрения скорости, CatBoost на CPU обычно проигрывает LightGBM, особенно при большом объёме данных, но на GPU разрыв сокращается, а ordered-режим на GPU может быть вполне конкурентоспособен. Основной рекомендацией остаётся: использовать CatBoost, когда в данных много категориальных признаков высокого качества, и по возможности обучать на GPU. Если же данные преимущественно числовые, а требования к скорости высоки, разумнее присмотреться к LightGBM или XGBoost.

4.10. ИТОГИ CATBOOST

CatBoost стал эталоном «честного» градиентного бустинга, созданным для решения главной проблемы классических алгоритмов — **prediction shift** (смещения предсказаний из-за утечки таргета). Библиотека жертвует частью скорости обучения ради радикального повышения обобщающей способности модели, особенно на данных с категориальными признаками.

Ключевые столпы алгоритма CatBoost

1. **Ordered-философия.** По умолчанию CatBoost использует случайные перестановки объектов. При кодировании признаков текущий объект использует только данные, предшествующие ему в выбранной перестановке, поэтому таргет самого текущего объекта не участвует в его кодировании. Исходный порядок данных используется только при has_time=True. В режиме Ordered аналогичная идея применяется при вычислении градиентов.
2. **Oblivious Decision Trees (ODT).** Использование симметричных («ленивых») деревьев, где на одном уровне глубины применяется одинаковый сплит. Это не только ускоряет инференс, но и служит дополнительной регуляризацией, балансируя вычислительные затраты на «честное» обучение.
3. **Скоринг через схожесть с антиградиентом.** Вместо прагматичной максимизации локального Gain, CatBoost выбирает сплиты, чей вектор предсказаний максимально совпадает с направлением антиградиента. Это делает процесс обучения более стабильным и согласованным.
4. **Продвинутая работа с категориями «из коробки».** Нативная поддержка через `One-hot split` (для признаков с малым количеством категорий) и математически обоснованный `TS-split` (Partition split) для большого количества категорий. Плюс уникальная фича — автоматическая генерация комбинаций категориальных признаков «на лету».

5. SketchBoost — градиентный бустинг для multiclass-/multilabel-/multiregression-задач

Полезные материалы

- Оригинальная статья SketchBoost: Fast Gradient Boosted Decision Tree for Multioutput Problems
- Библиотека (GitHub) PyBoost, в основе которой лежит SketchBoost
- Туториалы по PyBoost

Сегодня все основные библиотеки градиентного бустинга — XGBoost, LightGBM, CatBoost — умеют работать с многомерными ответами. Для multiclass они строят multivariate trees с d-мерными весами в листьях (multivariate trees). CatBoost также поддерживает multilabel и multiregression. Однако с ростом размерности d расчёт Gain в таких деревьях становится всё более затратным, и стандартные реализации могут замедляться.

SketchBoost, реализованный в библиотеке PyBoost, целенаправленно решает именно эту проблему. Он тоже использует multivariate trees, но применяет скетчинг — аппроксимацию матрицы градиентов матрицей меньшей размерности. Это радикально снижает вычислительную сложность и делает обучение быстрым даже при очень большом числе таргетов. На multilabel-задачах SketchBoost часто выигрывает у XGBoost, LightGBM и CatBoost и по скорости, и по качеству.

В этом разделе мы пройдём по следующему плану.

1. Типы multitarget-задач. Определим multiclass, multilabel и multiregression и покажем, что все они сводятся к предсказанию d-мерного вектора.
2. Multivariate Trees — общий подход. Обобщим формулы оптимальных весов и Gain на d-мерный случай и увидим, почему прямой расчёт требует в d раз больше операций.
3. Ускорение через скетчинг. Главная инновация SketchBoost: аппроксимацию матрицы градиентов с помощью Random Sampling, Random Projection и других схем.
4. Пример использования PyBoost. Покажем, как настроить модель, выбрать метод скетчинга и ключевые гиперпараметры.

Теперь перейдём к типам задач, которые решает SketchBoost.

5.1. ТИПЫ MULTITARGET-ЗАДАЧ

- Multiclass — классификация объектов на один из $d > 2$ классов (например, классификация цифр MNIST).
- Multilabel — классификация объектов на один или несколько классов из $d > 2$ возможных (например, проставить теги на картинки).
- Multiregression — для каждого объекта нужно предсказать d-значный ($d > 1$) вектор вещественных ответов (например, координаты на плоскости).

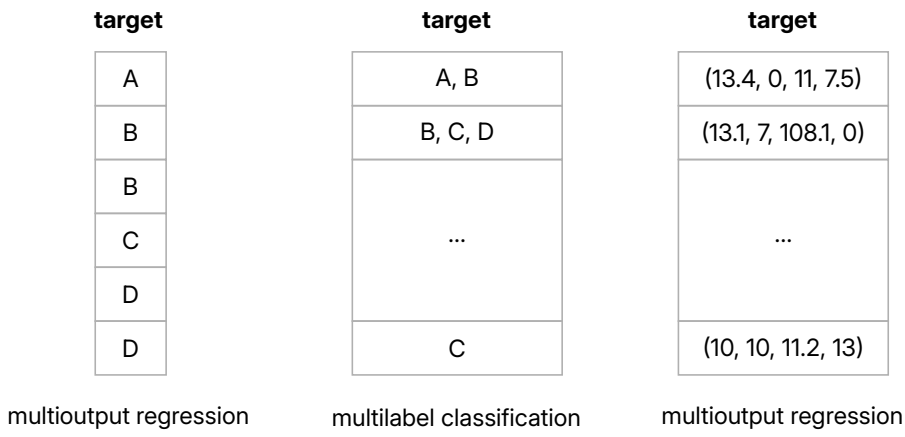


Рис. 31. multitarget-задачи

Ниже покажем, что эти задачи эквивалентны. С точки зрения требований от алгоритма он должен предсказывать d-значные векторы весов в листьях вместо скаляров. Для этого достаточно привести алгоритм, приводящий d-мерный вектор весов в ответ нужного формата.

- Для задачи multiclass d-мерный вектор — это скор, который получил каждый класс (чем больше, тем выше вероятность класса). Например, можно присвоить объекту класс с максимальным скором, но скоры каждого класса могут быть распределены по-разному (скор = 10 для одного класса может быть достаточным для одного и недостаточным для другого).
- Для задачи multilabel можно настроить пороги, так, что мы «добавляем» класс к предсказываемым, если соответствующий ему скор больше порога.
- Для задачи multiregression такой вектор уже можно считать ответом.

Нам неважно, какую из этих задач решать, — за это будет отвечать выбранный под задачу лосс. Задача алгоритма — уметь предсказывать d-мерные векторы весов в листьях.

5.2. КАК РЕШАЮТ ЭТУ ЗАДАЧУ ДРУГИЕ БИБЛИОТЕКИ

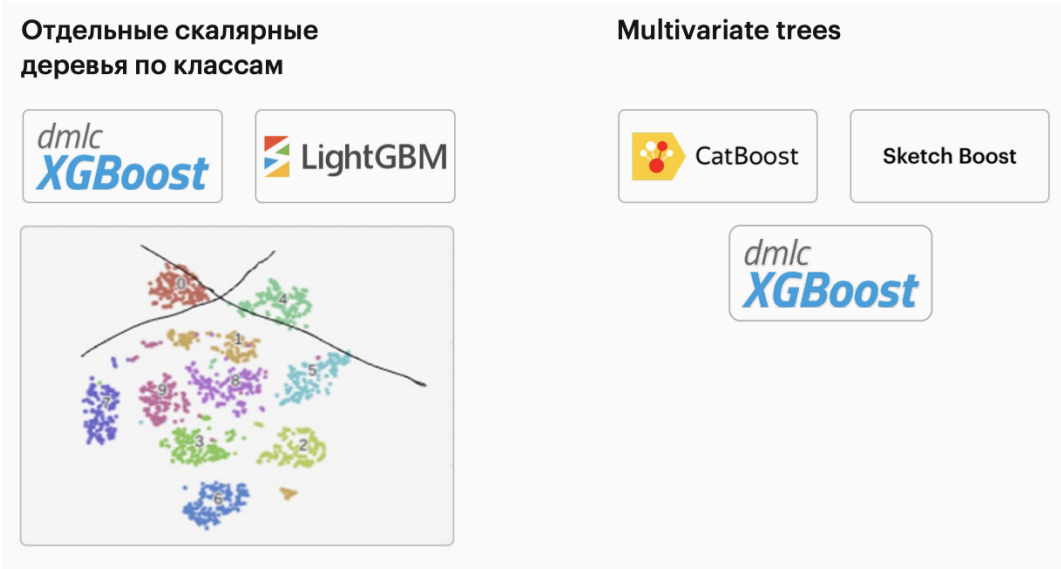


Рис. 32. Различные библиотеки и их подходы к решению Multitarget задач

XGBoost и LightGBM изначально решали задачу методом One-vs-rest, решая d задач бинарной классификации / одномерной регрессии. CatBoost же сразу строит деревья, которые умеют предсказывать d-мерные веса.

В обоих подходах результат — это d-мерный вектор предсказаний для каждого объекта,

- однако в первом обучение проходит по одной модели градиентного бустинга на каждую отдельную координату с «одномерным» лоссом;
- во втором одна модель с «d-мерным» лоссом сразу учится предсказывать d-мерный вектор ответов.

Очевидно, второй подход выигрывает, по памяти и интуитивно должен работать быстрее первого — всё зависит от реализации этой многомерности. Тем не менее на данный момент LightGBM и XGBoost уже перешли к работе с d-мерными весами в рамках одного дерева.

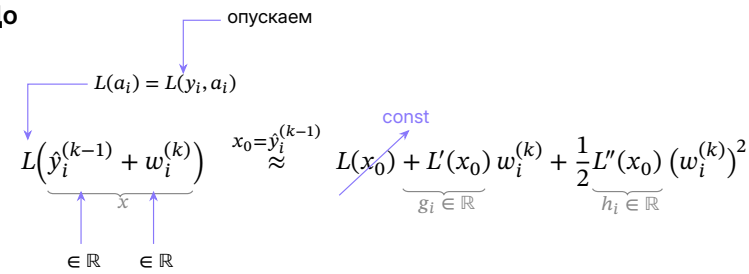
SketchBoost использует второй подход и работает поверх «классического» алгоритма построения дерева (depthwise или lossguide) — вычисления гистограмм, поиска оптимального сплита по Gain. Поэтому теперь нам необходимо разобраться, как классический алгоритм может быть раширен на d-мерный случай.

5.3. MULTIVARIATE TREES — ОБЩИЙ ПОДХОД

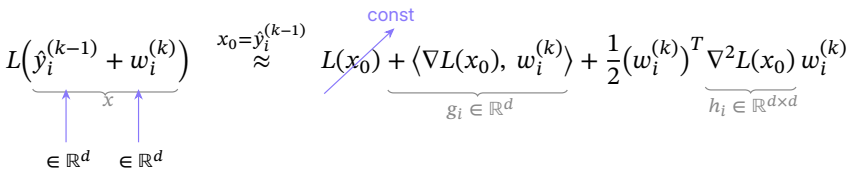
Разберём, как справиться с предсказанием d-мерного вектора без строительства d моделей (как в One vs. Rest подходе). Вспомним про градиентный бустинг в общем случае — мы выводили формулу оптимальных весов через разложение нашего лосса в ряд Тейлора до второго члена. Сделаем то же самое, но только теперь наш вектор весов $\vec{w}$ будет d-мерным. Посмотрим разницу на рисунках.

1.

До



После



Если лосс разделяем по размерностям таргета, то гессиан будет диагональный. Обычно так и бывает, но, если нет, его всё равно предполагают диагональным, чтобы в будущем не связываться с обращением матрицы.

$$g_i = \begin{pmatrix} g_i^1 \\ \vdots \\ g_i^d \end{pmatrix} \qquad H_i = \begin{pmatrix} h_i^1 & & 0 \\ & \ddots & \\ 0 & & h_i^d \end{pmatrix}$$

Все разобранные в курсе стандартные функции потерь являются разделимыми по размерностям таргета. Например, в задаче регрессии для каждого объекта необходимо предсказать вектор R^2. Если использовать MSE в качестве функции потерь, то лосс для объекта — сумма лоссов для каждой размерности, то есть функция потерь разделима. Таким образом, если функция потерь разделима, то её градиенты и гессианы также будут разделимы и могут быть представлены как сумма градиентов или сумма гессианов по каждой размерности.

2. Теперь перейдём к формуле оптимальных весов. Она точно такая же, но только добавляется индекс s, отвечающий за соответствующий таргет.

До

$$w_j^* = -\eta \frac{\sum_{i \in R_j} g_i}{\lambda + \sum_{i \in R_j} h_i}$$

После

$$w_j^{s*} = -\eta \frac{\sum_{i \in R_j} g_i^s}{\lambda + \sum_{i \in R_j} h_i^s}$$

3. Теперь перейдём к формуле Gain. На каждой картинке сверху она дана в привычной форме с суммами, а ниже — в матричной для удобства записи с d-мерным таргетом.

До

$$Gain \sim \frac{\left(\sum_{i \in R_L} g_i\right)^2}{\left(\sum_{i \in R_L} h_i\right) + \lambda} + \frac{\left(\sum_{i \in R_R} g_i\right)^2}{\left(\sum_{i \in R_R} h_i\right) + \lambda}$$

$$\sim S(R_L) + S(R_R)$$

$$S(R) = \frac{\langle \vec{g}, \vec{i}_R \rangle^2}{\langle \vec{h}, \vec{i}_R \rangle + \lambda}, \qquad \vec{i}_R = \begin{pmatrix} I_{[x_1 \in R]} \\ I_{[x_2 \in R]} \\ \vdots \\ I_{[x_n \in R]} \end{pmatrix}$$

После

$$Gain \sim \sum_{s=1}^{d} \frac{\left(\sum_{i \in R_L} g_i^s\right)^2}{n_L + \lambda} + \frac{\left(\sum_{i \in R_R} g_i^s\right)^2}{n_R + \lambda}$$

$$\sim \sum_{s=1}^{d} S^s(R_L) + S^s(R_R)$$

$$S(R) = \sum_{s=1}^{d} S^s(R) = \frac{\left\| G_{i_R}^T \right\|^2}{|R| + \lambda}, \qquad G = \begin{pmatrix} g_1^1 & g_2^1 & \cdots & g_n^1 \\ g_1^1 & g_2^2 & \cdots & g_n^d \\ \vdots & \vdots & \ddots & \vdots \\ g_1^1 & g_2^2 & \cdots & g_n^d \end{pmatrix}$$

Как видим, чтобы ускорить расчёт Gain, «гессианы» заменяются единицами (n_L и n_R — количество объектов в левом и правом листе), однако для вычисления финальных весов в листьях они уже используются, как показано на рисунке выше (пункт 2). То есть структура дерева определяется только на основе градиентов, а оптимальные веса для точности используют «гессианы».

В общем случае построение Multivariate Tree требует вычислить произведение матрицы G^T на вектор i_R, что делает расчёт Gain в d раз сложнее, чем в случае скалярного таргета. Это может существенно замедлить работу алгоритма, когда d достаточно велико. Аналогичные операции могли бы проводиться и с гессианом, у которого размерность $n \times d \times d$ (то есть сложность растёт в d^2 раз), но от этого отказались в угоду производительности.

5.3.1. Итог Multivariate Trees

Multivariate Trees позволяют одному дереву предсказывать сразу d-мерный вектор ответов, избавляя от необходимости обучать d отдельных моделей в стиле one-vs-rest. Формулы оптимальных весов и Gain обобщаются естественным образом: у каждого таргета появляется свой индекс s, отвечающий за свой класс / регрессионную метку, а суммы градиентов и гессианов становятся векторами и матрицами.

Чтобы не платить высокую цену за работу с гессианами размерности $d \times d$, Multivariate Trees при поиске сплитов заменяет их единицами — это сводит знаменатель Gain к простому количеству объектов в листе, а настоящие гессианы используются только для точного вычисления весов в уже готовых листьях.

5.4. АППРОКСИМАЦИЯ МАТРИЦ

Чтобы дополнительно решить проблему количества вычислений, авторы SketchBoost предлагают использовать скетч-матрицу G — аппроксимацию матрицей размерности $(n, d_{small} \ll d)$. Скетч-матрицу предлагается посчитать один раз перед построением каждого дерева, после чего Gain будет считаться быстрее.

Рассмотрим популярные схемы скетчинга, которые зарекомендовали себя на практике.

5.4.1. Random Sampling

Семплируем из матрицы G d_{small} колонок с вероятностями, которые пропорциональны их нормам. Идея в том, чтобы выбрать d_{small} размерностей, по которым ошибка будет максимальна по всем объектам обучающей выборки. Выбирать колонки с наибольшими ошибками неверно (см. статью), поэтому был предложен семплинг.

$$\hat{G} = \begin{pmatrix} | & | & & | \\ \hat{g}_{i_1} & \hat{g}_{i_2} & \cdots & \hat{g}_{i_{d_{small}}} \\ | & | & & | \end{pmatrix}$$

$$\hat{g}_{i_k} = g_{i_k} \cdot C_{i_k} \quad \text{норм. конст.}$$

Рис. 33. Схема Random sampling

Минус такого подхода в том, что мы отказываемся от большой части данных, удаляя $d - d_{small}$ колонок из матрицы G.

5.4.2. Random Projection

В этой схеме нужно умножить G на случайную матрицу из нормальных величин размерности (d, d_{small}).

$$\hat{G} = G \cdot \Pi$$

$$\Pi^{d \times d_{small}} \approx \mathcal{N}\left(0, \frac{1}{d_{small}}\right)$$

Рис. 34. Схема Random projection

Как ни странно, такой метод даёт довольно неплохую аппроксимацию матрицы G (мы используем все данные матрицы).

Эффективность схем варьируется от задачи к задаче, но Random Projection чаще даёт лучшее качество. Из экспериментов также видно, что реализация скетчинга на GPU в большинстве случаев работает быстрее xgboost, LightGBM и CatBoost в мультитаргет-задачах (подробнее см. в статье).

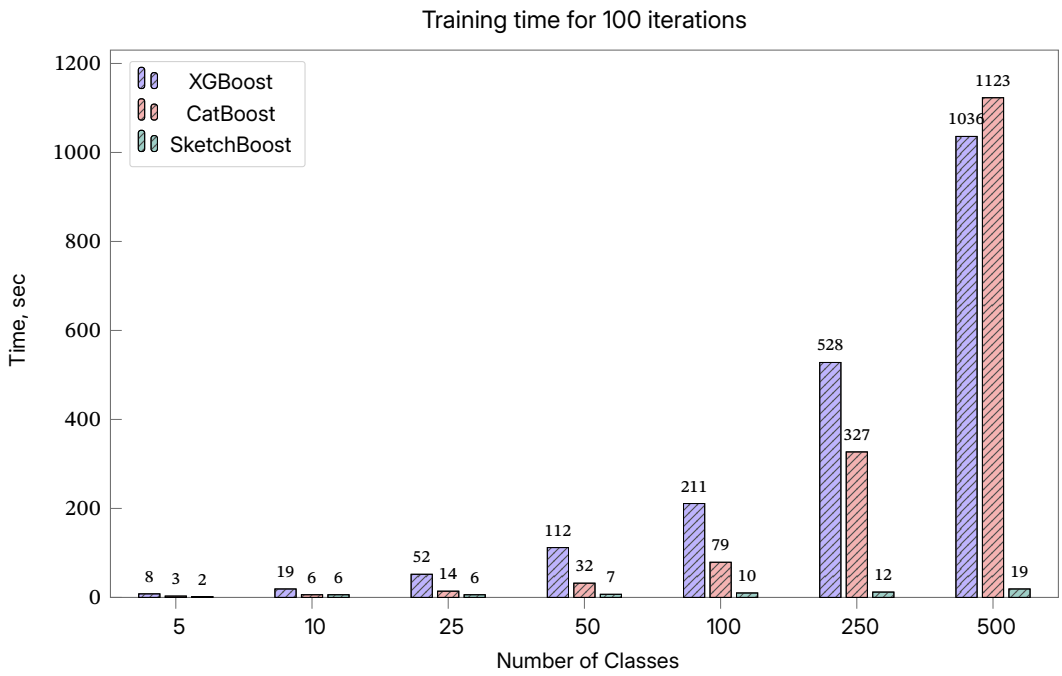


Рис. 35. Сравнение скорости обучения библиотек GB на multitarget задачах. Источник: https://arxiv.org/abs/2211.12858

Как видно на графике, удаётся многократно снизить вычислительную сложность для матрицы градиентов, что в итоге приводит к значимому приросту скорости. Выигрыш тем больше, чем классов или регрессионных меток модель должна предсказать. Причём разрыв растёт экспоненциально.

5.4.3. Итоги аппроксимации

Скетчинг матрицы градиентов — ключевая инновация SketchBoost, превращающая multivariate trees из потенциально медленных в практически быстрые. Вместо работы с полной матрицей G размера $n \times d$ алгоритм строит её сжатое представление размером n × d_{small}, где $d_{small} \ll d$. Это уменьшает сложность расчёта Gain с $O(d)$ до $O(d_{small})$ и даёт ускорение, пропорциональное степени сжатия.

Из рассмотренных схем на практике чаще выигрывает **Random Projection**: умножая G на случайную матрицу, он сохраняет информацию обо всех исходных размерностях. **Random Sampling** проще, но безвозвратно отбрасывает часть колонок, что может ухудшить качество, если отброшенные таргеты были информативны.

Выбор схемы и параметра d_{small} — это компромисс: чем меньше d_{small}, тем быстрее обучение, но выше риск потерять важные направления градиента.

Как показывают эксперименты, выигрыш от скетчинга растёт вместе с размерностью d: чем больше классов или регрессионных меток предсказывает модель, тем заметнее преимущество SketchBoost над XGBoost, LightGBM и CatBoost.

5.5. ПРИМЕР ИСПОЛЬЗОВАНИЯ PYBOOST

Теперь рассмотрим, как задаются основные гиперпараметры в PyBoost. Данный скрипт взят из официального примера использования.

```
from py_boost import GradientBoosting

# ==================== ВЫБОР МЕТОДА СКЕТЧИНГА ====================
# Скетчинг — это аппроксимация матрицы градиентов G размерности (n × d)
# матрицей меньшей размерности (n × d_small), где d_small << d.
# Это ускоряет расчёт Gain в d/d_small раз.

# RandomProjectionSketch: умножаем G на случайную матрицу из нормальных величин.
# Использует все данные, даёт хорошую аппроксимацию. Параметр — размерность скетча.
sketch = RandomProjectionSketch(1)

# RandomSamplingSketch: семплируем d_small колонок G с вероятностями,
# пропорциональными их нормам. Выбираем размерности с наибольшей ошибкой.
# Параметр — количество выбираемых колонок.
# sketch = RandomSamplingSketch(10)

# TopOutputsSketch: выбираем d_small размерностей с наибольшими градиентами.
# Авторы говорят, что это плохо работает
# sketch = TopOutputsSketch(10)

# SVDSketch: используем сингулярное разложение (SVD) для аппроксимации G.
# Параметр n_components — количество сингулярных чисел/векторов.
# sketch = SVDSketch(n_components=1)

# ==================== НАСТРОЙКА МОДЕЛИ ====================
model = GradientBoosting(
    'crossentropy',           # Функция потерь (для multiclass/multilabel)
    # ==================== ОСНОВНЫЕ ПАРАМЕТРЫ ====================
    ntrees=10000,              # Максимальное количество деревьев (критерий
      останова ансамбля)
    lr=0.03,                   # Learning rate
    verbose=100,               # Вывод метрик каждые 100 итераций
    es=300,                    # Early stopping: остановка, если метрика не растёт
      300 итераций

    # ==================== РЕГУЛЯРИЗАЦИЯ ====================
```

```
    lambda_l2=1,               # L2-регуляризация на веса листьев (аналог λ в XGBoost)
    # Количество шагов градиентного спуска для расчёта весов в листьях.
    # Аналог leaf_estimation_iterations в CatBoost.
    # gd_steps=1 – аналитическое решение (быстро).
    # gd_steps>1 – итеративное уточнение весов (точнее, но медленнее).
    gd_steps=1,

    # ==================== СЕМПЛИРОВАНИЕ ====================
    subsample=1,               # Доля объектов для каждого дерева (1.0 = все объекты)
    colsample=1,               # Доля признаков для каждого дерева (1.0 = все признаки)

    # ==================== СТРУКТУРА ДЕРЕВЬЕВ ====================
    min_data_in_leaf=10,       # Минимальное количество объектов в листе
      (критерий останова)
    max_bin=256,               # Количество бинов для гистограмм (В
      в histogram-based алгоритме)
    max_depth=6,               # Максимальная глубина дерева

    # ==================== ГЕССИАНЫ ====================
    # use_hess=False – заменяем гессианы на единицы (h_i = 1).
    # Тогда сумма гессианов в листе = количеству объектов (n_L или n_R).
    # Это упрощение ускоряет вычисления в multivariate случае,
    # но немного теряет в точности.
    use_hess=False,

    # ==================== СКЕТЧИНГ ====================
    # Ключевой параметр: метод скетчинга для ускорения расчёта Gain.
    # Без скетчинга сложность была бы O(d), со скетчингом – O(d_small).
    multioutput_sketch=sketch,

    debug=False                # Режим отладки (при True много логгов печается)
)

# ==================== ОБУЧЕНИЕ МОДЕЛИ ====================
# fit принимает обучающую выборку (X, y) и опционально eval_sets для early stopping.
# eval_sets – список словарей с ключами 'X' и 'y' для валидационных выборок.
model.fit(X, y, eval_sets=[{'X': X_test, 'y': y_test}])

# ==================== ПРЕДСКАЗАНИЯ ====================
# predict возвращает предсказания для тестовой выборки.
# Для multiclass – вероятности классов (после softmax).
# Для multilabel – вероятности принадлежности к каждому классу.
# Для multiregression – предсказанные значения.
y_pred = model.predict(X_test)
```

5.6. ИТОГИ SKETCHBOOST

SketchBoost — это специализированное решение, созданное для того, чтобы сделать градиентный бустинг быстрым и эффективным там, где классические библиотеки начинают справляться хуже: в задачах с множественными таргетами (multiclass, multilabel, multiregression).

Ключевые столпы алгоритма SketchBoost:

1. **Multivariate Trees (Единое дерево для всех выходов).** Вместо обучения d независимых моделей по схеме One-vs-Rest, алгоритм строит одно дерево, которое предсказывает d-мерный вектор весов в каждом листе. Это позволяет модели естественным образом учитывать корреляции между разными таргетами.
2. **Скетчинг матрицы градиентов (Главная инновация).** Чтобы избежать d-кратного замедления при расчёте Gain, алгоритм аппроксимирует полную матрицу градиентов размерности $(n \times d)$ матрицей гораздо меньшей размерности $(n \times d_{small})$, где $dsmall \ll d$.
3. **Эффективные схемы аппроксимации.** На практике метод **Random Projection** (умножение на случайную матрицу) чаще всего показывает лучший баланс скорости и качества, так как использует информацию обо всех размерностях, в отличие от Random Sampling, который просто отбрасывает часть колонок.
4. **Упрощение гессианов.** Для ещё большего ускорения при поиске сплитов гессианы заменяются на единицы ($h_i = 1$), что сводит знаменатель в формуле Gain к простому количеству объектов в листе. Настоящие гессианы используются только на финальном этапе для точного расчёта весов в листьях.

6. Yggdrasil Decision Forests (YDF): унифицированный фреймворк от Google

Полезные материалы

- Оригинальная статья Yggdrasil Decision Forests: A Fast and Extensible Decision Forests Library
- Официальный GitHub
- Документация

На первый взгляд может показаться, что YDF — это просто «очередная библиотека от Google», которая не предлагает ничего нового по сравнению с XGBoost, LightGBM или CatBoost. И в этом есть доля правды: алгоритмической магии (вроде ordered-философии CatBoost или скетчинга SketchBoost) здесь нет. YDF использует классический histogram-based-подход, знакомый нам по предыдущим разделам.

Google создал Yggdrasil Decision Forests (YDF) не для того, чтобы переизобрести математику бустинга, а чтобы решить проблемы экосистемы, унификации и масштабирования, с которыми сталкиваются крупные компании. YDF — это движок, который лежит в основе TensorFlow Decision Forests (TF-DF), но он написан на чистом C++ и имеет независимые Python-биндинги.

Более того, YDF честно заимствует лучшие практики у конкурентов: GOSS (из LightGBM), DART (из XGBoost), Focal Loss и некоторые нестандартные функции потерь. Это прагматичный подход — не изобретать велосипеды, а собрать проверенные решения в одном месте.

Однако у YDF есть и собственные уникальные наработки, которые отличают его от остальных библиотек.

1. Случайный поиск сплитов для категорий

В то время как LightGBM и CatBoost используют детерминированные алгоритмы (сортировка по Target Statistics с последующим перебором последовательных подмножеств), YDF предлагает альтернативу — **случайный поиск** сплитов вида $f_{cat} \in S$, где S — случайное подмножество категорий.

Этот подход, вдохновлённый статьей Breiman "Random Forest"(2001), аппроксимирует оптимальное разбиение CART, перебирая не все возможные подмножества категорий, а несколько случайных вариантов разбиения категорий на две группы.

Когда это полезно:

- при работе с признаками **очень высокой размерности** (тысячи уникальных категорий);
- когда детерминированный поиск слишком медленный;
- как защита от переобучения на маленьких датасетах.

2. Честные деревья (Honest Trees)

Хотя это концептуально похоже на ordered-философию CatBoost, Honest Trees в YDF решают другую задачу. Они разделяют обучающие примеры на две непересекающиеся группы:

- одна группа используется для **вывода структуры дерева** (выбора сплитов);
- другая группа — для **расчёта значений в листьях**.

Это регуляризация, описанная в статье "Generalized Random Forests"(Athey et al., 2016), которая жертвует частью данных ради уменьшения смещения оценок.

Отличия от CatBoost:

- CatBoost использует ordered approach для борьбы с target leakage при кодировании категорий и расчёте градиентов;
- YDF Honest Trees разделяют данные для регуляризации структуры и значений.

На практике это может как улучшить, так и ухудшить качество модели — зависит от задачи.

3. Продвинутая обработка пропусков

В XGBoost и LightGBM пропущенные значения отправляются в «лучшую» ветку, которая выбирается в процессе обучения. YDF предлагает более гибкий подход:

- заполнение глобальным средним (для числовых) или модой (для категориальных) по всему датасету. Но это довольно стандартный ход на уровне предобработки данных;
- заполнение средним/модой, вычисленными только по объектам в текущем узле;
- случайное заполнение значениями из объектов в текущем узле (вдохновлено Random Survival Forests (Ishwaran H. et al., 2008)).

4. Selective Gradient Boosting (SELGB) для ранжирования

YDF реализует специализированный метод семплирования **SELGB** (Selective Gradient Boosting), доступный только для ranking-задач. В отличие от GOSS, который семплирует объекты по величине градиента, SELGB использует более сложную стратегию отбора, оптимизированную специально для метрик ранжирования (NDCG, MRR).

Подробнее писать не имеет смысла, так как ранжирование в курсе не затрагивается, но на будущее о возможности использования YDF на больших данных для ранжирования стоит помнить.

5. Нативное распределённое обучение

Это главная сила YDF. В отличие от других библиотек, где для обучения на больших данных приходится интегрироваться с Dask, Spark или Ray, YDF имеет встроенную поддержку распределённого обучения через Apache Beam.

Как это работает:

- обучение происходит в стиле Map-Reduce: данные разбиты на партиции, обучение происходит параллельно, а результаты агрегируются;
- работает на любых кластерных системах, поддерживающих Apache Beam (Google Dataflow, Apache Spark, Apache Flink);
- не требует переписывания и доработок кода — обучение на 100 ГБ данных выглядит так же, как на 1 ГБ.

6.1. ПРИМЕР ИСПОЛЬЗОВАНИЯ YDF

Теперь рассмотрим, как задаются основные гиперпараметры в YDF.

```
import ydf
import pandas as pd

# ==================== 1. ПОДГОТОВКА ДАННЫХ ====================
# YDF отлично работает напрямую с pandas DataFrame.
# Однако для максимального контроля (особенно над категориальными признаками
# и стратегией обработки пропусков) рекомендуется создать спецификацию данных.
dataspec = ydf.GenericDataSpec.from_dataframe(
    df_train,
    # Явное указание категориальных признаков помогает алгоритму
    # сразу применить нужную стратегию (например, RANDOM или CART)
    categorical_columns=['cat_feature_1', 'cat_feature_2']
)

# ==================== 2. НАСТРОЙКА МОДЕЛИ ====================
# YDF предоставляет единый API. Для бустинга используем GradientBoostedTreesLearner.
# (Для распределённого обучения на кластере
  используется DISTRIBUTED_GRADIENT_BOOSTED_TREES)
model = ydf.GradientBoostedTreesLearner(
    label="target",                  # Название целевой переменной

    # ==================== ОСНОВНЫЕ ПАРАМЕТРЫ ====================
```

```
    num_trees=500,                    # Количество деревьев (итераций бустинга)
    # Стратегия роста: "BEST_FIRST_GLOBAL" (аналог leaf-wise в LightGBM, часто даёт
      лучшее качество)
    # или "LOCAL" (аналог level-wise в XGBoost)
    growing_strategy="BEST_FIRST_GLOBAL",

    # ==================== РЕГУЛЯРИЗАЦИЯ ====================
    l2_regularization=1.0,            # L2-регуляризация на веса листьев
    min_examples=10,                  # Минимальное кол-во объектов в листе
      (аналог min_data_in_leaf)

    # ==================== УНИКАЛЬНЫЕ ФИЧИ YDF ====================
    # 1. Обработка категорий: случайный поиск сплитов.
    # Полезно при тысячах уникальных категорий или для защиты от переобучения.
    categorical_algorithm="RANDOM",

    # 2. «Честные деревья» (Honest Trees): разделение данных
    # для построения структуры дерева и расчёта значений в листьях.
    # Снижает смещение (bias) оценок, жертвуя частью данных.
    honest=True,

    # 3. Продвинутая обработка пропусков: заполнение средним/модой
    # именно внутри текущего узла дерева, а не глобально по всему датасету.
    missing_value_policy="LOCAL_IMPUTATION",

    # 4. Контроль биннинга: принудительно разбивать ВСЕ числовые признаки на бины.
    # Полезно для ускорения и дополнительной регуляризации.
    force_numerical_discretization=True,
    max_unique_values_for_discretized_numerical=256, # Аналог max_bin

    # ==================== СЕМПЛИРОВАНИЕ ====================
    # YDF также поддерживает GOSS (из LightGBM) и DART (из XGBoost),
    # но здесь показан стандартный случайный семплинг
    subsample=0.8,                    # Доля объектов для каждого дерева
    feature_sample_rate=0.8,          # Доля признаков (аналог colsample_bytree)

    # ==================== РАННИЙ ОСТАНОВ ====================
    early_stopping_num_trees_look_ahead=30, # Аналог early_stopping_rounds
)

# ==================== 3. ОБУЧЕНИЕ МОДЕЛИ ====================
# Обучение происходит быстро благодаря оптимизированному ядру C++.
print("Начинаем обучение...")
model.fit(
    df_train,
    dataspec=dataspec,
    validation_dataset=df_val # Используется для early stopping
)

# ==================== 4. ПРЕДСКАЗАНИЯ И АНАЛИЗ ====================
# YDF позволяет явно и безопасно контролировать формат вывода предсказаний.

# Вероятности (по умолчанию apply_link_function=True, применяется сигмоида/softmax)
y_prob = model.predict(df_test, apply_link_function=True)

# «Сырые» скоры (логиты, до применения функции активации)
```

```
y_raw = model.predict(df_test, apply_link_function=False)

# ==================== ИНТЕРПРЕТАЦИЯ МОДЕЛИ ====================
# YDF имеет мощный встроенный инспектор для анализа обученной модели
inspector = model.make_inspector()

# 1. Важность признаков (поддерживает несколько методов оценки)
# "MEAN_DECREASE_IN_ACCURACY" (перестановочная важность)
# или "NUM_AS_ROOT" (как часто признак используется в корне дерева)
importances = inspector.variable_importances().get("MEAN_DECREASE_IN_ACCURACY", [])
# print(importances)

# 2. Визуализация структуры конкретного дерева в читаемом текстовом виде
# Отлично подходит для отладки и понимания логики модели
# print(inspector.extract_tree(tree_idx=0))
```

6.2. ИТОГИ YDF

Yggdrasil Decision Forests (YDF) — это не про алгоритмические революции, а про **инженерную надёжность, унификацию и масштабируемость**. Google создал эту библиотеку не для того, чтобы переизобрести математику бустинга, а чтобы решить реальные проблемы промышленных пайплайнов: скорость инференса, простоту поддержки и работу с петабайтами данных.

ВАЖНО

Автор решил включить данную библиотеку в рассмотрение, так как встречал её как часть ансамбля в некоторых топовых решениях в соревнованиях на Kaggle.

Ключевые особенности YDF

1. **Нативное распределённое обучение.** Главная «киллер-фича» библиотеки. Благодаря интеграции с Apache Beam, обучение масштабируется на кластеры (Spark, Dataflow, Flink) по принципу Map-Reduce без необходимости переписывать код или подключать тяжёлые обёртки вроде Dask.
2. **Унификация и прагматизм.** Единый API для обучения Random Forest, GBDT, CART и других алгоритмов. Библиотека честно заимствует лучшие практики конкурентов (GOSS из LightGBM, DART из XGBoost), не изобретая велосипеды.
3. **Уникальные эвристики.** В отличие от классики, YDF предлагает:
   - **случайный поиск сплитов** для категориальных признаков (спасение при высокой размерности);
   - **Honest Trees** (разделение данных на структуру и веса для снижения смещения);
   - **продвинутую обработку пропусков** (например, `LOCAL_IMPUTATION` внутри узла).
4. **Оптимизированное ядро C++.** YDF лежит в основе TensorFlow Decision Forests (TF-DF), обеспечивая молниеносный инференс и лёгкую интеграцию в продакшен-сервисы без зависимостей от Python.

INTERVIEW KIT

7. Сравнение библиотек

Посмотрим на сводные таблички с основными отличиями рассмотренных библиотек. Стоит отметить, что таблички ниже представлены на момент 23 июля 2026 года, поэтому библиотеки уже успели позаимствовать друг у друга некоторые особенности.

	XGBoost	LightGBM	Catboost	SketchBoost	YDF
Построение деревьев	Изначально по уровням (Level-wise/ Depth-wise). Потом добавили по листьям (Lossguide/ Leaf-wise)	По листьям по дефолту. Можно и по уровням	Ленивые деревья (ODT), можно по листьям и по уровням	По уровням	По листьям и по уровням
Построение расщеплений	Гистограммный подход	Гистограммный подход	Гистограммный подход	Гистограммный подход	Гистограммный подход
Критерий расщепления	Gain	Gain	Сходство с антиградиентом (eucledian/cosine)	Gain	Gain
Обработка пропусков	В лучшую ветку	В лучшую ветку	Min/max	В лучшую ветку	Заполняем средним в листе / глобальным средним / случайным из листа
Категории	One-hot- или partition-based-сплиты при включённой поддержке категориальных признаков	Нативные категориальные сплиты: категории сортируются по статистике градиентов и гессианов	One-hot для малой кардинальности и ordered target statistics для более сложных категорий	Аналогично LightGBM	Аналогично LightGBM
Фишки	DART RF Mode	Leafwise-рост, GOSS, EFB, нативные категориальные сплиты, DART, RF mode, линейные модели в листьях, CEGB	Ordered кодирование категорий Полный Ordered режим для градиентов и гессианов Работа с текстом из коробки RF Mode	Эффективный multitarget	Из коробки работает на кластере. Легко интегрируется с Tensorflow. DART RF Mode
Multiclass	One-vs-rest / Multivariate Trees (n моделей на n классов / сразу вектор предсказаний)	One-vs-rest/ Multivariate Trees	Multivariate Trees	Multivariate Trees	Multivariate Trees

	XGBoost	LightGBM	Catboost	SketchBoost	YDF
Важности признаков	Gain/Frequency	Gain/Frequency	Gain / Frequency / снижение метрики / функции потерь после перемешивания признака / Gain, взвешенный по Frequency / SHAP	Gain/Frequency	Gain / Frequency / снижение метрики после перемешивания признака / число использований признака в корне / насколько высоко в среднем признак в дереве
Device	cpu/gpu	cpu/gpu	cpu/gpu	gpu	cpu

Итоги

XGBoost

XGBoost стал первой по-настоящему масштабируемой реализацией градиентного бустинга и сформировал тот «классический» алгоритм, от которого отталкиваются все остальные. Его ключевой вклад — не новая математика, а инженерная оптимизация.

Что XGBoost добавляет к классическому бустингу

- Гистограммный подход к поиску сплитов: числовые признаки бьются на бины, что снижает потребление памяти с $O(F \cdot N)$ до $O(F \cdot B)$, где $B \ll N$.
- Трюк с вычитанием гистограмм: для «большой» дочерней вершины гистограмма не строится с нуля, а получается вычитанием гистограммы «маленькой» вершины из родительской. Это даёт почти двукратный выигрыш по скорости.
- Продуманная система регуляризации: L2 на веса листьев, L1 для разреженности, штраф за количество листьев (γ).
- Поддержка DART — dropout на уровне деревьев для борьбы с переспециализацией ансамбля.

Сильные стороны: отличная документация, стабильность, работает на CPU и GPU, хорош как baseline для любых табличных задач.

Ограничения: depthwise-рост по умолчанию менее эффективен по количеству, чем leafwise. Нативная работа с категориями появилась в XGBoost позже, чем в LightGBM и CatBoost. Современные версии поддерживают one-hot- и partition-based-сплиты, но требуют корректно передавать типы категориальных признаков.

Когда использовать: как надёжный baseline для табличных задач, когда важны зрелая экосистема, стабильность и совместимость с существующим пайплайном, когда инфраструктура уже построена вокруг XGBoost. LightGBM часто выигрывает по скорости на больших данных, но окончательный выбор библиотеки лучше подтверждать экспериментально.

LightGBM

LightGBM взял классический алгоритм XGBoost и адресно устранил его узкие места. Каждое улучшение бьёт в конкретный источник замедления или неэффективности:

Основные идеи и за счёт чего достигается ускорение

- **Leafwise (lossguide) рост деревьев** вместо depthwise: на каждом шаге разбивается один лист с максимальным Gain. Это даёт лучшую точность при том же числе листьев и адаптивную глубину ветвей. Цена leafwise-роста — более высокий риск переобучения, поэтому важно контролировать сложность дерева через num_leaves и min_data_in_leaf. При необходимости можно дополнительно ограничить глубину параметром max_depth.
- **GOSS** — умный семплинг объектов по величине градиента: «трудные» примеры сохраняются полностью, из «простых» берётся случайная подвыборка с нормировкой. Уменьшает эффективное число объектов N без заметной потери качества.
- **EFB** — объединение взаимоисключающих разреженных признаков в «бандлы». Уменьшает эффективное число признаков F, практически бесплатно на разреженных данных.
- **Нативные категориальные сплиты** — сортировка категорий по отношению сумм градиентов и гессианов с перебором последовательных подмножеств вместо полного перебора 2^K вариантов.

Сильные стороны: быстрая библиотека на CPU и GPU. Экономна по памяти из-за leafwise-роста. Богатая экосистема (кастомные метрики, сохранение деревьев в DataFrame, режим случайного леса, GBDT-PL, CEGB).

Ограничения: leafwise склонен к переобучению без тщательной настройки max_depth и min_data_in_leaf. С GOSS можно потерять обобщающую способность. EFB бесполезен на плотных числовых данных. Категориальные сплиты могут переобучаться на редких категориях, поэтому важно настраивать cat_smooth, cat_l2, min_data_per_group и другие параметры регуляризации.

Когда использовать: основной рабочий инструмент для большинства табличных задач. Особенно хорош на больших выборках, где важна скорость; когда в данных много разреженных или категориальных признаков.

Catboost

CatBoost вырос из задачи — устранить переобучение, вызванное утечкой целевой переменной (prediction shift). Отсюда — ordered-философия и два режима обучения.

Какие проблемы решает CatBoost и зачем нужен ordered-подход

- **Честное кодирование категорий (Ordered Target Statistics).** Вместо того чтобы кодировать категорию одним числом для всех объектов, CatBoost генерирует несколько случайных перестановок и вычисляет TS так, что объект «видит» только предшествующие ему в перестановке. Благодаря этому таргет текущего объекта не участвует в вычислении его target statistics, что снижает прямую утечку при кодировании категорий.
- **Борьба с prediction shift в градиентах (ordered-режим).** Даже при честных категориях градиенты считаются по предсказаниям, которые уже видели правильные ответы. Ordered-режим строит N параллельных моделей для каждой перестановки, чтобы градиент i-го объекта вычислялся на модели, обученной только на первых i – 1 объектах.

- **Симметричные (Oblivious) деревья** — один сплит на весь уровень глубины. Это ускоряет обучение и служит дополнительной регуляризацией.
- **Скоринг сплитов через сравнение с антиградиентом** — вместо максимизации Gain ищется сплит, чей вектор весов наиболее сонаправлен с направлением наискорейшего спуска. Это работает как дополнительная регуляризация.

Сильные стороны: лучшая работа с категориальными признаками «из коробки», автоматическая генерация комбинаций категорий, поддержка текстов, эмбеддингов и графов, меньше склонен к переобучению на малых выборках.

Ограничения: медленнее LightGBM на CPU; ordered-режим на CPU непрактичен на выборке продакшен-масштаба, документация местами запутанная, избыточен для чисто числовых данных.

Когда использовать: когда в данных много категориальных признаков с большим количеством категорий и их качественная обработка критична для результата, на малых выборках, где переобучение особенно опасно, для быстрого прототипирования с текстами и эмбеддингами без внешних инструментов, при возможности обучения на GPU.

SketchBoost (PyBoost)

SketchBoost — специализированное решение для multitarget-задач, где число выходов d измеряется десятками, сотнями или тысячами.

Ключевые идеи

- **Multivariate Trees** — каждое дерево предсказывает сразу d-мерный вектор весов, что позволяет модели учитывать корреляции между таргетами.
- **Скетчинг матрицы градиентов** — главная инновация. Матрица G размера $n \times d$ аппроксимируется матрицей $n \times d_{small}$, где $d_{small} \ll d$, что снижает сложность расчёта Gain с $O(d)$ до $O(d_{small})$.
- **Random Projection** как наиболее эффективная схема скетчинга на практике, так как использует информацию из всех исходных размерностей.

Сильные стороны: на multitarget-задачах с большим d часто быстрее и точнее XGBoost, LightGBM и CatBoost.

Ограничения: только GPU, маленькое сообщество и экосистема, для классических задач с одним таргетом избыточен, реализация не супербыстрая, так как написан на `Python` и для скорости использует `CuPy` и `Numba`.

Когда использовать: multiclass с сотнями классов, multilabel с большим числом меток, multiregression с многомерным выходом — везде, где стандартные библиотеки начинают замедляться из-за размерности d.

YDF (Yggdrasil Decision Forests)

YDF — это не новый алгоритм, а инженерный фреймворк от Google, который унифицирует обучение деревьев и бустинга под одной крышей.

Ключевые идеи

- **Единый API** для Random Forest, GBDT, CART и других древовидных методов.

- **Нативное распределённое обучение** через Apache Beam без необходимости интегрироваться с Dask, Spark или Ray.
- **Возможность использовать честные деревья (Honest Trees)** — разделение данных на построение структуры и расчёт весов в листьях как метод регуляризации.
- **Случайный поиск сплитов для категорий** — альтернатива детерминированному перебору, полезная при очень большом числе категорий.
- **Продвинутая обработка пропусков** — заполнение средним или модой локально в узле дерева.

Сильные стороны: нативное распределённое обучение для огромных данных, гибкая обработка пропусков, хорошая документация от Google, активно развивается. Эффективный инференс на CPU.

Ограничения: малое сообщество (потому что по большей части это внутренняя библиотека Google), нет GPU.

Когда использовать: когда данные измеряются терабайтами и нужно распределённое обучение без боли, когда нужен единый фреймворк и для Random Forest, и для GBDT в экосистеме TensorFlow. Это может пригодиться для построения ансамбля с нейросетями, написанными также на TensorFlow.